Meridian — the backend, line by line

Every line of the six backend Python files, explained against the Software Lab-1 practicals. Built from the files on disk, so the line numbers are the line numbers.

Files covered:

- **models.py** — Patient and Visit row objects — classes, Lab 4.
- **store.py** — CSV read / append / update / delete / backup — file handling, Lab 4.
- **registry.py** — Fuzzy search scoring and write validation — strings and dicts, Labs 2-3.
- **triage.py** — Set algebra over symptoms, severity bands, report — Labs 3 and 5.
- **analytics.py** — pandas profiling, risk bands, and the graph system — Labs 6 and 7.
- **app.py** — Flask routes and response shapes only — glue layer.

How to read the tag column: every explanation is tagged with the practical it comes from, so the viva answer to "where did you learn this?" is sitting beside the code. Lines that go beyond the 1–6 practicals are tagged **X:...** and each one has an explainer document in **docs/concepts/—that is the running rule of the project: nothing outside the syllabus without its own written explanation.**

Tag legend

- **L1** — Practical 1 — variables, operators, typecasting, if/elif
- **L2** — Practical 2 — strings, lists, tuples
- **L3** — Practical 3 — sets, dicts, comprehensions, sorted(key=)
- **L4** — Practical 4 — functions, files, os, classes
- **L5** — Practical 5 — math, formatted output, f-strings
- **L6** — Practical 6 — pandas
- **L7** — Practical 7 — matplotlib (graphs)
- **X:flask** — Beyond labs — documented in docs/concepts/flask.md
- **X:csv** — Beyond labs — documented in docs/concepts/csv-module.md
- **X:shutil** — Beyond labs — documented in docs/concepts/shutil-and-zipfile.md
- **X:json** — Beyond labs — documented in docs/concepts/json.md
- **X:dtypes** — Beyond labs — documented in docs/concepts/pandas-select-dtypes.md
- **X:nav** — Beyond labs — documented in docs/concepts/pandas-missing-values.md
- **std** — Python standard library used the labs could not avoid
- **txt** — Docstring or comment — the words themselves carry the meaning

`models.py` 158 lines — Patient and Visit row objects — classes, Lab 4.

1–8	`"""Data classes for Meridian records.`
	
	`Taught concepts: classes, __init__, methods, self, default arguments,`
	`static methods, list and dict handling.`
	
	`Rule enforced here: an unfilled field is the empty string "" and never`
	`"N/A", "unknown" or a made-up default. See spec section 4.2.`
	`"""`
		txt	Module docstring. States the two things this file must give you: classes (Lab 4) and the one rule the models enforce everywhere — a field nobody filled in is the empty string <code>""</code>, never “N/A” or a made-up default. Keeping that rule in one place is why CSV cells read consistently through the whole project.

9–10		txt	Blank separator lines — style only, no code.
11–12	`class Patient:` `    """One row of data/patients.csv."""`	L4	<code>class Patient:</code> opens a class (Practical 4). Its docstring says what it represents: one row of <code>data/patients.csv</code>. Every Patient object therefore mirrors exactly one line of a CSV file, which is what lets <code>to_row()</code> and <code>from_row()</code> below be so short.
13–13		txt	Blank line between the class header and its body.

14–18	`    FIELDS = [`
	`        "patient_id", "name", "age", "gender", "phone", "email",`
	`        "blood_group", "address_line", "area", "pincode", "city", "state",`
	`        "emergency_contact", "emergency_phone", "registered_on", "notes",`
	`    ]`
		L2	<code> Patient.FIELDS</code> is a class attribute — a plain list (Practical 2) of the sixteen column names in the true order they appear in the CSV. It is a class attribute rather than a per-object one so all patients share one list and the column order is defined exactly once; <code>to_row()</code> and <code>to_dict()</code> both walk this list.

19–19		txt	Blank line.

20–23	`    def __init__(self, patient_id, name, age="", gender="", phone="",`
	`                 email="", blood_group="", address_line="", area="",`
	`                 pincode="", city="", state="", emergency_contact="",`
	`                 emergency_phone="", registered_on="", notes=""):`
		L4	The <code>__init__</code> signature. Only <code>patient_id</code> and <code>name</code> are required parameters; the other fourteen come after = so they are default arguments (Practical 4), each defaulting to <code>""</code>. One normal patient record of 16 columns becomes a function call without having to pass empty strings by position.

24–24	`        """Create a patient. Only patient_id and name are required."""`	txt	Docstring for <code>__init__</code>, repeating the minimal-create rule: a patient whose address, blood group, everything is unknown is still a valid object.
25–25	`        self.patient_id = patient_id`	L4	<code>self.patient_id = patient_id</code> — stores the id (P-0001) on the object. <code>self</code> is the instance being built (Practical 4).
26–26	`        self.name = name`	L4	<code>self.name = name</code> — stores the name, which together with the id is the only required pair.
27–27	`        self.age = age`	L4	<code>self.age = age</code> — stored as a string even for a number; validation happens in registry.py, storage is typed by CSV anyway.

28–28	`        self.gender = gender`	L4	<code>self.gender = gender</code> — column 4.
29–29	`        self.phone = phone`	L4	<code>self.phone = phone</code> — column 5.
30–30	`        self.email = email`	L4	<code>self.email = email</code> — column 6.
31–31	`        self.blood_group = blood_group`	L4	<code>self.blood_group = blood_group</code> — column 7; may be "", which the search and the "no blood group" facet both rely on.
32–32	`        self.address_line = address_line`	L4	<code>self.address_line = address_line</code> — street-level address, column 8.
33–33	`        self.area = area`	L4	<code>self.area = area</code> — locality, column 9; searchable by fuzzy match.
34–34	`        self.pincode = pincode`	L4	<code>self.pincode = pincode</code> — column 10, validated as six digits at write time.
35–35	`        self.city = city`	L4	<code>self.city = city</code> — column 11.
36–36	`        self.state = state`	L4	<code>self.state = state</code> — column 12.
37–37	`        self.emergency_contact = emergency_contact`	L4	<code>self.emergency_contact = emergency_contact</code> — column 13; counted as a lower-weight search field.
38–38	`        self.emergency_phone = emergency_phone`	L4	<code>self.emergency_phone = emergency_phone</code> — column 14.
39–39	`        self.registered_on = registered_on`	L4	<code>self.registered_on = registered_on</code> — column 15.
40–40	`        self.notes = notes`	L4	<code>self.notes = notes</code> — column 16. Sixteen assignments, one per FIELDS entry, so the object and the CSV row can never drift apart in width.
41–41		txt	Blank line.
42–43	`    def to_row(self):` `        """Return values in FIELDS order, ready for csv.DictWriter."""`	L4	<code>to_row()</code> with its docstring. Its job is to produce the list form <code>csv.DictWriter</code> appends to the file (the CSV module itself is the csv-module.md explainer in docs/concepts).
44–44	`        return [getattr(self, field) for field in self.FIELDS]`	L3	A list comprehension (Labs 2–3): <code>[getattr(self, field) for field in self.FIELDS]</code>. <code>getattr(self, field)</code> looks the attribute up by its name, so the value order follows FIELDS order automatically. Add a column to FIELDS and both CSV direction and JSON direction pick it up with no other change.
45–45		txt	Blank line.
46–47	`    def to_dict(self):` `        """Return a plain dict for JSON responses."""`	L4	<code>to_dict()</code> and its docstring: the JSON-facing form sent to browsers by the API responses.
48–48	`        return {field: getattr(self, field) for field in self.FIELDS}`	L3	A dict comprehension (Practical 3): <code>{field: getattr(self, field) for field in self.FIELDS}</code>. Key and value both derive from the same loop variable, so the JSON keys use the CSV column names and cannot be spelt differently.
49–49		txt	Blank line.
50–51	`    def address(self):` `        """Return the address as one line, omitting anything unrecorded."""`	L2	<code>address()</code> puts a patient's address on one readable line for list views. The docstring promises it omits anything unrecorded.
52–53	`        parts = [self.address_line, self.area,` `                 f"{self.city} {self.pincode}".strip(), self.state]`	L2	<code>parts</code> is a list of the four fragments. Line 53 shows two string moves from Practical 2: an f-string (Lab 5 formatting) combining city and pincode, and <code>.strip()</code> removing the trailing space a missing pincode would leave.

54–54	`        return ", ".join(p for p in parts if p)`	L3	<code>", ".join(p for p in parts if p)</code> — a generator of the truthy parts joined with a comma. An empty segment cannot be “joined into nothing” and leave a double comma: a patient with no state simply does not get a trailing comma.
55–55		txt	Blank line.
56–57	`    def summary(self):` `        """Return a one-line description for list views."""`	L4	<code>summary()</code> returns a one-line description used by list views — the shortest honest description of a record.
58–58	`        parts = [self.patient_id, self.name]`	L3	<code>parts</code> starts with id and name, the two fields every row has.
59–60	`        if self.age:` `            parts.append(f"age {self.age}")`	L1	<code>if self.age:</code> — a truthiness guard (Practical 1): "" is falsy, so an unknown age is not announced. Inside, an f-string formats <code>age 34</code> (formatted output, Practical 5).
61–62	`        if self.city:` `            parts.append(self.city)`	L1	Same guard for <code>city</code>, then the parts are joined with <code>" - "</code>. Three <code>if</code> checks in a row are the whole method; no loops, no library calls.
63–63	`        return " - ".join(parts)`	txt	Blank line.
64–65	`    @staticmethod`	L4	<code>@staticmethod</code> (Practical 4): <code>from_row</code> needs no instance — it builds one. Marking it static keeps the call site honest: you do not need a patient to make a patient.
66–66	`    def from_row(row):`	txt	Docstring: builds a Patient object from a dict straight out of the CSV reader.
67–67	`        """Build a Patient from a dict read out of a CSV."""`	txt	Blank line (inside function).
68–68	`        return Patient(**{k: row.get(k, "") for k in Patient.FIELDS})`	L3	<code>Patient(**{k: row.get(k, "") for k in Patient.FIELDS})</code> — three ideas in one line: a dict comprehension (L3) that copies only FIELDS keys, <code>.get(k, "")</code> (L3) defaulting missing cells to <code>""</code> instead of <code>None</code>, and <code>**</code> keyword unpacking (L4) that turns the dict into the keyword arguments of <code>__init__</code>.
69–70		txt	Blank lines between the two classes.
71–71	`class Visit:`	L4	<code>class Visit:</code> with an added docstring note: visits are append-only, never rewritten — consultation history is never edited, which is why the store has no update path used for visits.
72–72	`    """One row of data/visits.csv. Append-only: never rewritten."""`	txt	Blank line.

73–80	`    FIELDS = [`
	`        "visit_id", "patient_id", "doctor_id", "hospital_id",`
	`        "scheduled_date", "scheduled_time", "checked_in_at", "duration_minutes",`
	`        "reason", "symptoms", "diagnosis", "severity",`
	`        "vitals_bp", "vitals_pulse", "vitals_temp", "vitals_spo2",`
	`        "vitals_weight", "vitals_height", "follow_up_days", "status", "notes",`
	`    ]`
		L2	<code>Visit.FIELDS</code> — the 21 columns of <code data-file="a">visits.csv</code>: ids and dates, reason, symptoms and diagnosis, four vitals columns, weight/height, follow-up, status, notes. A wider list than Patient's, same mechanism.

81–81		txt	Blank line.

82–87	`    def __init__(self, visit_id, patient_id, doctor_id="", hospital_id="",`
	`                 scheduled_date="", scheduled_time="", checked_in_at="",`
	`                 duration_minutes="", reason="", symptoms="", diagnosis="",`
	`                 severity="", vitals_bp="", vitals_pulse="", vitals_temp="",`
	`                 vitals_spo2="", vitals_weight="", vitals_height="",`
	`                 follow_up_days="", status="", notes=""):`
		L4	<code>__init__</code>: <code>visit_id</code> and <code>patient_id</code> required, the remaining nineteen default to <code>""</code> — a visit can be created before the consultation happens (checked_in_at, vitals and diagnosis arrive later).

88–88	`        """Create a consultation record."""`	txt	Docstring: creates a consultation record.

89–109	`        self.visit_id = visit_id`
	`        self.patient_id = patient_id`
	`        self.doctor_id = doctor_id`
	`        self.hospital_id = hospital_id`
	`        self.scheduled_date = scheduled_date`
	`        self.scheduled_time = scheduled_time`
	`        self.checked_in_at = checked_in_at`
	`        self.duration_minutes = duration_minutes`
	`        self.reason = reason`
	`        self.symptoms = symptoms`
	`        self.diagnosis = diagnosis`
	`        self.severity = severity`
	`        self.vitals_bp = vitals_bp`
	`        self.vitals_pulse = vitals_pulse`
	`        self.vitals_temp = vitals_temp`
	`        self.vitals_spo2 = vitals_spo2`
	`        self.vitals_weight = vitals_weight`
	`        self.vitals_height = vitals_height`
	`        self.follow_up_days = follow_up_days`
	`        self.status = status`
	`        self.notes = notes`
		L4	Twenty-one <code>self.x = x</code> assignments, one per column, exactly like Patient's: 89 visit and patient ids, 90 patient link, 91–92 doctor and hospital links, 93–95 the date, slot time and actual check-in, 96 duration, 97–99 reason and symptoms and the diagnosis, 100 severity, 101–105 the five vitals readings, 106–107 weight and height, 108–109 follow-up days and status + notes. The column order matches FIELDS(order) so a CSV row and an object always agree.

110–110		txt	Blank line.

111–112	`    def symptom_set(self):` `        """Return reported symptoms as a set, split on pipes."""`	L3	<code>symptom_set()</code> — the visit's symptoms as a set: the set algebra in triage.py consumes exactly this shape.
113–114	`        if not self.symptoms:` `            return set()`	L1	<code>if not self.symptoms:</code> returns an empty set for a blank cell — a visit with no reported symptoms yields <code>set()</code>, not an error, because a blank is a valid state (the spec's blank-means-unknown rule).
115–115	`        return {s.strip().casefold() for s in self.symptoms.split("|") if s.strip()}`	L3	The set comprehension: <code>{s.strip().casefold() for s in self.symptoms.split("|") if s.strip()}</code> — split on pipes (Practical 2), strip and casefold each fragment (L2), keep non-blanks, collect into a set (L3). <code>casefold()</code> normalises case so "Fever" and "fever" are one symptom.
116–116		txt	Blank line.

117–122	`    def bmi(self):`
	`        """Return BMI from the recorded height and weight, or None.`
	
	`        The Practical 1 categorisation problem applied to real records.`
	`        Height is recorded in centimetres, so it is converted first.`
	`        """`
		L5	<code>bmi()</code> — the Practical 1 BMI formula applied to real records. The docstring records the unit contract: height is stored in centimetres, so conversion happens here before the formula. It also names its ancestor: the categorisation problem from Practical 1.

123–127	`        try:`
	`            weight = float(self.vitals_weight)`
	`            height_cm = float(self.vitals_height)`
	`        except (TypeError, ValueError):`
	`            return None`
		L1	<code>try/except (TypeError, ValueError)</code> (breaking of Practical 1's cast pattern): <code>float(...)</code> casts the stored strings to numbers; a blank or non-numeric cell raises and returns <code>None</code> rather than crashing or pretending. <code>None</code> then means “BMI not computable” to every caller.

128–130	`        if not weight or not height_cm:` `            return None`	L1	<code>if not weight or not height_cm:</code> — a zero height or zero weight is unusable as well as a non-numeric one, so both are rejected before dividing, which is what stops a division by zero.
131–131		txt	Blank line.
132–132	`        metres = height_cm / 100.0`	L1	<code>metres = height_cm / 100.0</code> — the centimetre conversion, one division.
133–133	`        return weight / (metres * metres)`	L5	<code>return weight / (metres * metres)</code> — the BMI formula weight / height² with brackets making the square a real square rather than an order-of-operations accident. Operators and precedence are Practical 1.
134–134		txt	Blank line.
135–136	`    def waited_minutes(self):` `        """Return how long after the slot the patient actually arrived."""`	L2	<code>waited_minutes()</code> — how late the patient checked in versus their slot. Both endpoints are strings like "09:30".
137–138	`        if not (self.scheduled_time and self.checked_in_at):` `            return None`	L1	<code>if not (self.scheduled_time and self.checked_in_at):</code> — both or either missing means the wait is undefined: return None. The parenthesised <code>and</code> of two truth tests is Practical 1's boolean operators.
139–139		txt	Blank line (inside function).

140–142	`        try:`
	`            booked = [int(p) for p in self.scheduled_time.split(":")]`
	`            arrived = [int(p) for p in self.checked_in_at.split(":")]`
		L2	Lines 141–142 build minute-of-day lists: <code>[int(p) for p in self.scheduled_time.split(":")]</code> is the same list comprehension + cast pattern twice, once for the slot and once for the check-in. Splitting on ":" gives ["HH", "MM"].

143–143	`            return (arrived[0] * 60 + arrived[1]) - (booked[0] * 60 + booked[1])`	L1	Brings the arithmetic together: <code>(arrived[0] * 60 + arrived[1]) - (booked[0] * 60 + booked[1])</code> — converts both to minutes past midnight and subtracts. Indexing lists (L2) and arithmetic (P1).
144–145	`        except (ValueError, IndexError):` `            return None`	L4	<code>except (ValueError, IndexError): return None</code> — a malformed time string or an unexpected number of parts cannot crash the analytics; the wait is reported as unknown. Same fail-soft pattern as bmi().
146–146		txt	Blank line.
147–148	`    def to_row(self):` `        """Return values in FIELDS order, ready for csv.DictWriter."""`	L4	Visit's <code>to_row()</code> with the same docstring as Patient's — values in FIELDS order for <code>csv.DictWriter</code>.
149–149	`        return [getattr(self, field) for field in self.FIELDS]`	L3	Same list comprehension over <code>self.FIELDS</code> — a deliberate copy of the Patient method, because the two CSV-writing paths behave identically.
150–150		txt	Blank line.
151–152	`    def to_dict(self):` `        """Return a plain dict for JSON responses."""`	L4	Visit's <code>to_dict()</code> and its docstring — the dict form for JSON responses.
153–153	`        return {field: getattr(self, field) for field in self.FIELDS}`	L3	Same dict comprehension over FIELDS.
154–154		txt	Blank line.
155–156	`    @staticmethod` `    def from_row(row):`	L4	<code>@staticmethod</code> again on Visit's <code>from_row</code>, same reasoning as Patient's.
157–157	`        """Build a Visit from a dict read out of a CSV."""`	txt	Docstring.
158–158	`        return Visit(**{k: row.get(k, "") for k in Visit.FIELDS})`	L3	<code>return Visit(**{k: row.get(k, "") for k in Visit.FIELDS})</code> — identical shape to Patient.from_row: dict comprehension defaulting blanks, then <code>**</code> unpacking into the constructor.

store.py 490 lines — CSV read / append / update / delete / backup — file handling, Lab 4.

1–10	"""CSV persistence for Meridian. Taught concepts: open() in modes r/w/a/x, the with statement, readline, readlines, write, DictReader, DictWriter, seek, and the os module (listdir, walk, makedirs, path.exists, rename). Spec sections 4.1, 4.2 and 9. Every read returns (records, skipped) so a malformed row is reported rather than silently dropped. """	**txt**	Module docstring. store.py is the CSV persistence layer: it holds the column schemas, the table registry, and every read / append / update / delete / backup operation. It announces the taught concepts it is made of — open() in the modes r / w / a / <code>x</code>, the <code>with</code> statement, <code>readline</code>, <code>readlines</code>, <code>write</code>, DictReader, DictWriter, <code>seek</code>, and the os module (<code>listdir</code>, <code>walk</code>, <code>makedirs</code>, <code>path.exists</code>, <code>rename</code>) — and states the design rule that every return shape follows: (records, skipped), so a malformed row is reported, never dropped silently. 10 blank.
11–11	import csv	**X:csv**	<code>import csv</code> — the csv module itself, beyond the labs. It is the reader and writer behind every <code>csv.DictReader</code> and <code>csv.DictWriter</code> call in this file; its behaviour is documented in docs/concepts/csv-module.md.
12–12	import os	**std**	<code>import os</code> — os (Practical 4) supplies path joins, existence checks and directory creation here.
13–13	import shutil	**X:shutil**	<code>import shutil</code> — beyond the labs: the one-line file copy used by the backup routine. Explained in docs/concepts/shutil-and-zipfile.md.
14–15	from datetime import datetime	**std**	<code>from datetime import datetime</code> — the only import past the six lab syllabuses that the labs could not avoid: backup folders are named by timestamp. Tagged std rather than a lab practical.
16–18	DATA_DIR = os.environ.get("MERIDIAN_DATA_DIR", "data") BACKUP_DIR = os.path.join(DATA_DIR, "backups")	**std**	16: <code>DATA_DIR = os.environ.get("MERIDIAN_DATA_DIR", "data")</code> reads an environment variable with a default — standard library, no lab example. It exists so the Vercel deployment can point the data directory at a writable /tmp copy while a local run keeps the plain relative path. 17–18: <code>BACKUP_DIR</code> is <code>data/backups</code> built with <code>os.path.join</code> (Practical 4), the only directory that must accept writes at runtime.
19–24	PATIENT_FIELDS = ["patient_id", "name", "age", "gender", "phone", "email", "blood_group", "address_line", "area", "pincode", "city", "state", "emergency_contact", "emergency_phone", "registered_on", "notes",]	**L2**	<code>PATIENT_FIELDS</code> — a Python list of strings (Practical 2), the exact column order of patients.csv: ids, name, demographics, address, emergency contacts, registration date, notes. 16 columns. These constants are the schema; write operations loop them so no DictWriter call ever hard-codes a name. 23 closes the list, 24 blank.

25–32	`VISIT_FIELDS = ["visit_id", "patient_id", "doctor_id", "hospital_id", "scheduled_date", "scheduled_time", "checked_in_at", "duration_minutes", "reason", "symptoms", "diagnosis", "severity", "vitals_bp", "vitals_pulse", "vitals_temp", "vitals_spo2", "vitals_weight", "vitals_height", "follow_up_days", "status", "notes",]`	**L2**	<code>VISIT_FIELDS</code> — 21 columns for one consultation: ids, scheduling, check-in, duration, reason, symptoms pipe-list, diagnosis, severity, five vital signs, follow-up, status, notes.
33–37	`MEDICINE_FIELDS = [ "medicine_id", "name", "generic", "category", "form", "strength", "otc", "rx_required", "storage", "price", "manufacturer", ]`	**L2**	<code>MEDICINE_FIELDS</code> — 11 columns: id, name, generic, category, form, strength, the two yes/no dispensing flags (`otc` / `rx_required`), storage, price, manufacturer. 37 blank after the close.
38–42	`STORE_FIELDS = [ "store_id", "name", "type", "area", "city", "phone", "hours", "rating", "stock_csv", ]`	**L2**	<code>STORE_FIELDS</code> — 9 columns, ending with <code>stock_csv</code>, the pipe-separated medicine-id list that the set intersections in triage and registry later consume.
43–47	`DISEASE_FIELDS = [ "disease_id", "icd10_code", "name", "severity", "symptoms", "medication", "dosage", "age_range", "contraindications", "symptom_source", ]`	**L2**	<code>DISEASE_FIELDS</code> — the triage knowledge base as CSV: id, ICD-10 code, name, severity, symptoms, medication, dosage, age range, contraindications, plus <code>symptom_source</code> recording where the symptom list came from.
48–52	`HOSPITAL_FIELDS = [ "hospital_id", "name", "type", "address_line", "area", "city", "state", "pincode", "phone", "beds", "established_year", "accreditation", ]`	**L2**	<code>HOSPITAL_FIELDS</code> — 12 columns: id, name, type, full address, pincode, phone, bed count, established year, accreditation.
53–58	`DOCTOR_FIELDS = [ "doctor_id", "name", "qualification", "specialisation", "registration_no", "hospital_id", "department", "room_no", "phone", "consultation_fee", "experience_years", "languages", "availability", ]`	**L2**	<code>DOCTOR_FIELDS</code> — 13 columns: qualification, specialisation, MMC registration number, hospital link, department, room, phone, consultation fee, experience years, languages, availability.

59–62	txt	`# The rare disease registry, generated by scripts/build_rare_conditions.py`
		`# from the Human Phenotype Ontology and its disease annotations. Symptoms`
		`# are phenotypic findings rather than patient complaints, so this registry`
		`# is browsed rather than fed to the outpatient triage engine.`
			Comment block on the rare-condition registry: it is generated by scripts/build_rare_conditions.py from the Human Phenotype Ontology, its symptom column holds phenotypic findings rather than patient complaints, and that is why this registry is browsed instead of being fed to the outpatient triage engine.

63–67	L2	`RARE_CONDITION_FIELDS = [`
		`    "condition_id", "mondo_id", "name", "symptom_count", "symptoms",`
		`    "hpo_ids", "inheritance_mode",`
		`]`
			<code>RARE_CONDITION_FIELDS</code> — 7 columns: condition and MONDO ids, name, symptom count, symptoms, HPO id list, inheritance mode.

68–71	L2	`HPO_SYMPTOM_FIELDS = [`
		`    "hpo_id", "term", "lay_term", "has_lay_wording",`
		`]`
			<code>HPO_SYMPTOM_FIELDS</code> — the HPO lookup table: ids, term, lay wording and a flag recording whether a lay phrasing exists.

72–77	L2	`# The written half of a condition, kept apart from diseases.csv so that`
		`# editing prose cannot change how triage ranks. See`
		`# scripts/build_disease_articles.py.`
		`DISEASE_ARTICLE_FIELDS = ["disease_id", "summary", "red_flags",`
		`"refer_when", "medicine_ids", "provenance"]`
			72–74: the comment states a real architecture decision — the written half of a disease (article prose) is kept in a separate file from the machine-readable half in diseases.csv, so editing prose cannot change how triage ranks. 75–76: <code>DISEASE_ARTICLE_FIELDS</code> is one style of list: a 6-column schema written on a wrapped two-line literal (id, summary, red flags, refer-when, medicine ids, provenance). 77 blank.

78–93	RECORDS = {
	    "patients": ("patients.csv", PATIENT_FIELDS, "patient_id"),
	    "visits": ("visits.csv", VISIT_FIELDS, "visit_id"),
	    "medicines": ("medicines.csv", MEDICINE_FIELDS, "medicine_id"),
	    "stores": ("stores.csv", STORE_FIELDS, "store_id"),
	    "diseases": ("diseases.csv", DISEASE_FIELDS, "disease_id"),
	    "hospitals": ("hospitals.csv", HOSPITAL_FIELDS, "hospital_id"),
	    "doctors": ("doctors.csv", DOCTOR_FIELDS, "doctor_id"),
	    "rare_conditions": ("rare_conditions.csv", RARE_CONDITION_FIELDS,
	
	"condition_id"),
	    "hpo_symptoms": ("hpo_symptoms.csv", HPO_SYMPTOM_FIELDS, "hpo_id"),
	    "disease_articles": ("disease_articles.csv", DISEASE_ARTICLE_FIELDS,
	
	"disease_id"),
	}
		L3	<code>RECORDS</code> — the central directory of this module and a dict of tuples (Practical 3): record name → (filename, field list, id column), ten entries. Every helper below reads this table (tuple indexing, <code>RECORDS[record_type][0]</code>), which is why adding a registry is one data row, not new code. 86–87 and 89–90 wrap long entries over two lines. 92–93 blanks.

94–98	def path_for(record_type): """Return the CSV path for a record type.""" return os.path.join(DATA_DIR, RECORDS[record_type][0])	L4	<code>path_for(record_type)</code>: 95 docstring, 96 <code>os.path.join(DATA_DIR, RECORDS[record_type][0])</code> — tuple lookup by index plus a Practical 4 path join. 97–98 blanks.
99–103	def fields_for(record_type): """Return the column order for a record type.""" return RECORDS[record_type][1]	L4	<code>fields_for(record_type)</code>: one line (101, <code>RECORDS[record_type][1]</code>) returns that table's column order. Small accessors over the shared table keep the module honest — no caller is ever trusted to type the CSV name by hand. 102–103 blanks.

104–122

```
def ensure_csv(record_type):
    """Create the CSV with a header row if it is not there yet.

    Uses mode 'x' first, then falls back to 'a' when the file already
    exists. Mode 'x' raises FileExistsError, which is how we detect it.
    """
    path = path_for(record_type)
    fields = fields_for(record_type)

    try:
        with open(path, "x", newline="") as f:
            csv.DictWriter(f, fieldnames=fields).writeheader()
        print(f"created {path}")
        return True

    except FileExistsError:
        return False
```

L4

<code>ensure_csv(record_type)</code>: 105–109 docstring — create the CSV with a header row when it does not exist yet, using mode <code>'x'</code> (exclusive create, Practical 4) first and falling back when it raises. 110 derives the path, 111 the columns, 113–115 the <code>try: with open(path, "x", newline=""):</code> plus <code>csv.DictWriter(f, fieldnames=fields).writeheader()</code> — DictWriter is the csv module, beyond the labs (docs/concepts/csv-module.md); <code>newline=""</code> is its newline rule. 116 prints what was created (an f-string, Practical 5), 117 returns True, 119–120 the <code>except FileExistsError: return False</code> path — the file existing is normal, not an error. 121–122 blanks.

123–136

```
def read_all(record_type):
    """Read a CSV into (records, skipped).

    records  list of dicts, one per well-formed row
    skipped  list of {"line": int, "reason": str} for rows we could not use

    A row is skipped when it has too many fields, too few, or when the
    header does not match the expected columns. This happens if someone
    edits the file in a spreadsheet and leaves a stray comma. Spec
    section 9 requires this to be visible, not silent.

    Note that DictReader pads a short row with None values instead of
    omitting the keys, so too-few and too-many both need checking.
    """
```

txt

<code>read_all(record_type)</code> and its docstring (124–136) — the honest-read contract. It returns (records, skipped): well-formed rows as dicts, and one <code>{"line": int, "reason": str}</code> entry per unusable row (too many fields, too few, header mismatch from a stray comma in a spreadsheet edit). Spec section 9 demands this visibility. 134–135 pre-announce the trap exploited below: DictReader pads a short row with None values instead of omitting the keys, so too-few and too-many must both be checked.

137–142	`    path = path_for(record_type)`
	`    fields = fields_for(record_type)`
	
	`    if not os.path.exists(path):`
	`        ensure_csv(record_type)`
	`        return [], []`
		L4	The file-missing branch: 140–142 — if the CSV does not exist (<code>os.path.exists</code>, Practical 4), call <code>ensure_csv</code> and return two empty lists; an absent file is an empty table, not an exception. 137–138 grab path and columns, 139 blank.

143–148	`    records = []`
	`    skipped = []`
	
	`    with open(path, "r", newline="") as f:`
	`        reader = csv.DictReader(f)`
		X:csv	144–145 create the two result accumulators, 147 <code>with open(path, "r", newline="") as f:</code> (Practical 4 read mode, csv newline convention), 148 <code>reader = csv.DictReader(f)</code> — beyond the labs: streams rows as dicts keyed by the header row (docs/concepts/csv-module.md).

149–152	`        line = 1`
	
	`        for row in reader:`
	`            line += 1`
		L1	Row accounting: 149 <code>line = 1</code> starts at the header, 151 <code>for row in reader:</code> the loop, 152 <code>line += 1</code> — the accumulator pattern (Practical 1) so skip reports can name a human line number. 150 blank.

153–161	`            # A None key holds the fields that were present but not`
	`            # declared in the header, so the row has too many.`
	`            if None in row:`
	`                skipped.append({`
	
	`"line": line,`
	
	`"reason": f"too many fields, expected {len(fields)}",`
	`                })`
	`                continue`
		X:csv	Too-many-fields check. 154–155 comment explains it: a None key in a DictReader row holds values beyond the declared header. 156 <code>if None in row:</code> — a key-in-dict test (Practical 3) against csv-module behaviour itself. 157–160 append one skip record whose reason is an f-string naming the expected count, 161 <code>continue</code> to the next row.

162–170	`            missing = [k for k in fields if k not in row]`
	`            if missing:`
	`                skipped.append({`
	
	`"line": line,`
	
	`"reason": f"header mismatch, missing fields: "`
	
	`f"{', '.join(missing)}",`
	`                })`
	`                continue`
		L3	Header-mismatch check: 163 <code>missing = [k for k in fields if k not in row]</code> — a list comprehension (Practical 3) collecting every expected column absent from the row dict; 164–170 — if any are missing, append a skip record whose reason is a comma-joined list of names ("header mismatch, missing fields: …") and continue. The f-string with <code>', '.join(...)</code> is Practical 2/5 formatting.

171–184	L3
	`            # DictReader pads a short row with None rather than leaving the`
	`            # key out, so the checks above cannot see it. A None value here`
	`            # means the row ended early. Caught explicitly, because the`
	`            # (row[k] or "") below would otherwise blank the field and the`
	`            # record would look valid with data quietly lost.`
	`            padded = [k for k in fields if row[k] is None]`
	`            if padded:`
	`                skipped.append({`
	`"line": line,`
	`"reason": f"too few fields, missing values for: "`
	`f"{', '.join(padded)}",`
	`                })`
	`                continue`
		Too-few-fields check. 172–176: the comment carries the hard-won detail — DictReader pads a short row with None rather than dropping the key, so the checks above cannot catch it; a None value means the row ended early, and without this check the <code>(row[k] or "")</code> below would quietly blank the field and the record would look valid with data lost. 177 <code>padded = [k for k in fields if row[k] is None]</code> — comprehension plus an identity test (Practical 1); 178–184 the same skip-record shape as before.

185–186	L3 `            records.append({k: (row[k] or "").strip() for k in fields})`	186: <code>records.append({k: (row[k] or "").strip() for k in fields})</code> — the accepted row is normalised in one dict comprehension (Practical 3): an empty cell becomes "", whitespace is stripped, and the key order is the schema's own. Everything downstream can trust this shape.
187–187	txt	Blank line.
188–188	L4 `    return records, skipped`	188 <code>return records, skipped</code> — the tuple return promised by the docstring.

189–198	txt
	`def append_row(record_type, row):`
	`    """Append one row to a CSV and return the stored dict.`
	`    Before appending we check the file ends with a newline. If it does`
	`    not, the previous row has no line terminator and our new row would be`
	`    joined onto it, producing one corrupt line instead of two valid ones.`
	`    That failure is silent, so we guard against it explicitly.`
	`    """`
		189–190 blanks, then <code>append_row(record_type, row)</code> and its docstring (192–198): appends one row and returns the stored dict. The docstring records the nasty failure it guards: if the file does not end with a newline, the new row would join onto the previous line, producing one corrupt line instead of two valid ones — the failure is silent, so it is checked explicitly.

199–203	`    path = path_for(record_type)`
	`    fields = fields_for(record_type)`
	
	`    if not os.path.exists(path):`
	`        ensure_csv(record_type)`
		L4	199–200 path and columns from the table; 202–203 — if the file does not exist, <code>ensure_csv</code> creates it, so a fresh registry appends cleanly instead of crashing.

204–209	`    needs_newline = os.path.getsize(path) > 0`
	`    if needs_newline:`
	`        with open(path, "rb") as f:`
	`            f.seek(-1, 2)`
	`# seek to the last byte`
	`            needs_newline = f.read(1) != b"\n"`
		L4	The trailing-newline guard: 205 <code>needs_newline = os.path.getsize(path) > 0</code> (a non-empty file is presumed unterminated), reconciled at 207–209: open binary, <code>f.seek(-1, 2)</code> (two steps back from the end — the Practical 4 <code>seek()</code> clause with a negative offset from <code>whence=2</code>), and read one byte: is it <code>b"\\n"</code>? Only now is the presumption confirmed or refuted.

210–212	`    clean = {k: ("" if row.get(k) is None else str(row.get(k, "")).strip())` `             for k in fields}`	**L3**	211–212: <code>clean = {k: ("" if row.get(k) is None else str(row.get(k, "")).strip()) for k in fields}</code> — a dict comprehension (P3) with a conditional expression (P1): None becomes an empty cell, everything else is stringified and stripped, column order enforced by the schema list.

213–217	`    with open(path, "a", newline="") as f:`
	`        if needs_newline:`
	`            f.write("\n")`
	`        csv.DictWriter(f, fieldnames=fields).writerow(clean)`
		X:csv	214 <code>with open(path, "a", newline="") as f:</code> — append mode, the Practical 4 mode that preserves history. 215–216 the guard from the docstring runs here: if the last byte was not a newline, one is written first, 217 <code>csv.DictWriter(...).writerow(clean)</code> serialises the row — csv quoting is handled by the module, not by hand (docs/concepts/csv-module.md).

218–219	`    return clean`	**L4**	219 <code>return clean</code> — the caller echoes what was physically stored, not merely what was asked for.
220–221		**txt**	Blank lines between functions.

222–231	`def get_by_id(record_type, target):`
	`    """Return the record whose id matches, or None."""`
	`    id_field = RECORDS[record_type][2]`
	`    records, _ = read_all(record_type)`
	
	`    for record in records:`
	`        if record[id_field] == target:`
	`            return record`
	
	`    return None`
		L4	<code>get_by_id(record_type, target)</code>: 224 the id column from the table, 225 <code>read_all</code>, 227–229 a linear scan with <code>==</code> matching and an early <code>return record</code>, 231 honest <code>return None</code> after the loop unwinds. No index exists in a CSV world; O(n) at hundreds of rows is correct tooling.

232–233		**txt**	Blank lines.

234–240	txt
	`def next_id(record_type, prefix):`
	`    """Return the next free id, zero padded to 4 digits.`
	
	`    Reads existing ids, takes the largest numeric suffix and adds one.`
	`    Zero padding keeps ids in lexicographic order, so P-0009 sorts before`
	`    P-0010 rather than after it.`
	`    """`
		<code>next_id(record_type, prefix)</code> and its docstring (235–240): read the existing ids, keep the largest numeric suffix and add one; zero padding (P-0009 before P-0010) keeps ids in lexicographic order. The pattern is a vocabulary-walk, not code review material.

241–245	L4
	`    id_field = RECORDS[record_type][2]`
	`    records, _ = read_all(record_type)`
	
	`    highest = 0`
		241–242 id column and read_all again; 244 <code>highest = 0</code> — the accumulator (P1) that the scan will raise monotonically.

246–249	L4
	`    for record in records:`
	`        raw = record[id_field]`
	`        if not raw.startswith(prefix):`
	`            continue`
		The scan: 246 every record, 247 its raw id, 248–249 ids of the wrong prefix (<code>raw.startswith(prefix)</code>) are skipped — the P2 prefix test guarding against ids like M-0007 leaking into a patients table.

250–252	L2
	`        suffix = raw[len(prefix):]`
	`        if suffix.isdigit():`
	`            highest = max(highest, int(suffix))`
		250 <code>suffix = raw[len(prefix):]</code> — slicing with a computed start (Practical 2), 251–252 only digits: <code>suffix.isdigit()</code> then <code>highest = max(highest, int(suffix))</code> — cast (P1) wrapped in max() so gaps in the numbering do not matter.

253–254	L5 `    return f"{prefix}{highest + 1:04d}"`	254: <code>return f"{prefix}{highest + 1:04d}"</code> — the <code>:04d</code> format spec (Practical 5) is the zero-pad promise: 9 becomes 0010, 999 becomes 1000, in one expression.
255–256	txt	Blank lines.

257–262	L3
	`def visits_for(patient_id):`
	`    """Return every consultation for one patient, oldest first."""`
	`    records, _ = read_all("visits")`
	`    mine = [r for r in records if r["patient_id"] == patient_id]`
	`    mine.sort(key=lambda r: (r["scheduled_date"], r["scheduled_time"]))`
	`    return mine`
		<code>visits_for(patient_id)</code>: 259 read the visits table, 260 a list comprehension keeping rows whose <code>patient_id</code> matches, 261 <code>mine.sort(key=lambda r: (r["scheduled_date"], r["scheduled_time"]))</code> — the sorted(key=lambda) pattern (P3) sorting by a tuple so ties fall back to time; oldest first for a life history. 262 return.

263–264	txt	Blank lines.

265–268	L3
	`def doctors_at(hospital_id):`
	`    """Return the doctors practising at one hospital."""`
	`    records, _ = read_all("doctors")`
	`    return [d for d in records if d["hospital_id"] == hospital_id]`
		<code>doctors_at(hospital_id)</code>: one comprehension as the whole body (268) — filter the doctors table on <code>hospital_id</code>. Documents the relationship both files declare: the doctor row carries the hospital id.

269–270	txt	Blank lines.

271–277

```
def visits_for_doctor
(doctor_id):
    """Return every c
onsultation for one d
octor, newest firs
t."""
    records, _ = read
_all("visits")
    mine = [v for v i
n records if v["docto
r_id"] == doctor_id]
    mine.sort(key=lam
bda r: (r["scheduled_
date"], r["scheduled_
time"]),
              reverse
=True)
    return mine
```

L3

<code>visits_for_doctor(doctor_id)</code> — the doctor's diary: the same filter shape (274) but the sort is <code>reverse=True</code> (275–276), so a workday list starts with the freshest consultation rather than the earliest. Same code, opposite intent, one boolean difference.

278–279

```

```

txt

Blank lines.

280–286

```
def update_row(record
_type, id_field, targ
et, updates):
    """Rewrite one ro
w with some fields ch
anged.

    CSV has no update
mode, so this reads e
verything, changes th
e matching
    row, then rewrite
s the file using mode
'w'. This is the merg
e-and-
    overwrite pattern
from the Lab 4 file-h
andling practical.
    """
```

txt

<code>update_row(record_type, id_field, target, updates)</code> and its docstring (281–286): CSV has no update mode; this rewrites everything — read, change the matching row, then rewrite the file in mode <code>w</code>. The comment names it the merge-and-overwrite pattern of the Lab 4 file-handling practical.

287–291

```
    path = path_for(r
ecord_type)
    fields = fields_f
or(record_type)

    records, _ = read
_all(record_type)
    changed = None
```

L4

287–288 path and columns; 290 read everything into memory; 291 <code>changed = None</code> — the sentinel that will say whether any row matched.

292–299

```
    for record in rec
ords:
        if record[id_
field] == target:
            for key,
value in updates.item
s():
                if ke
y in fields and key !
= id_field:
                    r
ecord[key] = value
            changed =
record
            break
```

L4

Find-and-patch: 293–294 loop until <code>record[id_field] == target</code>; 295 loop the update dict (<code>updates.items()</code>, P3); 296–297 assign only if the key is a real column and not the id — unknown keys and id rewrites are refused by the guard, not by an exception; 298 remember the changed row, 299 <code>break</code> — one match per call, by definition.

300–302

```
    if changed is Non
e:
        return None
```

L1

301–302: <code>if changed is None: return None</code> — the sentinel test (P1) converts a silent no-op into an honest not-found for the route to turn into a 404.

303–310	`    with open(path, "w", newline="") as f:`
	`        writer = csv.DictWriter(f, fieldnames=fields)`
	`        writer.writeheader()`
	`        for record in records:`
	`            writer.writerow(record)`
	
	`    return changed`
		X:csv	The rewrite: 304 <code>open(path, "w", newline="")</code> — truncate (the mode without which merge-and-overwrite cannot be expressed), 305–306 DictWriter then <code>writeheader()</code>, 307–309 a plain for loop <code>writerow(record)</code> for every row — the batching docs/concepts/csv-module.md describes. 310 returns the changed record.

311–312		**txt**	Blank lines.

313–318	`def delete_row(record_type, id_field, target):`
	`    """Delete one row by rewriting the file without it. Returns True/False."""`
	`    path = path_for(record_type)`
	`    fields = fields_for(record_type)`
	
	`    records, _ = read_all(record_type)`
		L4	<code>delete_row(record_type, id_field, target)</code>: 314 docstring says the method — delete by rewriting the file without the row — 318 reads the table.

319–322	`    kept = [r for r in records if r[id_field] != target]`
	
	`    if len(kept) == len(records):`
	`        return False`
		L3	319: <code>kept = [r for r in records if r[id_field] != target]</code> — the inverse list comprehension; then 321–322: comparing lengths (<code>len()</code>, P1), <code>==</code> means nothing was dropped, so return False — the route's 404 answer, not an exception.

323–330	`    with open(path, "w", newline="") as f:`
	`        writer = csv.DictWriter(f, fieldnames=fields)`
	`        writer.writeheader()`
	`        for record in kept:`
	`            writer.writerow(record)`
	
	`    return True`
		X:csv	The rewrite, same as update: 324–326 mode <code>w</code> + writeheader, 328–329 write each kept row with <code>writerow</code> (docs/concepts/csv-module.md).

331–331		**L4**	331 <code>return True</code> after the loop — the route's success answer.

332–343	def backup_all(): """Copy the five registry CSVs into a timestamped backup fo lder. Scope is delibera tely limited to the r egistries. data/datas ets/ holds downloaded resear ch data that never ch anges at runtime, and it contains filenames that co llide with the regist ry files once flatten ed into a single folder, so including it would si lently overwrite one with the other. Returns the list of destination paths written. """	txt	<code>backup_all()</code> and its docstring (334–343): copy the registry CSVs into a timestamped backup folder. The scope is deliberately registries only: <code>data/datasets/</code> holds downloaded research data that never changes at runtime, and its filenames collide with the registry files when flattened into one folder — including them would silently overwrite one with the other. Returns the list of destination paths written.
344–346	if not os.path.ex ists(DATA_DIR): return []	L1	344–345: no data directory, nothing to back up — return an empty list instead of raising. 346 blank.
347–348	stamp = datetime. now().strftime("%Y%m% d_%H%M%S_%f") destination = os. path.join(BACKUP_DIR, stamp)	std	347: <code>stamp = datetime.now().strftime("%Y%m%d_%H%M%S_%f") </code> — the standard datetime module formatting a timestamp down to microseconds so two backups in one second still sort apart; 348 joins it under the backup dir (P4 os.path.join). <code>strftime</code> is std-lib, no lab.
349–351	if not os.path.ex ists(destination): os.makedirs(d estination)	L4	350–351: <code>if not os.path.exists(destination): os.makedirs(destination)</code> — the Practical 4 pattern (folder idea plus <code>makedirs</code> from the os list), write the directory only once.
352–353	written = []	L1	353: <code>written = []</code> — the accumulator (Practical 1's verbatim pattern).
354–360	for record_type i n RECORDS: filename = RE CORDS[record_type][0] source = os.p ath.join(DATA_DIR, fi lename) if not os.pat h.exists(source): continue	L4	The copy loop: 355 every record type in the table, 356–357 filename and source path, 359–360 skip a registry whose file has never been written. Missing input is a non-event for a backup.
361–364	target = os.p ath.join(destination, filename) shutil.copyfi le(source, target) written.appen d(target)	X:shutil	362 the destination path, 363 <code>shutil.copyfile(source, target) </code> — beyond the labs, the whole backup idea reduced to one call (docs/concepts/shutil-and-zipfile.md); 364 records the destination so the route can report what was written.
365–366	return written	L4	366 <code>return written</code> — the manifest, not a boolean; the caller can print the actual backup list.

367–372 **L4**

```
def count_records(record_type):
    """Return how many rows a CSV holds."""
    records, _ = read_all(record_type)
    return len(records)
```

<code>count_records(record_type)</code>: 371–372 read the table and return <code>len(records)</code> (P1 len). A thin wrapper the health endpoint and tests use so they need not know about (records, skipped) pairs themselves.

373–384 **txt**

```
# ---------------------------------------------------------- facets
#
# A facet is a named subset of a registry that corresponds to something a
# clinician would want to isolate: an incomplete record, a group that needs
# a different approach, or a gap in follow-up. They exist so the browse views
# can open on a real question rather than on an alphabetical list.
#
# Each facet is a predicate over one row. Counts are computed by applying the
# predicate to every row, so a count can never disagree with the list that the
# same flag returns.
```

373 blank; 374 the section-banner comment for the facets; 376–383 the design paragraph: a facet is a named subset of a registry that corresponds to something a clinician would want to isolate — an incomplete record, a group needing a different approach, a follow-up gap — so browse views open on a real question rather than an alphabetical list. Each facet is a predicate over one row; counts are computed by applying the predicate to every row, so a count can never disagree with the list the same flag returns. 384 blank.

385–388 **L2**

```
def _is_child(row):
    return row.get("age", "").isdigit() and int(row["age"]) < 18
```

<code>_is_child(row)</code>: 386 <code>row.get("age", "").isdigit() and int(row["age"]) < 18</code> — the string method from Practical 2 puts <code>isdigit()</code> first on purpose: short-circuit <code>and</code> (P1 blocks the int cast unless digits exist, and the two conditions chain in one Boolean line. 388 blank.

389–392 **L2**

```
def _is_senior(row):
    return row.get("age", "").isdigit() and int(row["age"]) >= 60
```

<code>_is_senior(row)</code>: identical shape, threshold <code>>= 60</code> — siblings, readable side by side; the literal per line is what the browse chips say.

393–396 **L3**

```
def _has_visit(row):
    return row["patient_id"] in _visited_ids()
```

<code>_has_visit(row)</code>: 394 <code>row["patient_id"] in _visited_ids()</code> — membership in a set (P3), O(1) per patient. 395–396 blanks.

397–399 **L3**

```
_visited_cache = {}
```

<code>_visited_cache = {}</code> — a module-level dict (P3) acting as a cache, so /api pages that ask about visits do not re-read visits.csv per request; 398–399 blanks.

400–411	def _visited_ids():
	    """Patient ids with at least one consultation on file."""
	    cached = _visited_cache.get("ids")
	
	    if cached is None:
	        records, _ = read_all("visits")
	        cached = {r["patient_id"] for r in records}
	        _visited_cache["ids"] = cached
	
	    return cached
		L4	<code>_visited_ids()</code>: 402 fetch the cached set with <code>.get("ids")</code>; 404–407 on a miss (<code>if cached is None</code>, P1) read the visits file and build the patient-id set with a set comprehension <code>{r["patient_id"] for r in records}</code>, then store it back; 409 return either way. A small, one-function memoisation pattern; 410–411 blanks.

412–415	def _blank(row, field): return not row.get(field, "").strip()	L3	<code>_blank(row, field)</code>: 413 <code>not row.get(field, "").strip()</code> — a blank-expression checker used by many facets: an <code>.get()</code> default plus <code>.strip()</code>, wrapped in <code>not</code>. 414–415 blanks.

416–451

```
FACETS = {
    "patients": [
        ("no_visit",
"No consultation on f
ile",
         "Registered
but never seen", lamb
da r: r["patient_id"]
not in _visited_ids
()),
        ("child", "Un
der 18",
         "Recorded wi
th a guardian", _is_c
hild),
        ("senior", "A
ge 60 and over",
         "Age-related
review due", _is_seni
or),
        ("no_blood_gr
oup", "No blood grou
p",
         "Field left
blank on the record",
lambda r: _blank(r,
"blood_group")),
    ],
    "hospitals": [
        ("no_accredit
ation", "No accredita
tion",
         "Not listed
as accredited", lambd
a r: _blank(r, "accre
ditation")),
    ],
    "doctors": [
        ("newly_quali
fied", "Qualified in
the last 3 years",
         "Registratio
n dated 2024 or late
r",
         lambda r: _r
egistration_year(r) >
= 2024),
    ],
    "stores": [
        ("no_stock",
"No stock recorded",
         "Stock has n
ever been entered",
         lambda r: _b
lank(r, "stock_cs
v")),
    ],
    "medicines": [
        ("rx_only",
"Prescription only",
         "Cannot be s
old over the counte
r",
         lambda r: r.
get("rx_required",
"").strip().lower() i
n {"yes", "true",
"1"}),
        ("otc", "Over
the counter",
         "Sold withou
t a prescription",
         lambda r: r.
get("otc", "").strip
().lower() in {"yes",
"true", "1"}),
    ],
}
```

L3

<code>FACETS</code> — the schema of available facets: dict of registry → list of (flag, label, note, predicate) tuples. 417–426 patients: no_visit (not in the visited set), child (<code>_is_child</code>), senior (<code>_is_senior</code>), no_blood_group (<code>_blank</code>); 427–430 hospitals no_accreditation; 431–435 doctors newly_qualified with <code>_registration_year(r) >= 2024</code>; 436–440 stores no_stock; 441–447 medicines rx_only / otc with <code>.lower() in {"yes", "true", "1"}</code> — an inline set membership test (P3) first: the yes/no cell can be written either way and still read. Note the inline lambdas sitting in the structure — exactly how sorted(key=) would behave, only as named data. The dict closes at 450; 448–451 close and blanks.

452–462

```
def _registration_yea
r(row):
    """The year out o
f a registration numb
er like MMC-2008-2034
16."""
    parts = row.get
("registration_no",
"").split("-")

    for part in part
s:
        if len(part)
== 4 and part.isdigit
():
            return in
t(part)

    return 0
```

L2

<code>_registration_year(row)</code>: 453 docstring explains the target — a year out of an MMC-2008-203416 shape. 454 <code>row.get("registration_no", "").split("-")</code> (P2), 456–458 the first fragment that is <code>len == 4 and isdigit()</code> is the year — cast at return, 460 otherwise 0, meaning unqualifiable-for-2024 and never a crash on a malformed id.

463–469

```
def apply_facet(recor
ds, record_type, fla
g):
    """Return (filter
ed records, facet lis
t with counts, unknow
n flag).

    The facet list is
returned alongside so
the frontend never ha
s to ask a
    second question a
bout how many rows a
tab would show. An em
pty flag
    means no facet, w
hich is not an error.
    """
```

txt

<code>apply_facet(records, record_type, flag)</code> + docstring (464–469): returns (filtered records, facet list with counts, unknown flag). The facet list is delivered with the results so the frontend never needs a second round-trip; an empty flag is not an error — it means no facet.

470–472

```
    defined = FACETS.
get(record_type, [])
    by_flag = {name:
(label, note, test) f
or name, label, note,
test in defined}
```

L3

470: <code>defined = FACETS.get(record_type, [])</code> (P3 <code>.get</code> defaulting to an empty list — registries without facets are legal); 471: <code>by_flag = {name: (label, note, test) for name, label, note, test in defined}</code> — a dict comprehension (P3) indexing facets by flag name.

473–482

```
    available = [
        {
            "flag": n
ame,
            "label":
label,
            "note": n
ote,
            "count":
sum(1 for r in record
s if test(r)),
        }
        for name, lab
el, note, test in def
ined
    ]
```

L3

473–481: <code>available</code> — a list of dicts holding <code>flag</code>, <code>label</code>, <code>note</code> and the count: <code>sum(1 for r in records if test(r))</code> — a generator inside sum(), the same predicate over every row (so the count and the list can never disagree), one per facet.

483–486

```
    unknown = bool(fl
ag) and flag not in b
y_flag

    if unknown or not
flag:
        return record
s, available, unknown
```

L1

483: <code>unknown = bool(flag) and flag not in by_flag</code> — an unknown named flag only exists when a flag was actually requested; 485–486: if unknown or empty, return the unfiltered records alongside the facet counts. Honest outcomes in both directions.

487–490

```
    _label, _note, te
st = by_flag[flag]

    return [r for r i
n records if test
(r)], available, Fals
e
```

L3

488: <code>_label, _note, test = by_flag[flag]</code> — tuple unpacking from the dict (P3); 490: <code>return [r for r in records if test(r)], available, False</code> — the filter itself, plus the counts and a False for the unknown flag. One call gives the page everything it needs.

registry.py 386 lines — Fuzzy search scoring and write validation — strings and dicts, Labs 2-3.

1–10 txt

```
"""Search and validat
ion across the Meridi
an registries.

Taught concepts: dict
.get() .items() .keys
(), list and set comp
rehensions,
sorted(key=lambda), s
tring .strip() .casef
old() .split(), if/el
if chains,
nested loops, len(),
isinstance().

Core rule from spec s
ection 4.2: a blank c
ell means unknown. Bl
ank fields
are skipped when scor
ing, never treated as
a mismatch, because s
coring a
missing value as wron
g would push good rec
ords down the rankin
g.
"""
```

Module docstring. Names the two jobs of this file — fuzzy search scoring and write validation — and lists the taught concepts it is built from (dict methods, comprehensions, <code>sorted(key=lambda)</code>, string methods, <code>if/elif</code>, nested loops, <code>len()</code>, <code>isinstance()</code>: Practical 2–3 material). It also states the rule that shapes every scoring decision: a blank cell means unknown and is skipped, never counted as a mismatch, because scoring a missing value as wrong would push good-but-incomplete records down the ranking.

11–11 txt

Blank line.

12–34 L3

```
FIELD_WEIGHTS = {
    "medicines": {
        "name": 5, "g
eneric": 4, "categor
y": 3, "form": 2,
        "manufacture
r": 2, "storage": 1,
"strength": 2,
    },
    "stores": {
        "name": 5, "t
ype": 3, "area": 2,
"city": 2, "hours":
1,
    },
    "patients": {
        "name": 5, "a
rea": 2, "city": 2,
"blood_group": 2, "ph
one": 1,
        "gender": 1,
"address_line": 1, "e
mergency_contact": 1,
    },
    "hospitals": {
        "name": 5, "t
ype": 3, "area": 2,
"city": 2,
        "address_lin
e": 1, "speciality":
2,
    },
    "doctors": {
        "name": 5, "s
pecialisation": 4, "d
epartment": 3, "hospi
tal_id": 0,
        "qualificatio
n": 2, "room_no": 1,
"languages": 2,
        "registration
_no": 1,
    },
}
```

<code>FIELD_WEIGHTS</code> — a dict of dicts (Practical 3): one inner dict per searchable registry, column name → importance. 13–16 medicines, 17–19 stores, 20–23 patients, 24–27 hospitals, 28–32 doctors, 33 closes the outer dict. These are the search engine's only tuning numbers: a <code>name</code> match (weight 5) outranks a <code>phone</code> match (weight 1). <code>hospital_id</code> gets weight 0 — recordable but never scoreable, matching the id rule below.

35–36	`DEFAULT_WEIGHT = 1`	**L3**	<code>DEFAULT_WEIGHT = 1</code> — the score a field contributes when the registry defines no weight for it, so an unlisted column stays searchable at minimum strength.
37–39	`# Never searched, and never allowed to cont ribute a point.` `ID_SUFFIX = "_id"`	**L2**	Comment stating the policy, then <code>ID_SUFFIX = "_id"</code>, a plain string constant (Practical 2) the scoring loop tests with <code>.endswith()</code>. Ids match every row equally, so a query like “3” would otherwise become “the whole table”.

40–47	`REQUIRED_FIELDS = {`
	`  "patients": ["nam e"],`
	`  "medicines": ["na me"],`
	`  "stores": ["nam e"],`
	`  "hospitals": ["na me"],`
	`  "doctors": ["nam e", "hospital_id"],`
	`}`
		L3	<code>REQUIRED_FIELDS</code> — the columns a create cannot omit (41–45: a name everywhere, plus a hospital for doctors, because a doctor practising nowhere is not recordable). Patients require only a name: ages, phones, addresses may all legitimately be unknown per the blank-means-unknown rule. 46 closes the dict.

48–55	`NUMERIC_FIELDS = {`
	`  "patients": ["ag e"],`
	`  "medicines": ["pr ice"],`
	`  "stores": ["ratin g"],`
	`  "hospitals": ["be ds", "established_yea r"],`
	`  "doctors": ["cons ultation_fee", "exper ience_years"],`
	`}`
		L3	<code>NUMERIC_FIELDS</code> — columns that must parse as numbers when filled (49 age, 50 price, 51 rating, 52 beds + established year, 53–54 fee and experience years). The validation loop below reads this table instead of a hand-written <code>if</code> per field — add a numeric column and the check comes free.

56–59	`# Six digits. Not ran ge checked here becau se a hospital may sit outside`
	`# the city's own pin prefix; verify_seed_d ata.py covers that ca se.`
	`PIN_FIELDS = {"pincod e"}`
		L3	Comment explaining that pin codes are shape-checked (six digits) rather than range-checked, then <code>PIN_FIELDS = {"pincode"}</code> — a set (Practical 3) so membership tests are O(1).

60–63	`VALID_BLOOD_GROUPS = {"A+", "A-", "B+", "B -", "AB+", "AB-", "O +", "O-"}`
	
	`# Spelled out rather than derived by trimm ing the last letter, which would`
	`# break on any plural that is not simply si ngular plus an s.`
		L3	<code>VALID_BLOOD_GROUPS</code> — the eight legal ABO/Rh strings as one closed set literal (Practical 3); the write check reads like English: <code>blood not in VALID_BLOOD_GROUPS</code>. 61 blank; 62–63 the comment defending the next constant against cleverness.

64–72	`ID_FIELDS = {`
	`  "patients": "pati ent_id",`
	`  "visits": "visit_ id",`
	`  "medicines": "med icine_id",`
	`  "stores": "store_ id",`
	`  "diseases": "dise ase_id",`
	`}`
		L3	<code>ID_FIELDS</code> — registry name → its id column (65–69 patients, visits, medicines, stores, diseases; 70) closes it), spelled out instead of derived by trimming an “s” because irregular plurals would break that. Its consumer is the summariser below. 71–72 blank.

73–77	`def normalise(text):`
	`    """Lower case and strip a value so comparisons are reliable."""`
	`    return str(text).strip().casefold()`
		L2	<code>normalise(text)</code> — the one normaliser every comparison in this module runs through. 74 docstring; 75 a three-call chain (Practical 2): <code>str()</code> survives a caller passing a number, <code>.strip()</code> removes padding, <code>.casefold()</code> lowercases more aggressively than <code>.lower()</code>.

78–85	`def query_terms(query):`
	`    """Split a search string into a list of normalised terms."""`
	`    if not query:`
	`        return []`
	
	`    return [normalise(t) for t in query.replace(",", " ").split() if t.strip()]`
		L2	<code>query_terms(query)</code> turns one typed query into normalised words. 79 docstring, 80–81 the empty-query guard returns an empty list (P1 truthiness), 83 the body: a list comprehension replacing commas with spaces (so <code>"a, b"</code> and <code>"a b"</code> are the same query), splitting on any whitespace, normalising each token and dropping the empties.

86–88	`EDGE_CHARS = " -.,;:'\"()"`	L2	<code>EDGE_CHARS</code> holds the punctuation allowed at token ends. This is what lets the search treat <code>(paracetamol)</code> and <code>paracetamol</code> alike.

89–93	`def strip_edges(text):`
	`    """Drop punctuation and padding from both ends of a string."""`
	`    return text.strip(EDGE_CHARS)`
		L2	<code>strip_edges(text)</code>: 90 docstring, 91 <code>text.strip(EDGE_CHARS)</code> removes any of those characters from both ends in one call — the same <code>str.strip</code> method from Practical 2 given a cut set instead of nothing.

94–104	`def is_short_token(term):`
	`    """True for terms too short to match a field by substring.`
	
	`    A bare "A-" scores every address containing "A-3" or "A-4", which is`
	`    how a house number turns into a false blood group match. A token needs`
	`    real content before substring matching means anything, so anything`
	`    shorter than three characters is compared against whole values only.`
	`    """`
	`    return len(strip_edges(term)) < 3`
		L2	<code>is_short_token(term)</code>. Its docstring (95–101) records a real observed failure it prevents: searching "A-" scored every address whose house number contains "A-3" or "A-4". The fix, 102: after stripping punctuation, <code>len(...) < 3</code> marks the token as short, and the scoring loop will then compare it against whole values only. <code>len()</code> is Practical 1.

105–110	`def match_score(record, terms, weights):`
	`    """Score one record against the search terms.`
	
	`    Returns (score, matched_fields). Each field can contribute at most`
	`    once no matter how many terms hit it, because the inner loop breaks.`
	`    """`
		L4	<code>match_score(record, terms, weights)</code> — the core of the search engine, one record at a time. The docstring (106–110) fixes the return shape (score, matched_fields) the UI displays as “matched via name, area”, and documents an invariant a bug once violated: each field scores at most once, no matter how many terms hit it (the inner loop <code>break</code>s).

111–112	`    score = 0` `    matched_fields = []`	L1	The two accumulators every scoring loop needs (Practical 1's accumulator pattern): <code>score = 0</code> and an empty <code>matched_fields</code> list.

113–116	`    for field in record:`
	`        if field.endswith(ID_SUFFIX):`
	`            continue`
		L3	<code>for field in record:</code> — looping a dict yields its keys (Practical 3), i.e. that row's CSV column names in file order. 115–116: any field ending in <code>_id</code> is skipped before it can score.

117–118	`        value = record[field]`	L3	<code>value = record[field]</code> (after the blank line at 117) — plain dict lookup by key, Practical 3.
119–121	`        if value == "" or value is None:` `            continue`	L1	The blank-means-unknown rule enforced where it matters: 120–121 skip an empty cell, contributing nothing and never subtracting. Both the <code>== ""</code> test and <code>is None</code> are Practical 1 operators.
122–123	`        value = normalise(value)`	L2	<code>value = normalise(value)</code> — both sides of every comparison normalised, so case and padding cannot hide a match. Reusing a small helper beats duplicating the chain at every comparison site.

124–127	`        for term in terms:`
	`            if term not in value:`
	`                continue`
		L2	<code>for term in terms:</code> — the inner loop of the nested pair (Practical 4), with 126–127 the matching predicate itself: <code>term not in value</code>, the substring test from Practical 2, <code>continue</code>-ing on a miss. Fuzzy substring matching, not equality, is the search's character.

128–131	`            # A short token has to match the whole value rather than a`
	`            # slice of it, so "A-" hits a blood group of "A-" and never`
	`            # an address that begins "A-3".`
		txt	Blank line, then the three-line comment restating the short-token rule: 129–131 spell out that "A-" may hit a blood group of "A-" but never an address beginning "A-3".

132–133	`            if is_short_token(term) and strip_edges(value) != strip_edges(term):` `                continue`	L1	<code>if is_short_token(term) and strip_edges(value) != strip_edges(term): continue</code> — short tokens must equal the whole stripped value. <code>and</code> of two conditions (P1), <code>strip_edges</code> reused (L4 function reuse).

134–137	`            score += weights.get(field, DEFAULT_WEIGHT)`
	`            matched_fields.append(field)`
	`            break`
		L3	On an actual hit: 135 <code>score += weights.get(field, DEFAULT_WEIGHT)</code> — dict <code>.get</code> with a default (Practical 3) and the <code>+=</code> accumulator (P1); 136 remember which field scored; 137 <code>break</code> leaves the term loop, enforcing the one-score-per-field invariant from the docstring (without it, one field hitting four terms would quadruple-count).

138–139	`    return score, matched_fields`	L4	<code>return score, matched_fields</code> (after the blank at 138) — a tuple return (Practical 2): two results, no side effects; the signature discipline the whole module keeps.
140–141		txt	Blank lines.

142–149	`def search_records(records, query, record_type):`
	`    """Return matching records sorted best first.`
	
	`    Every record scoring above zero is returned. Nothing is discarded for`
	`    being weak, because an empty result set reads as a broken search when`
	`    the table simply holds three medicines and none is the one asked for.`
	`    The caller decides what to dim.`
	`    """`
		L4	<code>search_records(records, query, record_type)</code> + docstring (143–149) carrying a deliberate policy: every record scoring above zero is returned, weak or not. When the catalogue holds three medicines and none is the one asked for, an empty page reads as a broken search; the caller decides what to dim. Filtering, ranking and display policy stay in separate layers.

150–153	`    terms = query_terms(query)`
	
	`    if not terms:`
	`        return []`
		L1	150 parses the query once; 152–153 an empty query yields nothing rather than the whole table. (The route already rejects a blank query with a 400 before this runs — both layers agree, defence in depth.)

154–155	`    weights = FIELD_WEIGHTS.get(record_type, {})`	L3	After the blank at 154: <code>weights = FIELD_WEIGHTS.get(record_type, {})</code> — table lookup defaulting to an empty dict, so an unweighted registry is still searchable.
156–158	`    results = []`	L3	<code>results = []</code> — the result accumulator (Practical 1), separated from the loop by blank lines.

159–162	`    for record in records:`
	`        score, fields = match_score(record, terms, weights)`
	
	`        if score > 0:`
		L4	The scoring loop opens: 159 every record, 160 scored by match_score, 162 <code>if score > 0:</code> the positive scores are the only survivors.

163–166	`            enriched = dict(record)`
	`            enriched["score"] = score`
	`            enriched["matched_fields"] = fields`
	`            results.append(enriched)`
		L3	Enrichment: 163 <code>dict(record)</code> copies the row (a dict of an existing dict) so the original data is never mutated, then 164–166 attach the score and the matched field names as new keys. New fields computed on a copy, a small purity win.

167–169	`    results.sort(key=lambda r: r["score"], reverse=True)` `    return results`	L3	168: <code>results.sort(key=lambda r: r['score'], reverse=True)</code> — the Practical 3 sorted(key=lambda) pattern in its mutation form, best first; 169 returns the whole ranked list.
170–171		txt	Blank lines.

172–177	`def search_by_id(records, id_field, target):`
	`    """Exact id lookup, the opposite of fuzzy search."""`
	`    for record in records:`
	`        if record[id_field] == target:`
	`            return record`
	`    return None`
		L4	<code>search_by_id(records, id_field, target)</code> — the exact counterpart of fuzzy search. Linear scan (174–176), <code>return None</code> on miss (177). A CSV registry has no index, so an honest O(n) scan is the right tool at hundreds of rows.

178–179		txt	Blank lines.

180–185	`def stock_set_for(store):`
	`    """Return the set of medicine ids a store holds.`
	
	`    A blank stock cell means not recorded, not stocking nothing, so it`
	`    returns an empty set and the caller skips the store.`
	`    """`
		L4	<code>stock_set_for(store)</code> + docstring (181–185) with a nuance worth repeating: a blank stock cell means not recorded, not “stocks nothing” — this function returns an empty set and the caller must skip the store, so absence of data can never inflate into the claim that a store is empty.

186–187	`    raw = store.get("stock_csv", "")`	L3	186 reads the cell defensively: <code>store.get("stock_csv", "")</code> — dict <code>.get</code> with a default, because a fixture row may lack the column.
188–189	`    if not raw:` `        return set()`	L3	The unknown case: blank → <code>set()</code>, the empty set (Practical 3) used as the “unknown” value the caller checks for.
190–191	`    return {s.strip() for s in raw.split("|") if s.strip()}`	L3	<code>{s.strip() for s in raw.split("|") if s.strip()}</code> — the pipe-split set comprehension (Practical 3) every multi-value cell in the schema shares (symptoms, stock, languages).

192–193		txt	Blank lines.

194–197	`def stores_with_medicine(medicine_name, medicines, stores):`
	`    """Return every store stocking a medicine, matching name or generic."""`
	`    target = normalise(medicine_name)`
		L4	<code>stores_with_medicine(medicine_name, medicines, stores)</code> + docstring — answers “where can this drug be bought?”. 196 normalises the query once, up front.

198–200	`    if not target:` `        return []`	L1	Empty target → empty result without touching stores (198–199): one shared early-exit shape used by every helper.
201–202	`    wanted = set()`	L3	<code>wanted = set()</code> — the accumulator as a set, because the interesting test to come is a set operation.

203–206	`    for m in medicines:`
	`        if target in normalise(m.get("name", "")) or \`
	`           target in normalise(m.get("generic", "")):`
	`            wanted.add(m["medicine_id"])`
		L3	Build <code>wanted</code>: for every medicine, 204–205 test the query against its name or its generic (both normalised; the backslash is a line continuation), and 206 add its id. Afterwards <code>wanted</code> is the id set of every plausibly matching medicine.

207–209	`    if not wanted:` `        return []`	L1	Nothing matched in the catalogue → no store can stock it: bail out before the store loop (208–209).
210–212	`    matches = []`	L1	<code>matches = []</code> — the result accumulator, then a blank line before the second loop.

213–217	`    for s in stores:`
	`        held = stock_set_for(s)`
	
	`        if not held:`
	`            continue`
		L4	The store loop: 213 each store, 214 its stock set, 216–217 skip stores whose stock is unknown — the docstring nuance from stock_set_for honoured at the call site, so an unrecorded store is never listed as stocking the medicine.

218–221	`        if held & wanted:` `            matches.append(s)`	L3	219: <code>if held & wanted:</code> — the set intersection operator from Practical 3, the same operator shape the triage engine uses on symptoms. Any shared id means the store stocks a match, so 220 appends the store.
222–224	`    return matches`	L4	<code>return matches</code>, in file order; the detail view counts them and the medicine page lists them. 223–224 blank.

225–227	`def medicines_for_disease(disease_name, medicines, diseases):`
	`    """Return catalogue entries whose name matches a disease's medication."""`
	`    target = normalise(disease_name)`
		L4	<code>medicines_for_disease(disease_name, medicines, diseases)</code> + docstring: for one disease name, find catalogue entries plausibly backing its written <code>medication</code> cell. 227 normalises the name once.

228–230	`    found = []` `    seen = set()`	L3	Two accumulators: <code>found</code> (list of results) and <code>seen</code> (set) — the set exists purely to make the dedup test O(1) (Practical 3).

231–234	`    for d in diseases:`
	`        if normalise(d["name"]) != target:`
	`            continue`
		L4	Outer loop over diseases (232), with 233–234 skipping every disease whose name is not equal to the target — equality, deliberately not substring, because names are keys here, not prose.

235–236	`        med = d.get("medication", "")`	L3	<code>med = d.get("medication", "")</code> — the written medication string read defensively with a default (L3).
237–239	`        for m in medicines:` `            key = m["medicine_id"]`	L3	Inner loop over medicines: 238 iterate, 239 take the candidate's id as the dedup key.

240–242	`            if key in seen:` `                continue`	L3	The dedup: 241–242 skip ids already <code>seen</code>, which stops the same medicine being appended once per disease row that mentions its generic.
243–245	`            if normalise(m.get("name", "")).startswith(normalise(med)) or \` `                normalise(med) in normalise(m.get("generic", "")):`	L3	The match test: the catalogue name must start with the medication (<code>.startswith()</code>, Practical 2) or the medication must appear in its generic. That slop is what catches “Metformin 500mg” written in the disease row as plain “metformin”.
246–247	`                seen.add(key)` `                found.append(m)`	L3	On a hit: mark the id seen and append the whole medicine row — the accumulated result is a list of full records, ready to render.
248–249	`    return found`	L4	<code>return found</code> (after the blank line created by the loop exits) — plain list return, loops fully unwound.
250–251		txt	Blank lines.

252–262	`def validate_record(record_type, payload, require_required=True):`
	`    """Check a record before writing it.`
	
	`    Returns (cleaned, errors). cleaned is safe to store even when errors`
	`    is non-empty, so one bad field does not block everything else.`
	`    Every problem is reported at once rather than one per round trip.`
	
	`    require_required is False for partial updates, where sending only a`
	`    city must not fail because no name was included. Format checks always`
	`    run, so age="old" is still caught either way.`
	`    """`
		L4	<code>validate_record(record_type, payload, require_required=True)</code> — the write-side gatekeeper every POST and PATCH runs through. The docstring (253–262) fixes the contract: deliver (cleaned, errors) where cleaned is safe to store even when errors exist (one bad field must not block the other nine), every problem is reported in one pass rather than one per round-trip, and require_required separates creates (name needed) from partial updates (a one-city PATCH must not demand a name) while the format checks always run.

263–264	`    errors = []` `    cleaned = {}`	L1	<code>errors = []</code> and <code>cleaned = {}</code> — the pair of accumulators the entire function fills.

265–274

```
    for key, value in payload.items():
        if value is None:
            cleaned[key] = ""
        elif isinstance(value, (int, float)):
            cleaned[key] = str(value)
        elif isinstance(value, (list, tuple)):
            cleaned[key] = "|".join(str(v).strip() for v in value if str(v).strip())
        else:
            cleaned[key] = str(value).strip()
```

L2

The cleaning loop (after the blank at 265), one <code>if/elif/else</code> branch per value shape (Practical 1's chain, <code>isinstance</code> the taught type test): 267–268 None → "" (missing means blank, the house rule), 269–270 numbers become strings (a CSV cell is always text), 271–272 lists/tuples are joined with pipes, the schema's multi-value convention, 273–274 everything else trimmed as a string. After this loop the payload is safe CSV-shaped data no matter what the JSON sent.

275–279

```
    if require_required:
        for field in REQUIRED_FIELDS.get(record_type, []):
            if not cleaned.get(field, ""):
                errors.append(f"{field} is required")
```

L3

Required-field check, only when creating: loop the registry's REQUIRED_FIELDS (277) and 278–279 fail an empty one with <code>f"{field} is required"</code>. The <code>.get(field, "")</code> pattern keeps it to two lines.

280–285

```
    for field in NUMERIC_FIELDS.get(record_type, []):
        raw = cleaned.get(field, "")

        if raw == "":
            continue
```

L4

Numeric validation: 281 loops NUMERIC_FIELDS, 282 fetches the cell, 284–285 skip the blank cell — blank stays legal, per the house rule that unknown is not an error.

286–291

```
        try:
            value = float(raw)
        except ValueError:
            errors.append(f"{field} must be a number, got {raw!r}")
            continue
```

L1

287–291: the cast sits inside <code>try/except ValueError</code> (the P1 cast pattern with its failure path written down): a non-number appends <code>f"{field} must be a number, got {raw!r}"</code> — the <code>!r</code> conversion (Lab 5 formatting) shows quotes so a whitespace problem is visible inside the error — then <code>continue</code>s to the next field.

292–294

```
        # Beds, fees and years have sensible floors. Rejecting a negative
        # bed count here is cheaper than finding it in a chart later.
```

txt

Comment justifying the next check: rejecting a negative bed count here is cheaper than finding it in a chart later.

295–296

```
        if value < 0:
            errors.append(f"{field} cannot be negative, got {raw!r}")
```

L1

<code>if value < 0</code> appends its own message — numeric range enforcement at the door.

297–302	`    for field in PIN_FIELDS:`
	`        raw = cleaned.get(field, "")`
	
	`        if raw and (not raw.isdigit() or len(raw) != 6):`
	`            errors.append(f"{field} must be six digits, got {raw!r}")`
		L2	Pin check: 298–299 loop and fetch, 301–302 fail unless <code>raw.isdigit() and len(raw) == 6</code> — the shape-validation pattern from Practical 2's string methods plus <code>len()</code> (P1).

303–305	`    if record_type == "patients":` `        blood = cleaned.get("blood_group", "")`	L4	Patient-specific rules open with <code>if record_type == "patients"</code>: 305 reads the blood group cell.

306–311	`        if blood and blood not in VALID_BLOOD_GROUPS:`
	`            errors.append(`
	`                f"blood_group must be one of {sorted(VALID_BLOOD_GROUPS)},"`
	`                f" got {blood!r}"`
	`            )`
		L3	The blood-group check (307–311): reject values outside VALID_BLOOD_GROUPS, formatting the sorted legal set into the message (<code>sorted(VALID_BLOOD_GROUPS)</code>, P3) so the fix is visible without opening the source.

312–314	`        gender = cleaned.get("gender", "").casefold()`	L2	313 normalises gender in place; 314 blank.
315–316	`        if gender and gender not in {"male", "female", "other"}:` `            errors.append(f"gender must be male, female or other, got {gender!r}")`	L3	Membership against an inline set <code>{"male", "female", "other"}</code> (L3) with the offending value quoted in the error.
317–319	`        age = cleaned.get("age", "")`	L3	<code>age = cleaned.get("age", "")</code> — the same fetch-with-default pattern as every other cell.
320–321	`        if age:` `            try:`	L1	<code>if age:</code> then <code>try:</code> — a truthiness guard around a cast, the two practicals meeting in three lines.
322–323	`                if not (0 <= float(age) <= 120):` `                    errors.append(f"age must be between 0 and 120, got {age!r}")`	L1	<code>if not (0 <= float(age) <= 120):</code> — a chained comparison (Practical 1) testing a whole range in one expression, with the off-range error ready at 323.
324–325	`            except ValueError:` `                pass`	L1	<code>except ValueError: pass</code> — swallows the cast failure because the numeric check above already reported it; one error per problem, no double reporting.
326–327	`    if record_type == "medicines":`	txt	Blank line, then the medicine-specific gate opens.
328–329	`        for flag in ("otc", "rx_required"):` `            raw = cleaned.get(flag, "").casefold()`	L2	The two yes/no columns are looped (328) and casefolded (329) before the test.
330–332	`            if raw and raw not in {"yes", "no"}:` `                errors.append(f"{flag} must be yes or no, got {raw!r}")`	L2	Reject anything other than yes/no (331–332). Worth validating because the UI's OTC/Rx chips read these flags.

333–334	`    if record_type == "doctors":`	txt	Blank, then the doctor gate opens.
335–336	`        hospital = store_get_by_id("hospitals", cleaned.get("hospital_id", ""))`	L4	Does the declared hospital exist? <code>store_get_by_id</code> (defined below, deliberately not imported at the top) answers.

337–340	`        if cleaned.get("hospital_id") and hospital is None:`
	`            errors.append(`
	`                f"hospital_id {cleaned['hospital_id']} does not exist")`
		L1	A hospital_id naming no row is an error (337–339) — referential integrity enforced at the door, letting a doctor record exist only when its hospital does.

341–342	`        registration = cleaned.get("registration_no", "")`	L2	<code>registration</code> fetched, blank line before the format check.

343–346	`        if registration and not registration.upper().startswith("MMC-"):`
	`            errors.append(`
	`                f"registration_no should look like MMC-YYYY-NNNNNN,"`
	`                f" got {registration!r}")`
		L2	343–346: must start with MMC- (<code>.upper().startswith()</code>, P2 chains) and the error shows the expected shape MMC-YYYY-NNNNNN.

347–348	`    if record_type == "stores":`	txt	Blank, then the store gate.

349–356	`        held = cleaned.get("stock_csv", "")`
	
	`        if held:`
	`            for mid in held.split("|"):`
	`                if mid and not mid.startswith("M-"):`
	`                    errors.append(`
	
	`f"stock_csv must contain medicine ids like M-01, got {mid!r}"`
	`                    )`
		L2	Stock validation: 349 fetch the cell, 351–353 when filled, split on pipes and reject any fragment not starting with M- — the cell holds medicine ids, and a stray word left in it would poison every set intersection computed from it later.

357–357		txt	Blank line closing the per-record-type rules.
358–360	`    return cleaned, errors`	L4	<code>return cleaned, errors</code> — the promised tuple: storeable data plus every problem found, totalled.

361–366	`def store_get_by_id(record_type, target):`
	`    """Look a record up without importing store at module load time.`
	
	`    store imports registry, so importing it here at the top would be`
	`    circular. Resolved lazily instead.`
	`    """`
		L4	<code>store_get_by_id(record_type, target)</code> + docstring (362–366) explaining the lazy import: store.py imports registry at its top, so registry importing store at its top would be a circular import and crash; resolved by deferring one direction to call time — a pattern worth memorising.

367–369	`    if not target:` `        return None`	L4	Empty target → None without importing anything (367–368): the cheap case pays no dependency cost.

370–371	`    import store`	**L4**	<code>import store</code> inside the function — the deferred import itself, guaranteed consistent no matter how many module-level import orders the codebase tries.
372–374	`    return store.get_by_id(record_type, target)`	**L4**	The delegated lookup (372); 373–374 blank.

375–377	`def summarise_results(results, record_type):`
	`    """Return a short label per result for list views."""`
	`    id_field = ID_FIELDS.get(record_type, "id")`
		L3	<code>summarise_results(results, record_type)</code>: one text line per result for plain listings and the tests. 376 docstring, 377 resolves the id column via ID_FIELDS with a fallback.

378–380	`    lines = []`	**L1**	<code>lines = []</code> — the line accumulator, blank lines bracketing it.

381–384	`    for r in results:`
	`        name = r.get("name", "(unnamed)")`
	`        matched = ", ".join(r.get("matched_fields", []))`
	`        lines.append(f"{r.get(id_field, '')} {name} [score {r['score']}] via {matched}")`
		L5	Per result: 381 iterate, 382 name with an <code>(unnamed)</code> fallback, 383 matched fields joined into one <code>"name, area"</code> string, 384 one f-string assembling id, name, score and the via-list — Lab 5 formatted output.

385–386	`    return lines`	**L4**	<code>return lines</code> — the list the route turns into JSON and the tests print.

triage.py 473 lines — Set algebra over symptoms, severity bands, report — Labs 3 and 5.

1–12 txt

```
"""Symptom triage for
Meridian.

Taught concepts: set
intersection and diff
erence (& -), dicts,
list and set
comprehensions, sorte
d(key=lambda), while
loops, string .strip
() and
.casefold(), f-string
s, default arguments.

Pure functions only.
Nothing here touches
Flask or writes file
s, so the
whole module is testa
ble on its own.

Scoring is set algebr
a pointed at symptom
s. It is the Practica
l 3 Venn
problem (comedy, fant
asy, romance) with fi
lms replaced by disea
ses.
"""
```

Module docstring. Three things are settled here: the taught concepts it uses (set intersection & difference, dicts, comprehensions, <code>sorted(key=lambda)</code>, <code>while</code> loops, <code>casefold</code>, f-strings, default arguments), the design rule (pure functions only — nothing touches Flask or writes files, which is what makes the whole module testable on its own), and the intellectual ancestor: scoring is set algebra pointed at symptoms — the Practical 3 Venn problem with films replaced by diseases.

13–15 L2

```
DISCLAIMER = "NOT A D
IAGNOSIS. TRIAGE GUID
ANCE ONLY."
```

<code>DISCLAIMER</code> (after the blank at 13) — the mandatory “NOT A DIAGNOSIS” string every human-facing output ends with. A constant, because it must be identical everywhere it is printed.

16–23 L3

```
SEVERITY_RANK = {
    "low": 0,
    "moderate": 1,
    "elevated": 2,
    "high": 3,
    "severe": 4,
}
```

<code>SEVERITY_RANK</code> — severity word → number (17–21: low 0 through severe 4). Numbers exist so severities can be ordered by a sort key; 23 blank.

24–28 L3

```
# Two diseases scorin
g within this distanc
e of each other are t
reated as
# tied, and severity
breaks the tie. Wider
than this and a sligh
tly better
# score would hide a
genuinely more urgent
condition.
BAND_WIDTH = 0.05
```

Comment (24–26) states what <code>BAND_WIDTH = 0.05</code> (27) is for: two diseases within this distance score close enough to be treated as tied, and severity breaks the tie. 28 blank.

29–31 L3

```
COVERAGE_WEIGHT = 0.6
PRECISION_WEIGHT = 0.
4
```

The two scoring weights: coverage (0.6) and precision (0.4). Two named constants instead of bare numbers make the formula on line 252 read the way the comment says.

32–43

```
# The penalty for sym
ptoms a disease canno
t account for, as a f
raction of
# the report rather t
han a flat charge per
symptom.
#
# A flat charge was d
efensible against a 1
5 condition knowledge
base, where
# any unmatched sympt
om was strong evidenc
e against. It is not
defensible
# against 95. A flat
0.5 per symptom drove
every score to zero a
s soon as
# a patient listed si
x or seven symptoms,
so a textbook cold ra
nked
# alongside pneumonia
at 0% and the orderin
g became arbitrary. C
harging a
# share of the report
instead keeps a well
explained long report
scoring
# above a badly expla
ined short one, which
is the actual distinc
tion.
EXTRA_PENALTY = 0.5
```

L3

The longest comment in the file, defending <code>EXTRA_PENALTY = 0.5</code> (42): the penalty charges the disease for the share of the report it cannot explain, as a fraction — not a flat fee per symptom. A flat fee was defensible for 15 conditions but drove every score to zero on a 95-condition base once a patient listed six or seven symptoms, making a textbook cold rank like pneumonia. Charging a share keeps a well-explained long report above a badly-explained short one. 43 blank.

44–46

```
MIN_MATCHED_SYMPTOMS
= 2
MAX_RESULTS = 5
```

L1

<code>MIN_MATCHED_SYMPTOMS = 2</code> and <code>MAX_RESULTS = 5</code> — the two policy thresholds the ranking function uses: at least this many shared symptoms, at most this many results.

47–50

```
# The score is a matc
h strength, not a pro
bability. Reporting t
wo common
# symptoms can score
above 50% simply beca
use precision is hig
h, yet the
# evidence is thin. T
his label lets the ca
ller say how much the
number is
# worth, so "55% from
2 of 8 symptoms" is n
ot read as a 55% chan
ce.
```

txt

Comment above evidence_strength: the score is a match strength, not a probability, so a thin match can still score high; a label exists to say how much the number is worth.

51–57

```
def evidence_strength
(matched_count, total
_symptoms=0):
    """Label how much
the matched symptom c
ount is actually wort
h.

    strong    5 or mo
re symptoms matched
    moderate  3 to 4
matched
    weak      2 match
ed, the minimum we ac
cept as evidence
    """
```

L4

<code>evidence_strength(matched_count, total_symptoms=0)</code> + docstring (52–57): the bands are strong (5+), moderate (3–4), weak (2, the minimum we accept). The second argument is a default parameter (Practical 4) kept unused on purpose so callers may pass context.

58–61	`    if matched_count >= 5:`
	`        return "strong"`
	`    if matched_count >= 3:`
	`        return "moderate"`
		L1	The two <code>if … return</code> comparisons (58–59 strong at 5, 60–61 moderate at 3) — the Practical 1 multiway-branch problem in miniature.

62–64	`    return "weak"`	L4	Everything below 3 is <code>weak</code>; 63–64 blank between this and the synonyms table.

65–72	`# A patient does not use the vocabulary in the knowledge base. They say`
	`# "tired", "loose motions", "sugar", "cannot sleep". Without folding those`
	`# into the canonical terms, set intersection finds nothing and the tool`
	`# looks broken rather than merely literal.`
	`#`
	`# Keys are what a person might type. Values are terms that appear in`
	`# scripts/build_disease_kb.py VOCABULARY, so every value here is checked`
	`# against the knowledge base by test_symonym_targets_are_real_symptoms.`
		txt	Comment block explaining why <code>SYNONYMS</code> exists: patients say “tired”, “loose motions”, “sugar” — without folding those onto the knowledge base vocabulary, the tool looks broken rather than merely literal. 69–72 add a testability claim: every value here is checked against the vocabulary by a test, so a mistyped synonym cannot silently match nothing.

73–172

```
SYNONYMS = {
    "tired": "fatigue",
    "tiredness": "fatigue",
    "exhausted": "fatigue",
    "no energy": "fatigue",
    "weakness": "weakness",
    "lethargy": "weakness",
    "loose motions": "diarrhoea",
    "loose stool": "diarrhoea",
    "diarrhea": "diarrhoea",
    "runs": "diarrhoea",
    "stomach upset": "abdominal pain",
    "stomach ache": "abdominal pain",
    "tummy ache": "abdominal pain",
    "belly pain": "abdominal pain",
    "cannot sleep": "sleep disturbance",
    "can't sleep": "sleep disturbance",
    "insomnia": "sleep disturbance",
    "sugar": "excessive thirst",
    "sugar patient": "excessive thirst",
    "thirsty": "excessive thirst",
    "pissing a lot": "frequent urination",
    "urinating often": "frequent urination",
    "blood in urine": "passing blood in urine",
    "blood in stool": "blood in stool",
    "heart burn": "heartburn",
    "acidity": "heartburn",
    "indigestion": "indigestion",
    "giddiness": "dizziness",
    "vertigo": "dizziness",
    "fainting": "fainting",
    "blackout": "fainting",
    "fit": "seizure",
    "convulsion": "seizure",
    "seizures": "seizure",
    "memory loss": "memory loss",
    "forgetful": "memory loss",
    "cannot concentrate": "poor concentration",
    "lack of concentr
```

L3

<code>SYNONYMS</code> — a 100-entry dict (Practical 3) mapping colloquial phrase → canonical symptom: tired/exhausted → fatigue, loose motions/diarrhea → diarrhoea, fit/convulsion → seizure, sugar patient → excessive thirst, and so on through the whole readable surface of the tool. It is data, not logic — which is exactly why it can sit at the top of the file where non-programmers can edit it.

```
ation": "poor concent
ration",
    "depressed": "low
mood",
    "depression": "lo
w mood",
    "sadness": "low m
ood",
    "feeling low": "l
ow mood",
    "worry": "anxiet
y",
    "anxious": "anxie
ty",
    "panic": "anxiet
y",
    "itch": "itchin
g",
    "itching": "itchi
ng",
    "rash": "rash",
    "skin rash": "ras
h",
    "blocked nose":
"nasal congestion",
    "congestion": "na
sal congestion",
    "running nose":
"runny nose",
    "sneezing": "snee
zing",
    "sore throat": "s
ore throat",
    "throat pain": "s
ore throat",
    "earache": "ear p
ain",
    "hearing proble
m": "hearing loss",
    "cannot hear": "h
earing loss",
    "blurred vision":
"blurred vision",
    "vision blur": "b
lurred vision",
    "chest pain": "ch
est pain",
    "tightness in che
st": "chest tightnes
s",
    "palpitations":
"palpitations",
    "heart racing":
"palpitations",
    "breathless": "br
eathlessness",
    "shortness of bre
ath": "breathlessnes
s",
    "wheezing": "whee
zing",
    "noisy breathin
g": "wheezing",
    "vomiting": "vomi
ting",
    "throwing up": "v
omiting",
    "nausea": "nause
a",
    "feeling sick":
"nausea",
    "joint pain": "jo
int pain",
    "joint pains": "j
oint pain",
    "body ache": "bod
y ache",
    "bodyache": "body
ache",
```

```
    "muscle ache": "m
uscle ache",
    "back pain": "bac
k pain",
    "lower back": "lo
wer back pain",
    "backache": "back
pain",
    "weight loss": "w
eight loss",
    "lost weight": "w
eight loss",
    "weight gain": "w
eight gain",
    "gained weight":
"weight gain",
    "no appetite": "l
oss of appetite",
    "lost appetite":
"loss of appetite",
    "fever": "fever",
    "high temperatur
e": "high fever",
    "high fever": "hi
gh fever",
    "shivering": "chi
lls",
    "chills": "chill
s",
    "night sweats":
"night sweats",
    "sweating": "swea
ting",
    "yellowish skin":
"jaundice",
    "yellow eyes": "j
aundice",
    "piles": "blood i
n stool",
    "burning while ur
inating": "painful ur
ination",
    "painful urinatio
n": "painful urinatio
n",
}
```

173–174		**txt**	Blank lines.

175–180	`def canonical_symptom(term):`
	`    """Fold one reported symptom onto the knowledge base vocabulary."""`
	`    if term in SYNONYMS:`
	`        return SYNONYMS[term]`
	
	`    return term`
		L3	<code>canonical_symptom(term)</code>: a dict-membership test (177–178) with the entry itself as the return value, and the raw term at 180 if it is already canonical. The whole normalisation vocabulary reduced to one <code>if</code>.

181–182		**txt**	Blank lines.

183–189	`def normalise_symptoms(symptoms):`
	`    """Turn a list or string of symptoms in to a clean set.`
	
	`    Accepts ["Fever", " cough "] or "fever|cough" or "fever, cough" and`
	`    returns {"fever", "cough"}. Blank entries are dropped and reported`
	`    phrasings are folded onto the vocabulary via SYNONYMS.`
	`    """`
		L4	<code>normalise_symptoms(symptoms)</code> + docstring (184–189) stating the accepted inputs: a list like <code>["Fever", " cough "]</code> or a string like <code>"fever|cough"</code> or <code>"fever, cough"</code>, all returning the set <code>{"fever", "cough"}</code> with blanks dropped and synonyms folded in.

190–193	`    if isinstance(symptoms, str):`
	`        parts = symptoms.replace("|", ",").split(",")`
	`    else:`
	`        parts = symptoms`
		L4	<code>isinstance</code> is the type test (a Practical 4 concept): a string is split on both possible delimiters — pipes replaced by commas first (191), so one <code>.split(",")</code> handles both storage formats; any other input is already a list (193).

194–198	`    return {canonical_symptom(str(p).strip().casefold())` `            for p in parts if str(p).strip()}`	**L3**	The set comprehension: 195–196 strip and casefold each part, fold it through canonical_symptom, and drop the empties. The dedup for free: a set. 197–198 blank.

199–204	`def load_diseases(path="data/diseases.csv"):`
	`    """Read diseases.csv and attach a symptom set to each row.`
	
	`    The symptoms cell is pipe delimited, so splitting is a single call.`
	`    See spec section 4.4 for why commas cannot be used here.`
	`    """`
		L4	<code>load_diseases(path="data/diseases.csv")</code> + docstring: the one impure function in the module (it reads a file), kept together with its default argument (P4) so both triage and the regression tests can call the same code.

205–205	`    import csv`	**X:csv**	<code>import csv</code> — the <code>csv</code> module, beyond the labs, documented in docs/concepts/csv-module.md. Imported inside the function, so the pure side of the module loads without it.
206–208	`    diseases = []`	**L4**	<code>diseases = []</code> — the accumulator, then blank.
209–210	`    with open(path, "r", newline="") as f:` `        for row in csv.DictReader(f):`	**X:csv**	209 <code>open(path, "r", newline="")</code> is the Practical 4 open-in-read-mode pattern (with the csv module's newline convention), 210 <code>csv.DictReader(f)</code> streams rows as dicts — csv module again, explained in docs/concepts/csv-module.md.

211–212	`            row["symptom_set"] = {s.strip().casefold()`
	
	`for s in row["symptoms"].split("|") if s.strip()}`
		L3	Per row, the symptom string is parsed into a set: split on pipes (Practical 2), strip and casefold each fragment, keep the truthy ones. The set algebra below needs exactly this shape.

213–217	`            row["contraindication_set"] = {`
	`                c.strip().casefold()`
	`                for c in (row.get("contraindications") or "").split("|")`
	`                if c.strip()`
	`            }`
		L3	The contraindications cell gets the same treatment, with one addition: <code>(row.get("contraindications") or "")</code> guards a missing key by substituting an empty string before splitting — an unknown cell yields an empty set, never an error.

218–220	`            diseases.append(row)`
	
	`    return diseases`
		L4	<code>diseases.append(row)</code> collects each enriched row; 220 returns the list. One pass, no copies of the file in memory beyond the rows themselves.

221–222		txt	Blank lines.

223–240	`def score_disease(patient_symptoms, disease):`
	`    """Score one disease against the reported symptoms.`
	
	`    Returns (score, coverage, precision, matched_count, missing_count).`
	
	`    coverage   how much of the disease's symptom list the patient shows`
	`    precision  how much of the patient's report the disease explains`
	`    penalty    charged for the share of the report left unexplained`
	
	`    Three numbers rather than one, because a single ratio is misleading in`
	`    both directions. A patient listing 14 symptoms should not match a`
	`    common cold on 4 of its 5; a patient reporting only fever, fatigue and`
	`    cough should not be told nothing is wrong. precision punishes the`
	`    first, coverage rescues the second.`
	
	`    The score is clamped to [0, 1] so a long list of unrelated symptoms`
	`    produces 0 rather than a negative percentage.`
	`    """`
		txt	def score_disease + its long docstring — the module's intellectual core. Returns five numbers: score, coverage, precision, matched count, missing count. Coverage = the share of the disease's symptoms the patient shows; precision = the share of the report the disease explains; a single ratio misleads in both directions — a patient listing 14 symptoms must not match a cold on 4 of its 5 (precision punishes that), a patient reporting only fever, fatigue and cough must not be told nothing is wrong (coverage rescues that). The score clamps to [0, 1].

241–242	`    if not patient_symptoms:` `        return 0.0, 0.0, 0.0, 0, len(disease["symptom_set"])`	L1	The empty-report guard: with no symptoms, score everything zero and return immediately — anything else would need set operations on an empty set and divide by zero len.

243–243		txt	Blank line.

244–246	`    matched = patient_symptoms & disease["symptom_set"]`
	`    extra = patient_symptoms - disease["symptom_set"]`
	`    missing = disease["symptom_set"] - patient_symptoms`
		L3	The set algebra, all three operators in one place: 244 <code>matched = patient_symptoms & disease["symptom_set"]</code> (intersection, Practical 3), 245 <code>extra = patient_symptoms - disease["symptom_set"]</code> (symptoms the disease cannot account for), 246 <code>missing = disease["symptom_set"] - patient_symptoms</code> (symptoms the disease expects but were not reported). This is the Practical 3 Venn problem.

247–247		txt	Blank line.

248–250	`    coverage = len(matched) / len(disease["symptom_set"])`
	`    precision = len(matched) / len(patient_symptoms)`
	`    penalty = EXTRA_PENALTY * (len(extra) / len(patient_symptoms))`
		L5	Three ratios: coverage (248) and precision (249) are len(matched) divided by the relevant side's size; 250 <code>penalty = EXTRA_PENALTY * (len(extra) / len(patient_symptoms))</code> — a charge proportional to the unexplained share, never a flat fee per symptom (lines 32–42 explain why).

251–251		txt	Blank line.
252–253	`    score = (coverage * COVERAGE_WEIGHT) + (precision * PRECISION_WEIGHT) - penalty` `    score = max(0.0, min(1.0, score))`	L3	252: the weighted score — <code>(coverage * COVERAGE_WEIGHT) + (precision * PRECISION_WEIGHT) - penalty</code>; 253: <code>max(0.0, min(1.0, score))</code> clamps into [0, 1] so unrelated symptoms produce 0, never a negative percentage.
254–254		txt	Blank line.
255–255	`    return score, coverage, precision, len(matched), len(missing)`	L4	<code>return</code> of the five-value tuple — score, coverage, precision, matched count, missing count. Tuple return, no side effects.
256–257		txt	Blank lines.
258–258	`def apply_severity_bands(results):`	L4	<code>def apply_severity_bands(results)</code> opens the banding stage.

259–269	`    """Sort by score, then let severity break near ties.`
	
	`    Severity is deliberately not multiplied into the score. Multiplying`
	`    would let a low severity condition climb arbitrarily high, which is`
	`    clinically wrong: severity changes urgency, not likelihood. Instead,`
	`    diseases whose scores sit within BAND_WIDTH are treated as tied and`
	`    ordered by severity.`
	
	`    Python's sort is stable, so equal score and equal severity keep their`
	`    existing relative order.`
	`    """`
		txt	Its docstring records the design decision: severity is not multiplied into the score — multiplying would let a low-severity condition ride a high score into urgency, which is clinically wrong: severity changes urgency, not likelihood. Instead, near-tied scores are banded and severity orders the band. Sort stability is used deliberately (267–268).

270–270	`    results.sort(key=lambda r: r["score"], reverse=True)`	L3	<code>results.sort(key=lambda r: r["score"], reverse=True)</code> — the Practical 3 sorted(key=lambda) pattern, best score first.

271–274	`    banded = []` `    i = 0`	L1	Banding state: <code>banded = []</code> accumulates the reordered list, <code>i = 0</code> is the while cursor (loops on an index — the Practical 1/4 loop pattern that a <code>for</code> cannot express, because the index moves by more than one).

275–278	`    while i < len(results):`
	`        band = []`
	`        anchor = results[i]["score"]`
		L3	Outer <code>while i < len(results)</code>: take <code>band</code> as a fresh list, <code>anchor</code> as this position's score. The band grows from wherever the cursor is.

279–281	`        while i < len(results) and anchor - results[i]["score"] < BAND_WIDTH:`
	`            band.append(results[i])`
	`            i += 1`
		L1	Inner <code>while</code>: every result whose score is within BAND_WIDTH of the anchor joins the band, and the cursor advances past each of them. The anchor is not re-evaluated as the band grows, so membership is against the first member — a deliberate, documentable choice.

282–282		txt	Blank line.

283–285	`        band.sort(key=lambda r: SEVERITY_RANK.get(r["severity"], 0),`
	`                  reverse=True)`
	`        banded.extend(band)`
		L3	283–284: <code>band.sort(key=lambda r: SEVERITY_RANK.get(r["severity"], 0), reverse=True)</code> — the tie broken by severity (an unknown severity sorts as 0); 285 the whole band is extended into the result.

286–287	`    return banded`	L4	<code>return banded</code> — same results, re-ordered by (score band, severity).
288–289		txt	Blank lines.
290–291	`def parse_age_range(text):` `    """Turn "35-70" into (35, 70). Returns None if it cannot be read."""`	L4	<code>parse_age_range(text)</code> + docstring: turns <code>"35-70"</code> into (35, 70), None when unreadable.
292–293	`    if not text:` `        return None`	L1	Blank cell → None (292–293): the blank-means-unknown rule again, this time about the knowledge base rather than the patient.
294–295	`    parts = text.split("-")`	L2	<code>parts = text.split("-")</code> — Practical 2 string split. 294 blank.
296–298	`    if len(parts) != 2:` `        return None`	L1	<code>if len(parts) != 2: return None</code> — a malformed range (dash in the wrong place) is unknown, not an error. <code>len()</code> is Practical 1; 298 the return.
299–301	`    low, high = parts[0].strip(), parts[1].strip()`	L2	<code>low, high = parts[0].strip(), parts[1].strip()</code> — tuple unpacking of the two bound strings with strip (Practical 2). 301 blank.
302–304	`    if not low.isdigit() or not high.isdigit():` `        return None`	L2	<code>if not (low.isdigit() and high.isdigit())</code> — the Practical 2 digit-shape test protecting <code>int()</code> from garbage. 304 blank.
305–307	`    return int(low), int(high)`	L1	<code>return int(low), int(high)</code> — casts (P1) into a tuple; 306–307 blank between functions.
308–309	`def age_outside_range(age, age_range):` `    """True when age is known and falls outside the disease's usual range."""`	L4	<code>age_outside_range(age, age_range)</code> + its one-line docstring.

310–313	`    bounds = parse_age_range(age_range)`
	
	`    if bounds is None or age in (None, ""):`
	`        return False`
		L1	Two unknowns, one false: 310 parse the range, 312–313 return False — evaluating to “not outside” — when the range is unreadable or the age is unknown. Nothing is ever reported from unknown data.

314–318	`    try:`
	`        age = int(ag`
	`e)`
	`    except (TypeErro`
	`r, ValueError):`
	`        return False`
		L1	The age arrives as a string; 315–318 cast inside try/except (TypeError, ValueError) with False on failure — a malformed age is still “not outside”.

319–321	`    low, high = bound`	L1	320 unpacks the bounds; 321 the comparison <code>age < low or age > high</code> — the two-sided range test, Practical 1's boolean operators.

	`s`
	`    return age < low`
	`or age > high`		

322–323		txt	Blank lines.
324–324	`def check_contraindic`	L4	<code>check_contraindications(disease, conditions)</code> opens.
	`ations(disease, condi` `tions):`		
325–330	`    """Return warning`	txt	Docstring: the conditions form a set like {pregnancy, kidney disease}; the return is warning strings. The same intersection operator, pointed the other way: instead of asking “which symptoms match” it asks “which conditions clash”.

	`strings for any condi`
	`tion the disease rule`
	`s out.`
	
	`    conditions is a s`
	`et of the patient's e`
	`xisting conditions,`
	`e.g.`
	`    {"pregnancy", "ki`
	`dney disease"}. Set i`
	`ntersection, same ope`
	`rator as`
	`    the symptom match`
	`but pointing the othe`
	`r way.`
	`    """`		

331–334	`    if not condition`	L3	No conditions at all → no warnings (331–332); then 334 <code>clashes = conditions & disease.get("contraindication_set", set())</code> — set intersection (P3) with a <code>.get</code> defaulting the disease's own side to the empty set when the column was absent.

	`s:`
	`        return []`
	
	`    clashes = conditi`
	`ons & disease.get("co`
	`ntraindication_set",`
	`set())`		

335–340	`    if not clashes:`	L3	Empty clashes → no warnings (336–337); otherwise 339 <code>", ".join(sorted(clashes))</code> — sorted (P3) for a stable order — and 340 one f-string warning: <code>{medication} is contraindicated for: {listed}</code>.

	`        return []`
	
	`    listed = ", ".joi`
	`n(sorted(clashes))`
	`    return [f"{diseas`
	`e['medication']} is c`
	`ontraindicated for:`
	`{listed}"]`		

341–342		txt	Blank lines.
343–343	`def rank_diseases(sym`	L4	<code>rank_diseases(symptoms, conditions=None, age=None, limit=MAX_RESULTS)</code> — the module's top-level function, three optional parameters typed after <code>=</code> (Practical 4 default arguments); the <code>conditions</code> and <code>age</code> extras arrive only through it.

	`ptoms, conditions=Non`
	`e, age=None, limit=MA`
	`X_RESULTS):`		

344–352	"""Rank candidate diseases for a set of reported symptoms. Returns a list of dicts, best first. Ea ch carries the score, the matched and missi ng symptoms, severit y, and any warnings. Diseases with few er than MIN_MATCHED_S YMPTOMS shared sympto ms are dropped. One shar ed symptom is not evi dence: "fatigue" appe ars in most rows of the knowledge base and wo uld match everything. """	txt	Its docstring states the output shape (dicts, best first, each carrying score, matched/missing symptoms, severity, warnings) and the filter below: fewer than two shared symptoms is dropped — “fatigue” appears in most rows and would match everything.
353–354	patient_symptoms = normalise_symptoms (symptoms) condition_set = n ormalise_symptoms(con ditions) if condition s else set()	L3	353 normalises the symptoms once; 354 <code>"if conditions else set()" </code> — a conditional expression + empty-set default so the contraindication check has something to intersect even when no conditions were given.
355–357	if not patient_sy mptoms: return []	L1	No symptoms after normalisation → no results (356–357). Mirrors the route's own guard: a legitimate empty answer, never an error.
358–360	results = []	L1	<code>results = []</code> — the accumulate-also-return pattern; 360 blank.
361–363	for disease in lo ad_diseases(): score, covera ge, precision, n_matc h, n_miss = score_dis ease(patient_s ymptoms, disease)	L4	The loop over every loaded disease; 362–363 unpack score_disease's five-tuple into named variables — the same tuple, unpacked at the call site, keeping the loop body readable.
364–366	if n_match < L1 MIN_MATCHED_SYMPTOMS: continue	L1	The minimum-evidence filter: <code>if n_match < MIN_MATCHED_SYMPTOMS: continue</code> — one matched symptom is not worth reporting.
367–371	matched = sor L3 ted(patient_symptoms & disease["symptom_se t"]) missing = sor ted(disease["symptom_ set"] - patient_sympt oms) unexplained = sorted(patient_sympto ms - disease["symptom _set"]) total = len(m atched) + len(missin g)	L3	Per survivor: 368 matched symptoms sorted for stable output, 369–370 missing and unexplained sorted the same way (<code>sorted(set) </code> is deterministic, P3), 371 total = matched + the disease's own unshown symptoms.
372–372		txt	Blank line.

```
        results.append({
            "disease_id": disease["disease_id"],
            "icd10_code": disease.get("icd10_code", ""),
            "name": disease["name"],
            "severity": disease["severity"],
            "score": round(score, 4),
            "percent": int(round(score * 100)),
            "evidence": evidence_strength(len(matched), total),
            "coverage": round(coverage, 4),
            "precision": round(precision, 4),
            "matched_count": n_match,
            "missing_count": n_miss,
            "total_symptoms": total,
            "matched_symptoms": matched,
            "missing_symptoms": missing,
            "unexplained_symptoms": unexplained,
            "medication": disease["medication"],
            "dosage": disease["dosage"],
            "age_range": disease["age_range"],
            "age_outside_range": age_outside_range(age, disease["age_range"]),
            "warnings": check_contraindications(disease, condition_set),
        })
```

L3

The result dict — what the API and report both consume: ids and name (374–376), severity (377), <code>score</code> rounded to four decimals and a percent from <code>int(round(score * 100))</code> (378–379), the evidence label (380), coverage and precision rounded (381–382), counts (383–385), the three sorted symptom lists (386–388), medication and dosage (389–390), age range plus an out-of-range flag (391–392), and warnings from the contraindication check (393). Everything a reader needs, computed once.

```
    results = apply_severity_bands(results)
    return results[:limit]
```

L4

396–397: the banded sort is applied, the list is cut to <code>[:limit]</code> — MAX_RESULTS honoured at exactly one place, at the boundary.

```
```

txt

Blank lines between functions.

```
def format_percent(result):
    """Return the score as a right aligned percentage string."""
    return f"{result['percent']:>3}%"
```

L5

<code>format_percent(result)</code>: an f-string with the alignment specifier <code>:>3</code> — right-aligned to three characters so single-digit percents keep their column in the report — Practical 5 formatted output, exactly.

```
```

txt

Blank lines.

405–405	`def render_report(patient, results, conditions=None):`	L4	<code>render_report(patient, results, conditions=None)</code> opens.

406–410	`    """Render the consultation report in the Lab 5 banner format.`
	
	`    patient is a dict with at least patient_id, name and optionally age.`
	`    The disclaimer is always the final line, per spec section 5.5.`
	`    """`
		txt	Docstring: the Lab 5 banner format — the report is plain text shaped like the practicals' console output; patient is a dict with at least patient_id and name; the disclaimer is always the final line, per spec.

411–415	`    line = "*" * 44`
	`    out = []`
	`    out.append(line)`
	`    out.append(" MERIDIAN CONSULTATION")`
	`    out.append(line)`
		L5	The banner: 411 <code>"*" * 44</code> — a string multiplied by an int, Practical 2 operator on strings; 413–415 line, title, line are appended to <code>out</code> (a list of strings joined at the end).

416–420	`    identity = f"{patient.get('patient_id', '')}  ({patient.get('name', '')})"`
	`    if patient.get("age"):`
	`        identity += f"  age {patient['age']}"`
	`    out.append(f"Patient       : {identity}")`
		L5	The patient line: 417 an f-string with id and name, 418–419 <code>if patient.get("age")</code> appends “age {age}” only when present (the blank-means-unknown rule), 420 append with field padding.

421–424	`    if conditions:`
	`        reported = ", ".join(sorted(normalise_symptoms(conditions)))`
	`        out.append(f"Conditions    : {reported}")`
		L3	Optional conditions line: 423 <code>", ".join(sorted(...))</code> — sorted (P3), joined; 424 appended only when conditions were reported (422).

425–433	`    if not results:`
	`        out.append("")`
	`        out.append("No disease matched the reported symptoms.")`
	`        out.append("")`
	`        out.append(line)`
	`        out.append(DISCLAIMER)`
	`        out.append(line)`
	`        return "\n".join(out)`
		L1	The no-match report: 426 when results are empty, 427–432 append a blank line, the honest sentence “No disease matched...”, the banner and the disclaimer, then 433 already return. Never fall through to a ranking that does not exist.

434–438	`    out.append("")`
	`    out.append("MOST LIKELY")`
	`    out.append("")`
		L5	A blank line, the MOST LIKELY header, another blank line — the report's own sectioning.

439–444	`    for i, r in enumerate(results, start=1):`
	`        out.append(`
	`            f"  {i}. {r['name']:<22} {format_percent(r)}"`
	`            f"   match {r['matched_count']}/{r['total_symptoms']}"`
	`            f"   {r['severity'].upper()}"`
	`        )`
		L5	The ranked loop using <code>enumerate(results, start=1)</code> (P4): 441–443 one f-string per line with <code>{r['name']:<22}</code> left-aligned, the percentage right-aligned via format_percent, the match ratio and the upper-cased severity (a string method, P2).

445–448	`        out.append(`
	`            f"     evidence: {r['evidence']}"`
	`            f"   medication: {r['medication']} {r['dosage']}"`
	`        )`
		L5	The evidence/medication line: evidence label and both dosage fields in one f-string (446–447) — the explanatory line beneath each candidate.

449–451	`        for warning in r["warnings"]:` `            out.append(f"     WARNING: {warning}")`	L1	Warning lines: loop the disease's warnings (450) and prefix each with WARNING: (451) — contraindication messages reach the patient-facing report here.
452–453	`        if r["age_outside_range"]:`	L1	The age note: 453 <code>if r["age_outside_range"]</code> — only reported conditions get the note.

454–457	`            out.append(`
	`                f"NOTE: usual age range is {r['age_range']},"`
	`                f" patient is {patient.get('age', 'unknown')}"`
	`            )`
		L5	The NOTE f-string (455–456): usual age range and the patient's own age with an “unknown” fallback — the report explains its own limitations on the record.

458–460	`        # A later row can outrank an earlier one on raw score. Say so, rather` `        # than letting the list look like it is mis-sorted.`	txt	Comment explaining the next check: a later row can outrank an earlier one on raw score because severity decides near-ties; the report says so instead of letting the list look mis-sorted.

461–466	`        if i > 1 and r["score"] > results[i - 2]["score"]:`
	`            out.append(`
	`                f"(listed below {results[i - 2]['name']} on score"`
	`                f" because severity outweighs a"`
	`                f" {abs(results[i - 2]['score'] - r['score']):.2f} difference)"`
	`            )`
		L5	The disclosure itself: 461 <code>if i > 1 and r["score"] > results[i - 2]["score"]</code> (boolean <code>and</code>, P1), then 463–465 an f-string naming the row above and the magnitude, <code>{...:.2f}</code> — two-decimal formatting, Practical 5.

467–468	`    out.append("")`	L5	Blank, then a blank line appended to the report.

469–473	`    out.append(line)`
	`    out.append(DISCLAIMER)`
	`    out.append(line)`
	
	`    return "\n".join(out)`
		L4	The trailer: banner line, DISCLAIMER, banner line (469–471) — the disclaimer is always the final text line, per the docstring — then a blank and 473 <code>return "\n".join(out)</code> materialises the whole report as one string.

analytics.py 754 lines — pandas profiling, risk bands, and the graph system — Labs 6 and 7.

1–14	"""Cohort analytics over the downloaded clinical datasets. Taught concepts: pandas read_csv, shape, describe, isnull, select_dtypes, groupby, value_counts, sort_values, head, plus the os module and matplotlib. Everything returned from here must survive json.dumps. pandas produces numpy types and NaN, neither of which is valid strict JSON, so every public function ends with to_native() and the test suite asserts json.dumps(..., allow_nan=False) succeeds. NaN becomes None rather than being dropped, because a missing value in a clinical column is information. Silence would read as a real zero. """	**txt**	Module docstring. analytics.py profiles the downloaded clinical datasets and renders the charts. Its taught list: pandas <code>read_csv</code>, <code>shape</code>, <code>describe</code>, <code>isnull</code>, <code>select_dtypes</code>, <code>groupby</code>, <code>value_counts</code>, <code>sort_values</code>, <code>head</code>, plus os and matplotlib (Practicals 6 and 7). Then the contract everything else obeys: every returned value must survive json.dumps, because pandas produces numpy types and NaN — neither is strict JSON — so each public function ends with <code>to_native()</code> and the test suite asserts <code>json.dumps(..., allow_nan=False)</code>. NaN becomes None, it is never dropped: a missing value in a clinical column is information, and silence would read as a real zero. 14 blank.
15–15	import json	**X:json**	<code>import json</code> — the json module, beyond the labs: every profile route's payload is serialised, and <code>json_check</code> at the bottom uses it as a validator. Explained in docs/concepts/json.md.
16–17	import os	**std**	<code>import os</code> — os (Practical 4) for paths and existence checks; 17 blank.
18–19	import pandas as pd	**L6**	<code>import pandas as pd</code> — pandas, Practical 6: the table engine this whole module drives. 19 blank.
20–22	# Both directories default to the plain relative paths a local run uses. # On Vercel the api/ entrypoint points them at writable copies in /tmp, # because the deployment bundle itself is read-only.	**txt**	Comment: both directories default to the plain relative paths a local run uses; on Vercel the api/ entrypoint points them at writable copies in /tmp because the deployment bundle itself is read-only.
23–26	DATA_DIR = os.environ.get("MERIDIAN_DATASET_DIR", os.path.join("data", "datasets")) REPORT_PATH = os.path.join(DATA_DIR, "profile_report.csv") CHART_DIR = os.environ.get("MERIDIAN_CHART_DIR", os.path.join("static", "charts"))	**std**	23–25: the three paths — <code>DATA_DIR</code> and <code>CHART_DIR</code> read environment variables with defaults (the Vercel overrides), <code>REPORT_PATH</code> is <code>data/datasets/profile_report.csv</code>, the sheet <code>list_datasets()</code> re-reads. <code>os.environ.get</code> with a default is std-lib, not a lab construct. 26 blank.

27–29	txt	`# UCI files mark unknown values with ? and not-applicable with -9. Without`
		`# these markers pandas reads the whole column as text and every numeric`
		`# count comes back zero.`
			Comment: UCI files mark unknown values with ? and not-applicable with -9. Without telling pandas about these markers, the whole column is read as text and every numeric count comes back zero.

30–31	L2	`MISSING_MARKERS = ["?", "-9", "NA", "N/A", ""]`	<code>MISSING_MARKERS = ["?", "-9", "NA", "N/A", ""]</code> — a plain list of strings (Practical 2) fed straight into <code>read_csv(na_values=...)</code> on line 72.
32–33	L2	`TARGET_NAMES = ["target", "outcome", "class", "diagnosis", "status", "malignancy"]`	<code>TARGET_NAMES</code> — the label-column spellings accepted across datasets (target, outcome, class, diagnosis, status, malignancy); find_target below scans this list. 33 blank.

34–45	txt	`# Risk band cut points per dataset column. Modelled on the BMI`
		`# categorisation problem from Practical 1, same if/elif shape.`
		`#`
		`# Cut points are the dataset's own quartiles rather than textbook clinical`
		`# thresholds, because these are cohort risk bands, not diagnosis. Using`
		`# textbook cutoffs made the bands useless: BMI quartiles are 27.3 / 32 /`
		`# 36.6, so standard cutoffs of 18.5 / 25 / 30 put 472 of 768 patients in`
		`# the top band. Quartiles give roughly equal groups.`
		`#`
		`# Glucose is the exception and keeps clinical cutoffs of 100 and 140,`
		`# because they align with the diabetes screening threshold and produce a`
		`# clean monotonic outcome rate: 8%, 29%, 50%, 82%.`
			The risk-band comment, the file's reasoning block. Cut points are the dataset's own quartiles rather than textbook clinical thresholds — these are cohort risk bands, not diagnosis: BMI quartiles here are 27.3 / 32.0 / 36.6, so standard cutoffs of 18.5 / 25 / 30 push 472 of 768 patients into the top band. Glucose is the exception and keeps clinical cutoffs of 100 and 140, because they align with the diabetes screening threshold and produce a clean monotonic outcome rate: 8%, 29%, 50%, 82%.

46–64	RISK_THRESHOLDS = {
	    "pima_diabetes.csv": {
	        "Glucose": [100, 140, 200],
	        "BMI": [27.3, 32.0, 36.6],
	        "Age": [24, 29, 41],
	    },
	    "heart_disease.csv": {
	        "chol": [200, 240, 280],
	        "trestbps": [120, 140, 160],
	        "oldpeak": [1.0, 2.0, 3.0],
	    },
	    "parkinsons.csv": {
	        "nhr": [0.0059, 0.0117, 0.0256],
	        "jitter_pct": [0.0035, 0.0049, 0.0074],
	        "rpde": [0.4213, 0.4960, 0.5876],
	    },
	}
		L3	<code>RISK_THRESHOLDS</code> — a dict of dicts (Practical 3): dataset filename → column → list of three cut points (Practical 2 lists). 47–51 pima diabetes (Glucose, BMI, Age), 52–56 heart disease (chol, trestbps, oldpeak), 57–61 parkinsons (nhr, jitter_pct, rpde), 62 closes the outer dict. Adding a dataset means adding data here, not code. 63–64 blanks.

65–71	def read_dataset(name):
	    """Read one dataset with missing markers converted to NaN."""
	    path = os.path.join(DATA_DIR, name)
	
	    if not os.path.exists(path):
	        raise FileNotFoundError(f"dataset not found: {name}")
		L4	<code>read_dataset(name)</code>: 66 docstring; 67 <code>os.path.join(DATA_DIR, name)</code>; 69–70 unknown dataset → <code>raise FileNotFoundError</code> with an f-string — a loud, specific failure the routes answer. Note the pattern returning repeated table access.

72–74	return pd.read_csv(path, na_values=MISSING_MARKERS, keep_default_na=True)	**X:nav**	72: <code>return pd.read_csv(path, na_values=MISSING_MARKERS, keep_default_na=True)</code> — the beyond-labs missing-value handling the comment on 27–29 set up: pandafc converts the markers (lines 30–31) to NaN on load. Explained in docs/concepts/pandas-missing-values.md. 73–74 blanks.

75–82	def find_target(df):
	    """Return the name of the target column, or None."""
	    for col in df.columns:
	        if col.strip().casefold() in TARGET_NAMES:
	            return col
	    return None
		L2	<code>find_target(df)</code>: loop <code>df.columns</code> (77, pandas attribute), test each name <code>col.strip().casefold() in TARGET_NAMES</code> (78 — the Practical 2 normalise-and-membership pair inside a Practical 1 loop), return at first hit (79), honest None on miss (81). 80, 82 blanks.

83–93	`def numeric_columns(df, exclude_target=True):`
	`  """Return numeric column names, optionally excluding the target."""`
	`  target = find_target(df)`
	`  cols = list(df.select_dtypes(include="number").columns)`
	
	`  if exclude_target and target:`
	`  cols = [c for c in cols if c != target]`
	
	`  return cols`
		X:dtypes	<code>numeric_columns(df, exclude_target=True)</code>: 86 <code>list(df.select_dtypes(include="number").columns)</code> — select_dtypes is beyond the labs (docs/concepts/pandas-select-dtypes.md): pandas picks columns by dtype instead of hand-listing names. 88–89 a list comprehension (P3) drops the target when asked, 91 returns. 87, 90, 92–93 blanks.

94–104	`def to_native(value):`
	`  """Convert pandas and numpy values into plain Python.`
	
	`  numpy int64 is not JSON serialisable. numpy float64 happens to be,`
	`  because it subclasses float, but it is converted anyway so the output`
	`  type is predictable. NaN and NaT become None, which is what JSON null`
	`  means, rather than the bare NaN token json.dumps emits by default.`
	
	`  Takes Any on purpose: it is a boundary function whose whole job is to`
	`  accept whatever pandas produced and return something safe.`
	`  """`
		txt	<code>to_native(value)</code> + docstring (95–104) — the boundary function the module contract stands on. numpy <code>int64</code> is not JSON-serialisable; <code>float64</code> happens to be (it subclasses float) but is converted anyway so the output type is predictable; NaN and NaT become None, which is what JSON null means, instead of the bare NaN token json.dumps emits. Takes anything on purpose: a boundary function's whole job.

105–107	`  if value is None:` `  return None`	**L1**	105–106: None passes through as None, then 107 blank.

108–113	`  if isinstance(value, (bool,)):`
	`  return bool(value)`
	
	`  if isinstance(value, int):`
	`  return int(value)`
		L4	Dispatch by <code>isinstance</code> — the taught type test: 108–109 bool first (bool is a subclass of int, so the order matters), 111–112 then plain <code>int</code>. Each branch returns the honest Python twin of what arrived. 110, 113 blanks.

114–118	`  if isinstance(value, float):`
	`  return None if pd.isna(value) else float(value)`
	
	`  if isinstance(value, str):`
	`  return None if value == "" else value`
		X:nav	114–115: the float branch — <code>return None if pd.isna(value) else float(value)</code>, the NaN-doc contract of docs/concepts/pandas-missing-values.md; 117–118: strings map empty <code>""</code> to None (a blank cell means unknown, never the text "").

119–124	`    if isinstance(value, dict):`
	`        return {str(to_native(k)): to_native(v) for k, v in value.items()}`
	
	`    if isinstance(value, (list, tuple)):`
	`        return [to_native(v) for v in value]`
		L3	The recursive container branches, both comprehensions (P3): 120–121 a dict comprehension whose keys are stringified (numpy types as JSON keys would crash), 123–124 a list comprehension over list/tuple. <code>to_native</code> calls itself — recursion (Practical 4, functions) — so nesting does not matter.

125–132	`    # Scalars only at this point. A Series or Index would make pd.isna`
	`    # return an array, whose truth value is ambiguous.`
	`    try:`
	`        if pd.isna(value):`
	`            return None`
	`    except (TypeError, ValueError):`
	`        pass`
		X:nav	Scalars only at this point: 126–127 comment warns a Series or Index would make <code>pd.isna</code> return an array whose truth value is ambiguous; 128–132 the try-guard: <code>pd.isna(value)</code> → None (pandas-missing-values.md), TypeError/ValueError swallowed when the value is not the right shape for isna.

133–137	`    if hasattr(value, "item"):`
	`        return to_native(value.item())`
	
	`    return value`
		L4	134–135: numpy scalars carry <code>.item()</code> — fetch it and recurse, which is the real exit door for int64/float64; 137 anything already safe returns unchanged. 133, 136 blanks.

138–147	`def list_datasets():`
	`    """Return the recorded profile of every dataset, or an empty list."""`
	`    if not os.path.exists(REPORT_PATH):`
	`        return []`
	
	`    return to_native(pd.read_csv(REPORT_PATH).to_dict("records"))`
		L6	<code>list_datasets()</code>: 142–143 — no report file yet, return <code>[]</code> (<code>os.path.exists</code>, P4); 145 <code>to_native(pd.read_csv(REPORT_PATH).to_dict("records"))</code> — re-reads the profile sheet the build script wrote and converts it record-by-record. 138–139 blanks above, 144 inside.

148–152	`def passing_datasets():`
	`    """Return names of datasets that met every selection criterion."""`
	`    return [d["file"] for d in list_datasets() if d.get("passes") == "PASS"]`
		L3	<code>passing_datasets()</code>: 150 one comprehension — <code>[d["file"] for d in list_datasets() if d.get("passes") == "PASS"]</code> the filter-then-extract shape (P3). Only datasets that passed every selection criterion may be profiled. 151–152 blanks.

153–162	`def describe_numeric(df):`
	`    """Return describe() for the numeric columns as a plain dict."""`
	`    cols = numeric_columns(df)`
	
	`    if not cols:`
	`        return {}`
	
	`    return to_native(df[cols].describe().to_dict())`
		L6	<code>describe_numeric(df)</code>: 155 numeric columns first; 157–158 none → empty dict; 160 <code>to_native(df[cols].describe().to_dict())</code> — (describe, count/mean/std/quartiles-summary) as plain keyed dict so JSON survives. 159 blank.

163–173	def value_counts(df, column, limit=10):
	    """Return the most common values in a column."""
	    if column not in df.columns:
	        return {}
	
	    counts = df[column].value_counts(dropna=True)
	    trimmed = counts.head(limit)
	
	    return to_native({str(k): int(v) for k, v in trimmed.items()})
		L6	<code>value_counts(df, column, limit=10)</code> — default argument (P4): 165–166 unknown column → empty dict; 168 <code>df[column].value_counts(dropna=True)</code> — capitalized value_counts: most-common first, missing values excluded; 169 <code>.head(limit)</code> trims; 171 keys stringified in one comprehension. 170 blank.

174–179	def group_means(df, limit=8): """Return mean of each numeric column grouped by the target. This is the analysis the whole project rests on: it is the Practical 6 groupby exercise applied to real clinical columns. """	txt	<code>group_means(df, limit=8)</code> + docstring (175–179): mean of each numeric column grouped by the target — the docstring names it plainly: the analysis the whole project rests on: the Practical 6 groupby exercise applied to real clinical columns.
180–184	target = find_target(df) if target is None: return {}	L1	182–183: no target column → empty dict — the empty-means-unknown answer again, this time for a pandas table. 180–181 lookups, 184 blank.
185–185	cols = numeric_columns(df)[:limit]	L2	185: <code>cols = numeric_columns(df)[:limit]</code> — slicing (Practical 2) caps how many columns the report carries.
186–188	if not cols: return {}	L1	187–188: no columns → empty; guard placed before the groupby.
189–192	return to_native(df.groupby(target)[cols].mean().to_dict())	L6	190: <code>return to_native(df.groupby(target)[cols].mean().to_dict())</code> — the one-line heart: groupby + mean, then to_dict so round-trip via JSON is possible. 191–192 blanks close the function.

193–206	def target_distribution(df):
	    """Return how the target column is distributed."""
	    target = find_target(df)
	
	    if target is None:
	        return {}
	
	    counts = df[target].value_counts(dropna=True)
	
	    return {
	        "column": target,
	        "counts": to_native({str(k): int(v) for k, v in counts.items()}),
	        "total": int(len(df)),
	    }
		L6	<code>target_distribution(df)</code>: 197–198 no target → empty; 200 <code>df[target].value_counts(dropna=True)</code>; 202–206 a hand-built response dict: the column's name, the counts converted with keys stringified, and total = int(len(df)) (P1 len + cast) so a chart can show n at its foot.

207–208		txt	Blank lines.
209–215	def strongest_separators(df, limit=5): """Rank numeric columns by how differently they spread across targets. For each column we compare the spread of group means against the overall mean. A large ratio means the target classes sit far apart on that column, so it is the most informative one to look at. """	txt	<code>strongest_separators(df, limit=5)</code> + docstring (210–214): rank numeric columns by how differently they spread across targets. For each column compare the spread of group means against the overall mean: a large ratio means the target classes sit far apart on that column — the most informative one to look at.
216–219	target = find_target(df) if target is None: return []	L1	218–219: no target, no separators → empty list; a single-line guard with the same shape as every other early exit. 216–217 lookup.

220–225	# The target itself is excluded. Grouping by target and then ranking the
	# target column always wins, which tells us nothing.
	cols = numeric_columns(df, exclude_target=True)
	means = df.groupby(target)[cols].mean()
	overall = df[cols].mean()
		L6	221–222: the comment states the trap guarded against — the target itself is excluded because grouping by target and then ranking the target column always wins, which tells us nothing; 224 <code>means = df.groupby(target)[cols].mean()</code> again, 225 <code>overall = df[cols].mean()</code> over the whole table.→

226–228	ranked = []	L1	227: <code>ranked = []</code> — the accumulator (P1). 226, 228 blanks.

229–236	for col in cols:
	    column_means = means[col].dropna()
	
	    if column_means.empty:
	        continue
	
	    spread = column_means.max() - column_means.min()
	    baseline = overall[col]
		L6	The per-column loop: 229 each numeric column; 230 <code>column_means = means[col].dropna()</code> (dropna: missing values doc); 232–233 all the groups miss → continue; 235 <code>spread = column_means.max() - column_means.min()</code> — the distance the classes actually differ by; 236 the column's overall mean as the baseline.

237–239	if baseline is None or pd.isna(baseline) or baseline == 0: continue	L1	238–239: skip a dead baseline — <code>None</code>, <code>pd.isna</code>, or exactly 0 (a division guard for two lines from now). Three truth conditions joined by <code>or</code> in one line (P1).

240–247	ranked.append({ "column": col, "spread": round(float(spread), 4), "relative_spread": round(float(spread / abs(baseline)), 4), "group_means": to_native({str(k): float(v) for k, v in column_means.items()}), })	**L3**	241–247: <code>ranked.append</code> one dict: column, <code>spread</code> rounded to 4 (round — P5 formatting vocabulary), <code>relative_spread = round(float(spread / abs(baseline)), 4)</code>, and the group means as a dict comprehension keyed by string. Relative — not raw — is the ranking number: a glucose gap of 60 is decisive, a BMI gap of a few is not.
248–250	ranked.sort(key=lambda r: r["relative_spread"], reverse=True) return ranked[:limit]	**L3**	249: <code>ranked.sort(key=lambda r: r["relative_spread"], reverse=True)</code> — the sorted(key=lambda) pattern (P3), best separators first; 250 return the <code>[:limit]</code> slice (P2).
251–257	def missing_report(df): """Return missing value counts per column.""" counts = df.isnull().sum() return to_native({str(k): int(v) for k, v in counts.items() if v > 0})	**X:nav**	<code>missing_report(df)</code>: 255 <code>counts = df.isnull().sum()</code> — nulls counted per column with isnull, beyond the labs (pandas-missing-values.md); 257 the peaks reported only: a comprehension keeping <code>if v > 0</code>, keys stringified for JSON. 251–252, 256 blanks.
258–264	def profile(name): """Return a full profile of one dataset as JSON-safe values.""" df = read_dataset(name) target = find_target(df)	**L4**	<code>profile(name)</code>: 262 reads the dataset, 263 finds the target, and nothing in the function is implemented here — it is an assembler.

265–280 L6

```
    return {
        "name": name,
        "rows": int(df.shape[0]),
        "cols": int(df.shape[1]),
        "target": target,
        "numeric_columns": numeric_columns(df),
        "describe": describe_numeric(df),
        "target_distribution": target_distribution(df),
        "group_means": group_means(df),
        "strongest_separators": strongest_separators(df),
        "missing": missing_report(df),
        "value_counts": {
            col: value_counts(df, col)
            for col in numeric_columns(df)[:4]
        },
    }
```

265–280: the profile dict the /api/analytics endpoint returns: name, rows / cols from df.shape (the Practical 6 attribute, turtle-cast to int), target, numeric columns, describe, target_distribution, group_means, strongest_separators, missing counts, and value_counts for the first four numeric columns as a nested comprehension (276–279). The whole vocabulary of the module in one payload.

281–282 txt

Blank lines.

283–293 txt

```
def risk_band(value, thresholds):
    """Bucket a value using cut points. Same shape as the BMI problem.

    thresholds is a list of three cut points, e.g. [100, 140, 200].
    Anything below the first is low, then moderate, elevated, high.

    None, NaN and unparseable values return "unknown" rather than falling
    through to "high". Every NaN comparison is False, so without the
    guard a missing measurement would be reported as the highest risk in
    the cohort, which is the opposite of what not knowing a value means.
    """
```

<code>risk_band(value, thresholds)</code> + docstring (284–292): bucket a value with three cut points — same shape as the BMI problem. The docstring writes the rule out: None, NaN and unparseable values return "unknown", never "high". Every NaN comparison is False, so without the guard a missing measurement would land in the highest band in the opposite of what not knowing a value means.

294–300	`    if value is None or thresholds is None:`
	`        return "unknown"`
	
	`    try:`
	`        value = float(value)`
	`    except (TypeError, ValueError):`
	`        return "unknown"`
		L1	The unknown guards: 294–295 either argument None → "unknown"; 297–300 the cast inside <code>try / except (TypeError, ValueError)</code> — the Practical 1 cast-with-failure pattern; a string like "high" becomes unknown, same honest answer.

301–303	`    if pd.isna(value):` `        return "unknown"`	L1	302–303: <code>if pd.isna(value): return "unknown"</code> — the NaN guard after the cast. If the input is a float NaN slipped through isinstance, it compares False to everything below and would otherwise fall through to high (see pandas-missing-values.md).

304–311	`    if value < thresholds[0]:`
	`        return "low"`
	`    if value < thresholds[1]:`
	`        return "moderate"`
	`    if value < thresholds[2]:`
	`        return "elevated"`
	`    return "high"`
		L1	305–311: the four-way ladder, the Practical 1 multi-branch shape verbatim: below cuts[0] → low, below cut[1] → moderate, below cut[2] → elevated, otherwise high. No else — the last return is the fall-through.

312–313		txt	Blank lines.
314–315	`def risk_bands(name, column):` `    """Return the distribution of one column across its risk bands."""`	txt	<code>risk_bands(name, column)</code> + one-line docstring: the distribution of one column across its risk bands.

316–320	`    df = read_dataset(name)`
	
	`    if column not in df.columns:`
	`        raise KeyError(f"{name} has no column {column}")`
		L4	316 reads the dataset; 318–319 unknown column → <code>raise KeyError</code> with an f-string — the route answers 400s on KeyError with the dataset's real column list.

321–324	`    thresholds = RISK_THRESHOLDS.get(name, {}).get(column)`
	
	`    if thresholds is None:`
	`        raise KeyError(f"no thresholds defined for {name} / {column}")`
		L3	321: <code>thresholds = RISK_THRESHOLDS.get(name, {}).get(column)</code> — two chained dict .get()s (P3): dataset, then column inside it, None if either is absent. 323–324 raise KeyError naming the absent piece — different-cause errors, different f-strings.

325–327	`    bands = {}`	L1	326: <code>bands = {}</code> — the label → count accumulator, blank lines around it.

328–330	`    for value in df[column]:`
	`        label = risk_band(value, thresholds)`
	`        bands[label] = bands.get(label, 0) + 1`
		L3	Counting: 328 every value in the series (iterating a column directly), 329 bucketed by risk_band, 330 <code>bands[label] = bands.get(label, 0) + 1</code> — the counting pattern dict.get with a 0 default (P3).

331–333	`    order = ["low", "moderate", "elevated", "high", "unknown"]`	L2	332: <code>order = ["low", "moderate", "elevated", "high", "unknown"]</code> — the display order as a list (P2), which a dict iteration order alone cannot guarantee when some keys are absent.

334–341

```
    return {
        "dataset": name,
        "column": column,
        "thresholds": thresholds,
        "bands": {k: bands.get(k, 0) for k in order if bands.get(k, 0) > 0},
        "unreachable_bands": unreachable_bands(df[column], thresholds),
        "total": int(len(df)),
    }
```

L3

334–341: the answer: dataset, column, thresholds; 338 the bands kept in the order list but only for labels present — a comprehension looping a list while reading a dict (two P3 structures at once); 339 <code>unreachable_bands(...)</code> pre-computes the honest-chart note; 340 total = int(len(df)).

342–343

```
```

txt

Blank lines.

344–351

```
def unreachable_bands(series, thresholds):
    """Return band names the column's range can never enter.

    A band no row can reach is misleading in a chart, because an empty bar
    looks like "nobody in this group" rather than "this group cannot
    exist". Glucose tops out at 199, so a cut point of 200 leaves the high
    band permanently empty.
    """
```

txt

<code>unreachable_bands(series, thresholds)</code> + docstring (345–350): returns band names the column's range can never enter. An empty bar in a chart reads as "nobody in this group" rather than "this group cannot exist" — misleading. Example written down: glucose tops out at 199, so a cut of 200 leaves the high band permanently empty.

352–358

```
    values = series.dropna()

    if values.empty:
        return ["low", "moderate", "elevated", "high"]

    smallest = float(values.min())
    largest = float(values.max())
```

L6

352: <code>values = series.dropna()</code> — missing values never speak to reachability (dropna is the missing-value doc's vocabulary); 354–355 empty series → all four bands unreachable; 357–358 the observed min and max.

359–370

```
    unreachable = []

    if smallest >= thresholds[2]:
        unreachable.append("low")
        unreachable.append("moderate")
        unreachable.append("elevated")
    elif smallest >= thresholds[1]:
        unreachable.append("low")
        unreachable.append("moderate")
    elif smallest >= thresholds[0]:
        unreachable.append("low")
```

L1

360: <code>unreachable = []</code>; 362–370 — the floor test (<code>smallest</code> vs cut[2], cut[1], cut[0]): if the range starts above a cut, every band under it can overly during the list-most-likely appends (an if/elif ladder, P1's chain shape).

371–380	L1
	`    if largest < thresholds[0]:`
	`        unreachable.append("moderate")`
	`        unreachable.append("elevated")`
	`        unreachable.append("high")`
	`    elif largest < thresholds[1]:`
	`        unreachable.append("elevated")`
	`        unreachable.append("high")`
	`    elif largest < thresholds[2]:`
	`        unreachable.append("high")`
		371 blank; 372–380 the ceiling test (<code>largest</code> vs cut[0], cut[1], cut[2]): if the range ends before a cut, every band over it is unreachable — the same ladder mirrored at the top end.

381–384	L4 `    return unreachable`	382: <code>return unreachable</code> — 383–384 blanks.

385–389	L3
	`def available_risk_bands(name):`
	`    """Return which columns have thresholds defined for this dataset."""`
	`    return sorted(RISK_THRESHOLDS.get(name, {}).keys())`
		<code>available_risk_bands(name)</code>: 387 <code>sorted(RISK_THRESHOLDS.get(name, {}).keys())</code> — sorted (P3) so pickers stay stable; the .get default makes an unconfigured dataset an empty list, not an error. 388–389 blanks.

390–403	txt
	`# ----------graphs`
	`#`
	`# save_charts renders the three figures every profile automatically gets.`
	`# A reader often wants to look at a different column than the one the`
	`# separator ranking happened to pick, so the same figures are exposed as`
	`# buildable graphs: one endpoint, a choice of kind, the dataset's own`
	`# columns as inputs. Each kind states which columns it needs in the spec`
	`# below; the endpoint validates against that before touching matplotlib,`
	`# so a bad pick is a 400 naming the usable columns, never a half-rendered`
	`# file.`
	`#`
	`# Explicit plt calls rather than df.plot() everywhere possible, same rule`
	`# as save_charts: every parameter stays explicable.`
		The graphs banner comment (390, 392–402): save_charts renders the three figures every profile automatically gets, but a reader often wants other columns, so the same figures are exposed as buildable graphs: one endpoint, a choice of kind, the dataset's own columns as inputs. Each kind states which columns it needs in the spec below; the endpoint validates against that before touching matplotlib, so a bad pick is a 400 naming the usable columns, never a half-rendered file. And explicit plt calls rather than df.plot(), wherever possible — every parameter stays explicable.

404–434	GRAPH_SPECS = {
	    "histogram": {
	        "label": "His
	togram",
	        "needs":
	["x"],
	        "note": "How
	one numeric column is
	distributed, split by
	the target.",
	    },
	    "bar_groupmeans":
	{
	        "label": "Mea
	n by target",
	        "needs":
	["x"],
	        "note": "Grou
	p means of one colum
	n, shown as a fractio
	n of its own "
	                "over
	all mean so different
	scales stay comparabl
	e.",
	    },
	    "box": {
	        "label": "Box
	plot",
	        "needs":
	["x"],
	        "note": "Spre
	ad and outliers of on
	e numeric column, per
	target group.",
	    },
	    "scatter": {
	        "label": "Sca
	tter",
	        "needs":
	["x", "y"],
	        "note": "Two
	numeric columns again
	st each other, split
	by target.",
	    },
	    "line": {
	        "label": "Lin
	e",
	        "needs":
	["x"],
	        "y-optional":
	True,
	        "note": "A tr
	end across one numeri
	c axis: counts along
	it, or the "
	                "mean
	of a second column if
	one is given.",
	    },
	}
		L7	<code>GRAPH_SPECS</code> — the catalog of kinds (a dict of dicts, P3, purpose is the graph work so tagged L7): 405–409 histogram (needs x); 410–415 bar_groupmeans — its note explains group means are drawn as a fraction of the column's own overall mean so different scales stay comparable; 416–420 box; 421–425 scatter (needs x and y); 426–432 line, marked <code>y-optional</code>. The <code>needs</code> key is what the route validates against.

435–437	DEFAULT_GRAPH_KINDS = ("histogram", "bar_gr oupmeans", "box", "sc atter", "line")	L7	435: <code>DEFAULT_GRAPH_KINDS = ("histogram", "bar_groupmeans", "box", "scatter", "line")</code> — a tuple (P2), the fixed display order the UI reads. 436–437 blanks.

438–454

```
def graph_kinds():
    """Return the kin
d registry as plain d
icts for the API and
the UI."""
    cards = []

    for kind in DEFAU
LT_GRAPH_KINDS:
        spec = GRAPH_
SPECS[kind]
        cards.append
({
            "kind": k
ind,
            "label":
spec["label"],
            "needs":
spec["needs"],
            "y_option
al": bool(spec.get("y
-optional")),
            "note": s
pec["note"],
        })

    return cards
```

L3

<code>graph_kinds()</code>: 440 <code>cards = []</code>; 442–450 — for every kind build a flat card dict from the spec, <code>bool(spec.get("y-optional"))</code> turning the flag into an honest True/False; 452 return. The /api/graphs catalog is these cards verbatim. 453–454 blanks.

455–467

```
def categorical_colum
ns(df):
    """Return text co
lumn names. The targe
t rides along: it is
the one
    grouping a graph
is usually drawn b
y."""
    target = find_tar
get(df)
    cols = [c for c i
n df.select_dtypes(ex
clude="number").colum
ns
            if c != t
arget]

    if target:
        cols.append(t
arget)

    return cols
```

X:dtypes

<code>categorical_columns(df)</code>: 459–460 <code>df.select_dtypes(exclude="number")</code> — select_dtypes again (docs/concepts/pandas-select-dtypes.md), the mirror of line 86, excluding the target so it is not listed twice; 462–463 then the target is appended explicitly — it is the grouping a graph is usually drawn by. 465 returns; 466–467 blanks.

468–478

```
def graph_columns(nam
e):
    """Return the dat
aset's usable column
s, sorted for stable
pickers."""
    df = read_dataset
(name)

    return {
        "numeric": so
rted(numeric_columns
(df)),
        "categorica
l": sorted(categorica
l_columns(df)),
        "target": fin
d_target(df),
    }
```

L3

<code>graph_columns(name)</code>: rereads the dataset (470) and returns both column lists sorted for stable pickers (473–474, the P3 sorted pattern) and the target under its own key (475). 477–478 blanks.

479–491

```
def _plt():
    """Import matplotlib lazily and force the headless Agg backend.

    A server has no display, and importing matplotlib at module top would
    pull ~30 MB into every process that only wanted a profile dict.
    """
    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt

    return plt
```

L7

<code>_plt()</code>: 480–484 the reason: a server has no display, importing matplotlib at module top would pull ~30 MB into every process that only wanted a profile — lazily with the headless <code>matplotlib.use("Agg") </code> backend (485–487). 489 returns the alias. 490–491 blanks.

492–502

```
def _graph_slug(text):
    """Turn a column name into a filename-safe fragment.

    Keep letters, digits and underscores and fold everything else to a
    dash, because a filename carries kind and columns and must not grow a
    slash or a space from whatever a header happened to hold.
    """
    return "".join(c if c.isalnum() or c == "_" else "-"
                   for c in text.strip().replace(" ", "_"))
```

L3

<code>_graph_slug(text)</code>: 499–500 one comprehension filters the characters — letters/digits and underscore keep, everything else → dash — so a filename cannot grow a slash or a space from whatever the header held (P2 .strip().replace feeding a P3 filter).

503–514

```
def _graph_file(kind, name, x, y):
    """Build the PNG path for one graph, newest-safe under CHART_DIR."""
    slug = name.replace(".csv", "")
    parts = [_graph_slug(kind), _graph_slug(x)]

    if y is not None:
        parts.append(_graph_slug(y))

    return os.path.join(CHART_DIR,

f"{slug}_{'_'.join(p.lower() for p in parts)}.png")
```

L4

<code>_graph_file(kind, name, x, y)</code>: 505 strips ".csv" (P2 .replace), 506–509 the kind and column slugs as parts, y only when present; 511–512 the full path via os.path.join (P4) around one f-string that lowercases and dash-joins the parts.

515–531

```
def _split_by_target (df, column):
    """Yield (label, values) for the colum n, one series per tar get group.

    A dataset with no target column yields the single whole-colu mn series.
    The graph code do es not need to know w hich case it is rende ring: one
    loop over these p airs draws both.
    """
    target = find_tar get(df)

    if target is Non e:
        yield "valu e", df[column].dropna ()
        return

    for label, group in df.groupby(targe t):
        yield f"{targ et}={label}", group[c olumn].dropna()
```

L6

<code>_split_by_target(df, column)</code>: 516–520 docstring — yields (label, values) one series per target group; a dataset with no target yields the whole column, so one loop over these pairs draws both. 524–526 is the no-target case; 528–529 <code>for label, group in df.groupby(target): yield f"{target}={label}", group[column].dropna()</code> — <code>yield</code> is the file's one lazy-sequence construct (a Practical 4 return stretched to a stream of results), paired with the P6 groupby.

532–540

```
def render_graph(nam e, kind, x=None, y=No ne):
    """Render one gra ph of the requested k ind and return its me tadata.

    Validation raises KeyError for an unkno wn dataset or kind (t he route
    answers those wit h a 400 listing what is available, the sam e contract
    as risk_bands) an d ValueError when the chosen columns do not fit the
    kind. Returns {"f ile": basename, "titl e", "kind", "datase t",
    "columns"}; the f ile lands in CHART_DI R and is served by /c harts/.
    """
```

txt

<code>render_graph(name, kind, x=None, y=None)</code> + docstring (533–540): renders one graph and returns its metadata. The error contract is written here: KeyError for an unknown dataset or kind (the route answers 400 listing what is available, the same contract as risk_bands), ValueError when the chosen columns do not fit the kind. Returns {"file": basename, "title", "kind", "dataset", "columns"} — the file lands in CHART_DIR and is served by /charts/.

541–546	`    if name not in passing_datasets():`
	`        raise KeyError(f"unknown dataset {name}")`
	
	`    if kind not in GRAPH_SPECS:`
	`        raise KeyError(f"unknown graph kind {kind!r}, "`
	`f"available: {', '.join(DEFAULT_GRAPH_KINDS)}")`
		L4	Validation, part one: 541–542 an unknown dataset → KeyError; 544–546 an unknown kind → KeyError whose message lists the available kinds (', '.join(DEFAULT_GRAPH_KINDS)), so the 400 teaches the client what exists.

547–551	`    df = read_dataset(name)`
	`    numeric = numeric_columns(df)`
	`    categorical = categorical_columns(df)`
	`    columns = {"x": x, "y": y}`
		L6	548–550: read the dataset, list the numeric and categorical columns, 551 <code>columns = {"x": x, "y": y}</code> — echoed back to the caller in the metadata.

552–563	`    if x is None:`
	`        raise ValueError("x column is required")`
	`    if x not in numeric:`
	`        raise ValueError(f"x must be a numeric column of {name}, "`
	`f"available: {', '.join(numeric)}")`
	
	`    if y is not None and y not in numeric:`
	`        raise ValueError(f"y must be a numeric column of {name}, "`
	`f"available: {', '.join(numeric)}")`
	`    if kind == "scatter" and y is None:`
	`        raise ValueError("scatter needs a y column")`
		L1	Validation, part two: 553–554 no x → ValueError; 555–557 x not numeric → ValueError naming the available columns; 559–561 the same test for an optional y; 562–563 scatter without y → ValueError ("scatter needs a y column"). Four checks, one honest error message each.

564–566	`    if not os.path.exists(CHART_DIR):` `        os.makedirs(CHART_DIR)`	L4	565–566: same makedirs-once pattern as the rest of the project: the chart directory is created only when needed.
567–569	`    plt = _plt()` `    path = _graph_file(kind, name, x, y)`	L7	568 matplotlib is imported only now, via _plt(); 569 the destination filename is computed up-front, before any pixels exist.

570–578

```
    title, x_label, y_label = {
        "histogram": (f"{name}: distribution of {x}", x, "patients"),
        "bar_groupmeans": (f"{name}: mean {x} as a fraction of its overall "
                           "mean", x, "x average"),
        "box": (f"{name}: spread of {x}", x, x),
        "scatter": (f"{name}: {y} against {x}", x, y),
        "line": (f"{name}: {'mean ' + y + ' by ' + x if y else 'count by ' + x}",
                 x, f"mean {y}" if y else "rows"),
    }[kind]
```

L5

570–578: titles and axis labels — a dict of five f-string tuples (P5 formatted output) for the five kinds, looked up with <code>[kind]</code> and unpacked into <code>title, x_label, y_label</code> in one statement (P3/P4 idiom): the chart text is decided by the same spec data as the validation.

579–580

```
    fig, ax = plt.subplots(figsize=(7.5, 4.2))
```

L7

580: <code>fig, ax = plt.subplots(figsize=(7.5, 4.2))</code> — one figure, one axes; everything below draws on this ax.

581–587

```
    if kind == "histogram":
        for label, values in _split_by_target(df, x):
            ax.hist(values, bins=30, alpha=0.6, label=label)

        if find_target(df) is not None:
            ax.legend()
```

L7

<code>if kind == "histogram"</code> (582): 583–584 — for each target group a stacked-overlay <code>ax.hist(values, bins=30, alpha=0.6, label=label)</code>, alpha so overlapping distributions stay readable; 586–587 a legend only when a target exists to chart.

588–601

```
    elif kind == "bar_groupmeans":
        target = find_target(df)

        if target is None:
            means = df[[x]].mean()
            relative = means / means.mean()
            ax.bar(relative.index, relative[x])
        else:
            means = df.groupby(target)[x].mean()
            relative = means / means.mean()
            ax.bar(relative.index.astype(str), relative.values)

        ax.axhline(1.0, color="#888", linewidth=0.8, linestyle="--")
```

L7

<code>elif kind == "bar_groupmeans"</code> (589): 592–599 group means divided by their own overall mean (<code>relative = means / means.mean()</code>) — the normalisation the spec's note promised — then <code>ax.bar</code>; 601 a dashed grey <code>ax.axhline(1.0)</code> so the "equal to average" reference line is visible.

602–610

```
    elif kind == "box":
        pairs, labels = [], []

        for label, values in _split_by_target(df, x):
            pairs.append(values)
            labels.append(label)

        ax.boxplot(pairs, tick_labels=labels, vert=True)
```

L7

<code>elif kind == "box"</code> (603): 604–609 collect one values-series and one label per target group; 610 <code>ax.boxplot(pairs, tick_labels=labels, vert=True)</code> — spread and outliers, one box per class.

611–621

```
    elif kind == "scatter":
        target = find_target(df)

        if target is None:
            ax.scatter(df[x], df[y], s=14, alpha=0.55)
        else:
            for label, group in df.groupby(target):
                ax.scatter(group[x], group[y], s=14, alpha=0.55,

label=f"{target}={label}")
            ax.legend()
```

L7

<code>elif kind == "scatter"</code> (612): without a target, one cloud (616, <code>s=14, alpha=0.55</code> for density legibility); with one, 618–621 a cloud per group, each labelled <code>f"{target}={label}"</code> and a legend.

622–631

```
    elif kind == "line":
        frame = df[[x]] if y is None else df.dropna(subset=[x, y])

        if y is None:
            counts = frame.groupby(x).size()
            ax.plot(counts.index, counts.values, marker="o", linewidth=1.2)
        else:
            means = frame.groupby(x)[y].mean()
            ax.plot(means.index, means.values, marker="o", linewidth=1.2)
```

L7

<code>elif kind == "line"</code> (623): 624 the frame keeps only rows where the needed columns are present (<code>df.dropna(subset=[x, y])</code>); 626–628 without y it plots row counts along x (size per x value), 630–631 with y it plots the mean of y per x — both drawn <code>marker="o", linewidth=1.2</code>.

632–637

```
    ax.set_title(title)
    ax.set_xlabel(x_label)
    ax.set_ylabel(y_label)
    fig.savefig(path, dpi=100, bbox_inches="tight")
    plt.close(fig)
```

L7

Finishing: 633–635 set_title / set_xlabel / set_ylabel from the f-string block; 636 <code>fig.savefig(path, dpi=100, bbox_inches="tight")</code>; 637 <code>plt.close(fig)</code> — without it, every render would leak a figure (matplotlib-basics.md).

638–645	`    return {`
	`        "dataset": name,`
	`        "kind": kind,`
	`        "columns": columns,`
	`        "title": title,`
	`        "file": os.path.basename(path),`
	`    }`
		L4	639–645: the metadata dict — dataset, kind, columns, title, and file: os.path.basename(path): the route only hands out the basename, served later from /charts/, so no client may read a full filesystem path.

646–647		txt	Blank lines.

648–660	`def save_charts(name):`
	`    """Render charts to static/charts and return the filenames written.`
	
	`    Uses explicit plt calls rather than df.hist() or df.plot() so that`
	`    every parameter can be explained.`
	
	`    The group means chart normalises each column to its own mean before`
	`    plotting. Without that, the widest-ranging column flattens the others.`
	`    Measured on pima_diabetes: Glucose spans 0 to 199 while`
	`    DiabetesPedigreeFunction sits around 0 to 0.4, so plotting raw means`
	`    made the DPF bars a flat line at the bottom of the axis. Dividing each`
	`    column by its own mean puts them on one readable scale.`
	`    """`
		txt	<code>save_charts(name)</code> + docstring (649–660): the three figures every profile gets, written to static/charts. The docstring carries one measured story: the group means chart normalises each column to its own mean — without it the widest column flattens the others (glucose spans 0–199 while DiabetesPedigreeFunction sits 0–0.4; raw bars were a flat line). Explicit plt calls, every parameter explicable.

661–663	`    import matplotlib`
	`    matplotlib.use("Agg")`
	`    import matplotlib.pyplot as plt`
		L7	661–663: the lazy-import block again, locally: <code>import matplotlib</code>, <code>matplotlib.use("Agg")</code>, <code>import matplotlib.pyplot as plt</code> — written before _plt() existed and kept explicit here.

664–673	`    df = read_dataset(name)`
	`    target = find_target(df)`
	`    cols = numeric_columns(df)`
	
	`    if not os.path.exists(CHART_DIR):`
	`        os.makedirs(CHART_DIR)`
	
	`    written = []`
	`    slug = name.replace(".csv", "")`
		L6	665–667: dataset, target and numeric columns gathered; 669–670 the chart directory is made if missing; 672–673 <code>written = []</code> (the accumulator) and the dataset slug.

674–677 L1

```
    # Histogram of the strongest separating column, split by target.
    separators = strongest_separators(df)
    focus = separators[0]["column"] if separators else (cols[0] if cols else None)
```

676–677: <code>focus = separators[0]["column"] if separators else (cols[0] if cols else None)</code> — two nested conditional expressions (P1/P4): the strongest separator if one exists, else the first numeric column, else nothing.

678–696 L7

```
    if focus:
        fig, ax = plt.subplots(figsize=(7, 4))

        if target is not None:
            for label, group in df.groupby(target):
                ax.hist(group[focus].dropna(), bins=30, alpha=0.6,
                        label=f"{target}={label}")
            ax.legend()
        else:
            ax.hist(df[focus].dropna(), bins=30)

        ax.set_title(f"{name}: distribution of {focus}")
        ax.set_xlabel(focus)
        ax.set_ylabel("patients")
        path = os.path.join(CHART_DIR, f"{slug}_{focus}_hist.png")
        fig.savefig(path, dpi=100, bbox_inches="tight")
        plt.close(fig)
        written.append(path)
```

Chart 1, the focused histogram: 679–696 — figure made; hist drawn per target group with alpha 0.6 and a legend (682–686) or the single whole-column hist (688); 690–692 title and axes names; 693 the filename f-string; 694–696 savefig, plt.close (close the figure — the leak rule) and append to <code>written</code>.

697–704 L7

```
    # Bar chart: each column as a fraction of its own mean, by target.
    # Dividing by the column mean puts glucose, insulin and BMI on one
    # readable scale, and picks the columns that actually separate.
    if target is not None and cols:
        chosen = [s["column"] for s in separators[:5]] or cols[:5]
        means = df.groupby(target)[chosen].mean()
        relative = means / means.mean()
```

698–703: chart 2 selection — the comment explains dividing by the column mean puts glucose, insulin and BMI on one readable scale and picks the columns that actually separate; 702 up to five separator columns, <code>cols[:5]</code> as fallback; 703–704 the groupby means and the division.

```
705–717          fig, ax = pl  L7
t.subplots(figsize=
(9, 4.5))
        relative.plot
(kind="bar", ax=ax)
        ax.set_title
(f"{name}: mean as a
fraction of overall m
ean, by {target}")
        ax.set_ylabel
("x average")
        ax.axhline(1.
0, color="#888", line
width=0.8, linestyle
="--")
        ax.legend(tit
le=target, loc="upper
center",
                  bbo
x_to_anchor=(0.5, -0.
12), ncol=3, frameon=
False)
        plt.xticks(ro
tation=0)
        path = os.pat
h.join(CHART_DIR, f"
{slug}_groupmeans.pn
g")
        fig.savefig(p
ath, dpi=100, bbox_in
ches="tight")
        plt.close(fi
g)
        written.appen
d(path)
```

Chart 2, the grouped bar: 706–707 figure and <code>relative.plot(kind="bar", ax=ax)</code>; 708–709 title and y-label; 710 the axhline at 1.0; 711–713 the legend laid out under the axes (<code>bbox_to_anchor</code>, <code>ncol=3</code>, no frame) and readable unrotated ticks; 714–717 savefig, close, append.

718–743

```
    # Risk band chart for the first defined column.
    for column in available_risk_bands(name):
        try:
            bands = risk_bands(name, column)
        except KeyError:
            continue

        labels = list(bands["bands"].keys())
        values = [bands["bands"][k] for k in labels]

        fig, ax = plt.subplots(figsize=(6, 4))
        ax.bar(labels, values, color="#4a6fa5")
        ax.set_title(f"{name}: {column} risk bands")
        ax.set_ylabel("patients")

        if bands["unreachable_bands"]:
            ax.set_title(f"{name}: {column} risk bands\n"
                         f"(no {', '.join(bands['unreachable_bands'])} band: "
                         f"outside the data range)")

        path = os.path.join(CHART_DIR, f"{slug}_{column}_bands.png")
        fig.savefig(path, dpi=100, bbox_inches="tight")
        plt.close(fig)
        written.append(path)
        break
```

L7

Chart 3, risk bands: 720–724 loop the configured columns, skipping any whose bands cannot be computed (KeyError try/except → continue); 726–730 labels/values extracted from the bands dict and drawn <code>ax.bar(labels, values, color="#4a6fa5")</code>; 734–737 the unreachable bands are written into the title ("no high band: outside the data range") instead of drawn as empty bars; 739–743 savefig, close, append and break — only the first defined column is charted.

744–747

```
    return written
```

L7

745: <code>return written</code> — the filenames the profile embeds; 746–747 blanks.

748–754

```
def json_check(payload):
    """Return (ok, message). Used by tests to prove a payload is safe."""
    try:
        json.dumps(payload, allow_nan=False)
        return True, "serialises as strict JSON"
    except (TypeError, ValueError) as e:
        return False, str(e)
```

X:json

<code>json_check(payload)</code>: 751 <code>json.dumps(payload, allow_nan=False)</code> inside try/except — strict JSON: numpy types and NaN raise, so returning (True, message) / (False, str of error) gives tests a direct proof of payload hygiene. The json module itself is beyond the labs; see docs/concepts/json.md.

app.py 1064 lines — Flask routes and response shapes only — glue layer.

1–17	"""Flask API for Meridian. Taught concepts: none new in this file. Flask is the only concept added beyond the lab syllabus and it gets its own explainer in docs/concepts/flask.md. Layering rule from the spec: this module routes and nothing else. Every handler calls functions in store, registry, triage or analytics and returns their result. No business logic lives here. Run from the project root: python backend/app.py then the API is on http://127.0.0.1:8000 """	txt	Module docstring. Three declarations set the frame: no taught concept is new in this file — Flask is the one idea beyond the lab syllabus and it has its own explainer, docs/concepts/flask.md; the layering rule — this module routes and nothing else, every handler calls store / registry / triage / analytics and returns the result, no business logic lives here; and how to run it (127.0.0.1:8000). 17 blank.
18–18	import os	std	<code>import os</code> — os (Practical 4) for path joins and checks.
19–20	import sys	std	<code>import sys</code> — used once, on line 23, to steer the import path. 20 blank.
21–22	from flask import Flask, jsonify, request, send_from_directory	X:flask	<code>from flask import Flask, jsonify, request, send_from_directory</code> — the four Flask names this file uses: the app maker, the JSON reply maker, the reader of the incoming request, and the static file server. Beyond the labs; see docs/concepts/flask.md. 22 blank.
23–24	sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))	std	23: <code>sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))</code> puts this directory at the front of the import path, so <code>python backend/app.py</code> can <code>import store</code> even when the working directory is the project root. 24 blank.
25–29	import analytics import registry import store import triage	L4	<code>import analytics / registry / store / triage</code> (25–28) — the four layers this routing module glues together, in the order their calls appear in the file. 29 blank.
30–33	# static_folder stays at the default backend/static so generated charts are # served from /charts/<file>. static_url_path must NOT be "" because that # makes Flask claim every unmatched path and the frontend route below # never runs.	txt	Comment guarding a real Flask trap: the static folder stays at the default backend/static so generated charts are served from <code>/charts/<file></code>, and <code>static_url_path</code> must not be set to "" — that makes Flask claim every unmatched path and the frontend catch-all route below would never run.
34–35	app = Flask(__name__)	X:flask	34: <code>app = Flask(__name__)</code> — the application object every <code>@app...</code> decorator hangs off. docs/concepts/flask.md. 35 blank.

36–37	`FRONTEND_DIR = os.path.join(os.path.dirname(os.path.dirname(os.path.abspath(__file__))), "frontend")`	L4	<code>FRONTEND_DIR</code> — the frontend directory, computed with nested <code>os.path</code> calls (Practical 4): dirname of dirname of this file, joined with "frontend". Being derived from <code>__file__</code>, it is correct no matter which directory the server was started from. 37 blank.
38–40	`SEARCHABLE = ("patients", "medicines", "stores", "hospitals", "doctors",  "rare_conditions")`	L2	<code>SEARCHABLE</code> — a tuple of the six legal search targets (Practical 2). A tuple on purpose: a constant that must never be appended to by accident. The membership test <code>record_type not in SEARCHABLE</code> at line 708 is the 404 check. 40 blank.
41–48	`# ---------- -- paging # # The registries hold 600 patients, 540 doctors and 520 hospitals. Serving # every row on every page view meant /api/patients answered with 334 KB and # /api/visits with 1.4 MB, which is the whole registry to draw a screen # that shows fifty. Pagination is what lets the frontend show a count it # can trust instead of a number it happened to receive.`	txt	The paging banner comment. Real numbers: 600 patients, 540 doctors, 520 hospitals — serving every row per page view made <code>/api/patients</code> answer 334 KB and <code>/api/visits</code> 1.4 MB, the whole registry just to draw a screen that shows fifty. Pagination lets the frontend show a count it can trust rather than a number it happened to receive.
49–52	`PAGE_LIMIT_DEFAULT = 50 PAGE_LIMIT_MAX = 500`	L1	49–50: the two page bounds, plain integer constants (Practical 1): default 50 rows, hard ceiling 500 — no caller can ask for "everything" and get away with it. 51–52 blanks.
53–59	`def page_args():  """Read ?limit= and ?offset= as integers, clamped to sane bounds.  A missing, negative or non-numeric value falls back to the default  rather than raising, because this is a query string and a bad one  should still render a page.  """`	txt	<code>page_args()</code> signature and docstring: read <code>?limit=</code> and <code>?offset=</code> as integers, clamped to sane bounds. A missing, negative or non-numeric value falls back to the default rather than raising — a bad query string should still render a page.
60–61	`    def whole(name, default, ceiling):  raw = request.args.get(name)`	X:flask	60–61: the nested helper <code>def whole(name, default, ceiling)</code> and its first line <code>raw = request.args.get(name)</code> — reading a query parameter is Flask's request object (beyond the labs, flask.md). Nesting the helper inside <code>page_args</code> keeps it local to its only caller (Practical 4 functions).
62–64	`        if raw is None or raw == "":  return default`	L1	62–63: <code>if raw is None or raw == "":</code> return the default — the Practical 1 truth test handling both a missing and an empty parameter in one line. 64 blank.
65–68	`        try:  value = int(raw)  except ValueError:  return default`	L1	The cast with a failure path: 65–68 — <code>try: value = int(raw)</code>, <code>except ValueError: return default</code>. A literal <code>?limit=fifty</code> degrades gracefully instead of crashing the route; this is the Practical 1 typecast pattern wrapped in its error cage.

69–70	`        return max(0, min(value, ceiling))`	L1	70: <code>return max(0, min(value, ceiling))</code> — the clamp. Practical 1 comparison operators inside two built-ins pin the value between 0 and the constant. 69 blank.
71–75	`    return (whole("limit", PAGE_LIMIT_DEFAULT, PAGE_LIMIT_MAX),` `            whole("offset", 0, 10 ** 9))`	L1	72–73: the return builds its tuple by calling <code>whole</code> twice — limit gets <code>10 ** 9</code> as a ceiling (the Practical 1 exponent operator as "a number no one will ever hit"), offset gets 0. 71 blank; 74–75 blanks.

76–82	`def paged(records, key, skipped=None, facets=None):`
	`    """Slice records to the requested page and report the true total.`
	
	`    The window is clamped at both ends so an offset past the end returns`
	`    an empty page rather than an error, which is what a user who was on`
	`    page 9 when a record was deleted should see.`
	`    """`
		txt	<code>paged(records, key, skipped=None, facets=None)</code> and its docstring: slice records to the requested page and report the true total. The window is clamped at both ends so an offset past the end returns an empty page, not an error — what a user sitting on page 9 when a record got deleted should see. Three default arguments (all keyword-callable from the routes, Practical 4).

83–84	`    limit, offset = page_args()` `    total = len(records)`	L1	83: <code>limit, offset = page_args()</code> — tuple unpacking of the two ints; 84 <code>total = len(records)</code> — the honest total, taken before the slice (Practical 1 <code>len</code>).
85–86	`    window = records[offset:offset + limit]`	L2	86: <code>window = records[offset:offset + limit]</code> — the page is a slice (Practical 2), computed on a plain list so nothing exotic is needed to page it. 85 blank.

87–95	`    body = {`
	`        "count": len(window),`
	`        "total": total,`
	`        "offset": offset,`
	`        "limit": limit,`
	`        "has_more": offset + len(window) < total,`
	`        key: window,`
	`    }`
		L3	88–95: the reply <code>body</code> dict (Practical 3) — <code>count</code> (rows in this page), <code>total</code>, <code>offset</code>, <code>limit</code>, <code>has_more</code> (a Practical 1 comparison: <code>offset + len(window) < total</code>), and the records under the dynamic key <code>key: window</code> — the caller's name (patients, visits, ...) becomes the JSON field name, which is how one helper serves every registry. 87 blank.

96–101	`    if skipped is not None:`
	`        body["skipped"] = skipped`
	
	`    if facets is not None:`
	`        body["facets"] = facets`
		L1	97–101: two optional enrichments — <code>if skipped is not None: body["skipped"] = skipped</code>, same shape for <code>facets</code>. Absent means absent: the JSON key only exists when there is something real to report, which is the sentinel test (P1) used to keep payloads honest. 96 blank.

102–103	`    return jsonify(body)`	X:flask	103: <code>return jsonify(body)</code> — Flask turns the dict into a JSON HTTP response (flask.md). Every list route ends here. 102 blank.
104–106		txt	Blank lines after <code>paged</code>.

107–111	`# How many rows a search returns. The rare condition registry holds 11,655 # diseases and a single phrase like "low muscle tone" matches 7,559 of # them, so returning everything is not an option. The total is reported # alongside the page so the interface can say "showing 50 of 7,559" # rather than implying the list is complete.`	txt	Why search has its own page size: the rare condition registry holds 11,655 diseases and one phrase — "low muscle tone" — matches 7,559 of them, so returning everything is not an option. The total travels with the page so the interface can say "showing 50 of 7,559" instead of implying the list is complete.
112–114	`SEARCH_PAGE_SIZE = 50`	L1	112: <code>SEARCH_PAGE_SIZE = 50</code> — the constant the comment argues for. 113–114 blanks.
115–116	`def payload():` `    """Return the request body as a dict, or an empty dict if absent."""`	txt	<code>payload()</code> signature and one-line docstring: return the request body as a dict, or an empty dict if absent.
117–119	`    return request.get_json(silent=True) or {}`	X:flask	117: <code>return request.get_json(silent=True) or {} </code> — Flask's request reader with <code>silent=True</code> so a missing or malformed body yields <code>None</code> instead of raising, and the <code>or {}</code> idiom (P1) turns that None into an empty dict every route can treat uniformly (flask.md). 118–119 blanks.

120–126	`def full_record(record_type, cleaned):`
	`    """Return every column for a record type, blanks filled in.`
	
	`    A create response must show the whole row, not just the fields the`
	`    caller happened to send. Otherwise a client cannot tell the difference`
	`    between "field is empty" and "field does not exist".`
	`    """`
		txt	<code>full_record(record_type, cleaned)</code> and its docstring: a create response must show the whole row, not just the fields the caller happened to send — otherwise a client cannot tell "field is empty" apart from "field does not exist".

127–127	`    complete = {}`	L1	<code>complete = {}</code> — the accumulator (Practical 1).

128–134	`    for field in store.fields_for(record_type):`
	`        complete[field] = cleaned.get(field, "")`
	
	`    return complete`
		L4	129–130: loop <code>store.fields_for(record_type)</code> — the schema list from the store layer (L4 delegation) — and fill every column with <code>cleaned.get(field, "")</code> (the dict.get default, Practical 3). 131 returns the complete record; 132–134 blanks.

135–136	`def error(message, status=400, **extra):` `    """Build a JSON error response."""`	txt	<code>error(message, status=400, **extra)</code> and its docstring: build a JSON error response. The <code>**extra</code> parameter collects any named details (errors=..., available=...) — keyword-bag syntax, Practical 4 functions.
137–138	`    body = {"error": message}` `    body.update(extra)`	L3	137–138: <code>body = {"error": message}</code> then <code>body.update(extra)</code> merges the keyword bag into the dict (Practical 3 dict update).
139–141	`    return jsonify(body), status`	X:flask	139: <code>return jsonify(body), status</code> — the two-tuple reply: Flask reads a (body, status) pair so one helper answers 400, 404 or 500 alike (flask.md). 140–141 blanks.

142–143	`# ---------- frontend`	txt	142 the frontend section banner; 143 blank.
144–147	`@app.get("/") def index(): """Serve the frontend shell.""" return send_from_directory(FRONTEND_DIR, "index.html")`	X:flask	The shell route: 144 <code>@app.get("/")</code> registers the function for <code>/</code> (flask.md); 145–146 the handler and its docstring; 147 <code>return send_from_directory(FRONTEND_DIR, "index.html")</code> — Flask serves one file from a safe directory. The whole SPA loads from this single line.
148–149		txt	Blank lines.
150–157	`@app.get("/<path:filename>") def frontend_assets(filename): """Serve a frontend asset.  Only three file types are allowed, and the path is resolved and checked against the frontend directory so a crafted request cannot walk out of it. """`	txt	<code>@app.get("/<path:filename>")</code> and <code>frontend_assets</code> with its docstring (152–157): the <code>path</code> converter accepts slashes, so this is the catch-all for every frontend asset — and therefore the place where the three defences below live: allow only three file types, resolve the path and check it against the frontend directory so a crafted request cannot walk out of it.
158–160	`allowed = (".html", ".css", ".js")  if not filename.endswith(allowed):`	L2	158: <code>allowed = (".html", ".css", ".js")</code> — a tuple again (P2); 160 <code>if not filename.endswith(allowed):</code> — endswith happily takes a tuple of suffixes, the Practical 2 string method doing the whitelist work in one call.
161–161	`return error(f"{filename} is not a frontend asset", 404)`	X:flask	<code>return error(f"{filename} is not a frontend asset", 404)</code> — the f-string names the offender; error() is the file's one JSON error builder.
162–163	`target = os.path.normpath(os.path.join(FRONTEND_DIR, filename))`	L4	163: <code>target = os.path.normpath(os.path.join(FRONTEND_DIR, filename))</code> — the Practical 4 join, then <code>normpath</code> collapses every <code>../</code> into a resolved absolute-looking path. 162 blank.
164–165	`if not target.startswith(os.path.normpath(FRONTEND_DIR)):`	L2	165: <code>if not target.startswith(os.path.normpath(FRONTEND_DIR)):</code> — the string prefix test (P2) is the security gate: anything that escaped the frontend directory (<code>../../etc</code> style) fails it.
166–166	`return error("path outside the frontend directory", 404)`	X:flask	<code>return error("path outside the frontend directory", 404)</code> — the traversal attempt is answered, not acted on.
167–168	`if not os.path.exists(target):`	L4	168: <code>if not os.path.exists(target):</code> — the Practical 4 existence check; an unknown asset is 404 below rather than an exception traceback. 167 blank.
169–169	`return error(f"no asset named {filename}", 404)`	X:flask	<code>return error(f"no asset named {filename}", 404)</code> — the honest missing file.
170–173	`return send_from_directory(FRONTEND_DIR, filename)`	X:flask	171: <code>return send_from_directory(FRONTEND_DIR, filename)</code> — only after all three checks does Flask serve the file (flask.md). 170 blank; 172–173 blanks.
174–175	`# ---------- health`	txt	174 the health section banner; 175 blank.

176–184	def registry_page(record_type, key, after=None): """Read a registry, apply ?flag=, then return one page of it. The facet is applied before the window is cut, otherwise total would report the size of the page rather than the size of the answer. A flag that no registry defines is a 400 rather than a silently ignored parameter, because a filter that does nothing is indistinguishable from one that matched everything. """	txt	<code>registry_page(record_type, key, after=None)</code> and its docstring (177–184) — the shared helper behind every list route. The docstring carries two decisions: the facet runs before the window is cut (otherwise <code>total</code> would report the page size, not the size of the answer), and a flag no registry defines is a 400, because a filter that does nothing is indistinguishable from one that matched everything.
185–185	records, skipped = store.read_all(record_type)	L4	<code>records, skipped = store.read_all(record_type)</code> — the L4 delegation every route is made of, carrying the honest (records, skipped) pair.
186–188	if after is not None: after(records)	L4	187–188: <code>if after is not None: after(records)</code> — a function passed as a value (Practical 4): the caller hands in its own enrichment hook and this generic helper runs it. The after=attach_hospital pattern below lives on this line. 186 blank.
189–190	flag = request.args.get("flag", "")	X:flask	190: <code>flag = request.args.get("flag", "")</code> — the facet flag arrives on the query string (flask.md). 189 blank.
191–191	records, available, unknown = store.apply_facet(records, record_type, flag)	L4	<code>records, available, unknown = store.apply_facet(records, record_type, flag)</code> — three-result unpacking of the store layer's facet engine (triple tuple return).
192–195	if unknown: return error(f"unknown flag {flag!r}", 400, available=[f["flag"] for f in available])	X:flask	193–195: an unknown flag is a 400 whose payload includes <code>available=[f["flag"] for f in available]</code> — a list comprehension (P3) embedded in the error body, so the 400 teaches the client which flags exist (flask.md for the response). 192 blank.
196–201	body = paged(records, key, skipped, available) return body	L4	197–199: <code>body = paged(records, key, skipped, available)</code> then <code>return body</code> — slicing last, after the filter, is the ordering the docstring defends. 196 blank; 200–201 blanks.

202–209	def attach_visit_counts(records): """Add visit_count to each patient row, in place. The browse list needs it to say who has never been seen without asking the reader to open 600 records to find out. Counting here rather than in the frontend also means the "no consultation on file" facet and the row beside it cannot disagree. """	txt	<code>attach_visit_counts(records)</code> and its docstring (203–209): the browse list needs <code>visit_count</code> so it can say who has never been seen without opening 600 records; counting here also means the "no consultation on file" facet and the number beside the row cannot disagree — one computation feeds both.
210–211	visits, _ = store.read_all("visits") counts = {}	L4	210: <code>visits, _ = store.read_all("visits")</code> — the underscore discards the skip report deliberately: malformed visit rows must not block the count column. 211: <code>counts = {}</code> the accumulator.
212–215	for visit in visits: pid = visit["patient_id"] counts[pid] = counts.get(pid, 0) + 1	L3	213–215: the counting loop — <code>pid = visit["patient_id"]</code>, then <code>counts[pid] = counts.get(pid, 0) + 1</code>, the dict.get-default counting pattern (Practical 3): build a tally without ever checking <code>if key in dict</code> first. 212 blank.
216–218	for record in records: record["visit_count"] = counts.get(record["patient_id"], 0)	L3	217–218: the join loop — for every patient row, <code>record["visit_count"] = counts.get(record["patient_id"], 0)</code> — get's 0 default inlines the "never visited" case. 216 blank.
219–220		txt	Blank lines.
221–227	def attach_symptom_sets(records): """Parse each condition's symptom string into a sorted list, in place. The triage engine stores symptoms pipe-delimited; the condition browser wants chips it can sort and compare. Done on the way out so the raw column keeps the storage shape the CSV requires. """	txt	<code>attach_symptom_sets(records)</code> and its docstring (222–227): the triage engine stores symptoms pipe-delimited (the CSV-safe shape); the condition browser wants a list of chips it can sort. Converted on the way out, in place, so the raw column keeps its storage shape.
228–230	for record in records: record["symptom_set"] = sorted(triage.normalise_symptoms(record["symptoms"]))	L4	228–230: <code>record["symptom_set"] = sorted(triage.normalise_symptoms(record["symptoms"]))</code> — delegation to the triage layer (L4) wrapped in <code>sorted</code> (P3): chip lists come out alphabetical, view to view.
231–232		txt	Blank lines.
233–235	@app.get("/api/health") def health(): """Report that the API is up, with record counts per registry."""	X:flask	The health route: 233 <code>@app.get("/api/health")</code>, 234–235 handler and docstring (flask.md). This is what uptime checks and the frontend status lamp call.

236–236	`    counts = {}`	**L1**	<code>counts = {}</code> — the accumulator again.

237–242	`    for record_type in store.RECORDS:`
	`        try:`
	`            counts[record_type] = store.count_records(record_type)`
	`        except OSError:`
	`            counts[record_type] = None`
		L4	238–242: loop <code>store.RECORDS</code> (the registry table itself, read as data), and for each type <code>try: counts[type] = store.count_records(type) </code> / <code>except OSError: counts[type] = None</code> — a table that cannot be read reports None, telling the truth about a broken registry instead of refusing to report health at all. 237 blank.

243–248	`    return jsonify({`
	`        "status": "ok",`
	`        "counts": counts,`
	`        "datasets": analytics.passing_datasets(),`
	`    })`
		X:flask	244–248: the reply — <code>status: "ok"</code>, the counts, and <code>analytics.passing_datasets()</code> (L4 delegation) — the health page also proves the dataset pipeline loaded). 243 blank.

249–252	`# ---------- patients`	**txt**	249–250 blanks; 251 the patients section banner; 252 blank.

253–258	`@app.get("/api/patients")`
	`def list_patients():`
	`    """Return one page of patient records, plus the true total."""`
	`    return registry_page("patients", "patients", after=attach_visit_counts)`
		X:flask	The list route: 253 <code>@app.get("/api/patients")</code> (flask.md), 254–255 handler and docstring, 256 <code>return registry_page("patients", "patients", after=attach_visit_counts)</code> — the whole route is one L4 delegation call, with the enrich function passed along as the <code>after</code> hook. 257–258 blanks.

259–262	`@app.post("/api/patients")`
	`def create_patient():`
	`    """Add a patient. Only name is required."""`
	`    body = payload()`
		X:flask	The create route: 259 <code>@app.post("/api/patients")</code>, 260–261 handler and docstring ("Add a patient. Only name is required."), 262 <code>body = payload()</code> — the POST body read through the request object (flask.md).

263–263	`    cleaned, errors = registry.validate_record("patients", body)`	**L4**	<code>cleaned, errors = registry.validate_record("patients", body)</code> — the registry layer validates and normalises; the route never sees field rules.
264–266	`    if errors:` `        return error("validation failed", 400, errors=errors)`	**X:flask**	265–266: <code>if errors: return error("validation failed", 400, errors=errors)</code> — the error dict travels to the client so every failed field is visible at once (flask.md). 264 blank.

267–270	`    cleaned["patient_id"] = store.next_id("patients", "P-")`
	`    cleaned.setdefault("registered_on", "")`
	`    stored = store.append_row("patients", cleaned)`
		L4	268–270: <code>cleaned["patient_id"] = store.next_id("patients", "P-") </code>, then <code>cleaned.setdefault("registered_on", "")</code> — the setdefault (Practical 3) guarantees the column exists even when the caller skipped it — then <code>stored = store.append_row(...)</code>. 267 blank.

271–274	`    return jsonify({"created": full_record("patients", stored)}), 201`	**X:flask**	272: <code>return jsonify({"created": full_record("patients", stored)}), 201</code> — Created status, the whole row echoed back. 271 blank; 273–274 blanks.

275–281

```txt
@app.get("/api/patients/<patient_id>")
def get_patient(patient_id):
    """Return one patient, their visit history, and who they saw.

    Each visit is enriched with its doctor and hospital so the client does
    not have to make three more round trips to render one record.
    """
```

The detail route header (275–281): <code>@app.get("/api/patients/<patient_id>")</code>, handler, docstring — return one patient, their visit history, and who they saw, so the client does not have to make three more round trips to render one record.

282–282

```L4
    record = store.get_by_id("patients", patient_id)
```

<code>record = store.get_by_id("patients", patient_id)</code> — the linear scan; None means absent.

283–285

```X:flask
    if record is None:
        return error(f"no patient with id {patient_id}", 404)
```

284–285: <code>if record is None: return error(f"no patient with id {patient_id}", 404)</code> — the honest 404 with the id in the message (flask.md). 283 blank.

286–288

```L4
    visits = store.visits_for(patient_id)
```

287: <code>visits = store.visits_for(patient_id)</code> — the store helper that also sorts by date/time. 286 and 288 blanks.

289–290

```L4
    doctors, _ = store.read_all("doctors")
    hospitals, _ = store.read_all("hospitals")
```

289–290: read the whole <code>doctors</code> and <code>hospitals</code> tables once — two reads instead of 3×N lookups.

291–292

```L3
    by_doctor = {d["doctor_id"]: d for d in doctors}
    by_hospital = {h["hospital_id"]: h for h in hospitals}
```

291–292: two dict comprehensions (Practical 3) build indexes: <code>by_doctor = {d["doctor_id"]: d for d in doctors}</code>, same for hospitals. The join structure of a CSV world in two lines.

293–296

```L3
    for visit in visits:
        doctor = by_doctor.get(visit["doctor_id"], {})
        hospital = by_hospital.get(visit["hospital_id"], {})
```

294–296: for each visit, <code>doctor = by_doctor.get(visit["doctor_id"], {})</code> and the hospital twin — <code>.get</code> with an empty dict default so a dangling id yields blanks, never a KeyError. 293 blank.

297–306

```L3
        visit["doctor"] = {
            "doctor_id": doctor.get("doctor_id", ""),
            "name": doctor.get("name", ""),
            "specialisation": doctor.get("specialisation", ""),
            "qualification": doctor.get("qualification", ""),
            "department": doctor.get("department", ""),
            "room_no": doctor.get("room_no", ""),
            "consultation_fee": doctor.get("consultation_fee", ""),
        }
```

298–306: <code>visit["doctor"]</code> becomes an embedded dict literal whose every value is <code>doctor.get(field, "")</code> — seven fields, all blank-safe. The client gets exactly the fields the UI needs, shaped at the server. 297 blank.

307–312	`        visit["hospital"] = {`
	`            "hospital_id": hospital.get("hospital_id", ""),`
	`            "name": hospital.get("name", ""),`
	`            "area": hospital.get("area", ""),`
	`            "city": hospital.get("city", ""),`
	`        }`
		L3	307–312: the hospital card, the identical pattern — four <code>.get(field, "")</code> entries. Two dict literals, copied structure, one honest filler value: the empty string.

313–318	`    return jsonify({`
	`        "patient": record,`
	`        "visit_count": len(visits),`
	`        "visits": visits,`
	`    })`
		X:flask	314–318: the reply — <code>patient</code>, <code>visit_count: len(visits)</code>, <code>visits</code> (flask.md jsonify). The count rides along so the page can show "N consultations" without counting in JavaScript. 313 blank.

319–320		**txt**	Blank lines.

321–328	`@app.patch("/api/patients/<patient_id>")`
	`def update_patient(patient_id):`
	`    """Change some fields of an existing patient.`
	
	`    A partial update is not a create, so required-field checks do not`
	`    apply. Sending only a city must not fail because no name was included.`
	`    Field format checks still run, so age="old" is still rejected.`
	`    """`
		txt	The update route header (321–328): <code>@app.patch</code>, handler, and a docstring that carries a real rule — a partial update is not a create, so required-field checks do not apply (sending only a city must not fail for lack of a name), but field format checks still run, so <code>age="old"</code> is still rejected.

329–332	`    body = payload()`
	
	`    if not body:`
	`        return error("no fields to update")`
		X:flask	329 reads the body (flask.md); 331–332 <code>if not body: return error("no fields to update")</code> — an empty patch is refused before any validation runs (the P1 truthiness of a dict). 330 blank.

333–335	`    cleaned, errors = registry.validate_record("patients", body,`
	
	`require_required=False)`
		L4	334–335: <code>registry.validate_record("patients", body, require_required=False)</code> — the keyword argument (Practical 4) switches off only the required-field half of validation. 333 blank.

336–338	`    if errors:` `        return error("validation failed", 400, errors=errors)`	**X:flask**	337–338: format failures are still a 400 with the error list (flask.md). 336 blank.
339–340	`    cleaned.pop("patient_id", None)`	**L3**	340: <code>cleaned.pop("patient_id", None)</code> — pop with a default (P3): the id cannot be edited through a patch, and a patch without one must not crash on the pop.
341–343	`    if not cleaned:` `        return error("no updatable fields supplied")`	**X:flask**	342–343: <code>if not cleaned: return error("no updatable fields supplied")</code> — where everything the caller sent was an id or an unknown field, there is genuinely nothing to write. 341 blank.

344–346	`    updated = store.update_row("patients", "patient_id", patient_id, cleaned)`	**L4**	345: <code>updated = store.update_row("patients", "patient_id", patient_id, cleaned)</code> — the merge-and-overwrite rewrite; None means no row matched. 344, 346 blanks.
347–348	`    if updated is None:` `        return error(f"no patient with id {patient_id}", 404)`	**X:flask**	347–348: the None sentinel (P1) becomes the 404 — <code>error(f"no patient with id {patient_id}", 404)</code> (flask.md).
349–352	`    return jsonify({"updated": updated})`	**X:flask**	350: <code>return jsonify({"updated": updated})</code> — the stored row as it now reads on disk. 349 blank; 351–352 blanks.

353–355	`@app.delete("/api/patients/<patient_id>")`
	`def delete_patient(patient_id):`
	`    """Remove a patient row."""`
		txt	The delete route (353–355): <code>@app.delete</code>, handler, one-line docstring — Remove a patient row.

356–359	`    if not store.delete_row("patients", "patient_id", patient_id):`
	`        return error(f"no patient with id {patient_id}", 404)`
	
	`    return jsonify({"deleted": patient_id})`
		X:flask	356–358: <code>if not store.delete_row(...)</code> — the store's False is the L4 delegation; False becomes <code>error(..., 404)</code>, True falls through to <code>jsonify({"deleted": patient_id})</code>. The whole route, four lines (flask.md). 359 blank.

360–363	`# ---------- visits`	**txt**	360–361 blanks; 362 the visits section banner; 363 blank.

364–366	`@app.get("/api/visits")`
	`def list_visits():`
	`    """Return visits, optionally filtered by patient_id."""`
		txt	<code>@app.get("/api/visits")</code> / <code>list_visits</code> (364–366): docstring — return visits, optionally filtered by patient_id. Visits are the table that grows fastest, so the filter matters.

367–367	`    records, skipped = store.read_all("visits")`	**L4**	<code>records, skipped = store.read_all("visits")</code> — the full history, with the skip report kept this time.
368–369	`    patient_id = request.args.get("patient_id")`	**X:flask**	369: <code>patient_id = request.args.get("patient_id")</code> (flask.md). 368 blank.
370–371	`    if patient_id:` `        records = [r for r in records if r["patient_id"] == patient_id]`	**L3**	370–371: <code>if patient_id: records = [r for r in records if r["patient_id"] == patient_id]</code> — a list comprehension as the filter (Practical 3), comparison on the plain string field
372–375	`    return paged(records, "visits", skipped)`	**L4**	373: <code>return paged(records, "visits", skipped)</code> — the shared paging helper, skip report handed through. 372 blank; 374–375 blanks.

376–378	`@app.post("/api/visits")`
	`def create_visit():`
	`    """Append a visit. Requires patient_id and visit_date."""`
		txt	<code>@app.post("/api/visits")</code> / <code>create_visit</code> (376–378): docstring — append a visit; requires patient_id and visit_date.

379–379	`    body = payload()`	**X:flask**	<code>body = payload()</code> — the POST body (flask.md).
380–380	`    cleaned, errors = registry.validate_record("visits", body)`	**L4**	<code>cleaned, errors = registry.validate_record("visits", body)</code> — bring the row into schema shape.

381–384	`    for field in ("patient_id", "visit_date"):`
	`        if not cleaned.get(field, ""):`
	`            errors.append(f"{field} is required")`
		L1	382–384: the belt-and-braces loop — <code>for field in ("patient_id", "visit_date"):</code> iterates a tuple literal (P2), the falsy check <code>if not cleaned.get(field, ""):</code> catches blank and absent alike, and <code>errors.append(f"{field} is required")</code> grows the error list. Clinic-level validation written in four Practical 1/2 lines. 381 blank.

385–387	`    if errors:` `        return error("validation failed", 400, errors=errors)`	**X:flask**	386–387: any error at all — registry's or the loop's — is the same 400 with <code>errors=errors</code> (flask.md). 385 blank.
388–389	`    patient = store.get_by_id("patients", cleaned["patient_id"])`	**L4**	389: <code>patient = store.get_by_id("patients", cleaned["patient_id"])</code> — a visit must point at a real patient. 388 blank.
390–391	`    if patient is None:` `        return error(f"no patient with id {cleaned['patient_id']}", 404)`	**X:flask**	390–391: unknown patient → <code>error(...404)</code> with the id in the message (flask.md).
392–394	`    cleaned["visit_id"] = store.next_id("visits", "V-")` `    stored = store.append_row("visits", cleaned)`	**L4**	393–394: <code>cleaned["visit_id"] = store.next_id("visits", "V-")</code> then <code>store.append_row("visits", cleaned)</code> — numbered and stored. 392 blank.
395–398	`    return jsonify({"created": full_record("visits", stored)}), 201`	**X:flask**	396: <code>return jsonify({"created": full_record("visits", stored)}), 201</code> — the 21-column row, blanks filled, echoed as Created. 395 blank; 397–398 blanks.
399–400	`# ---------- medicines`	**txt**	399 the medicines section banner; 400 blank.

401–404	`@app.get("/api/medicines")`
	`def list_medicines():`
	`    """Return one page of the medicine catalogue."""`
	`    return registry_page("medicines", "medicines")`
		X:flask	<code>@app.get("/api/medicines")</code> / <code>list_medicines</code> (401–403), 404 <code>return registry_page("medicines", "medicines")</code> — no enrich hook at all: the plain registry page.

405–410	`@app.post("/api/medicines")`
	`def create_medicine():`
	`    """Add a medicine to the catalogue."""`
	`    body = payload()`
		X:flask	405–406 blanks; 407–410 the create route <code>@app.post("/api/medicines")</code>, handler, docstring, <code>body = payload()</code> — same shape as create_patient (flask.md + L4 delegation below).

411–411	`    cleaned, errors = registry.validate_record("medicines", body)`	**L4**	<code>registry.validate_record("medicines", body)</code> — schema shape first.
412–414	`    if errors:` `        return error("validation failed", 400, errors=errors)`	**X:flask**	413–414: errors → 400 with the field list (flask.md). 412 blank.
415–417	`    cleaned["medicine_id"] = store.next_id("medicines", "M-")` `    stored = store.append_row("medicines", cleaned)`	**L4**	416–417: <code>cleaned["medicine_id"] = store.next_id("medicines", "M-")</code> and <code>store.append_row(...)</code>. 415 blank.
418–421	`    return jsonify({"created": full_record("medicines", stored)}), 201`	**X:flask**	419: <code>return jsonify({"created": full_record("medicines", stored)}), 201</code> — the 11-column row echoed. 418 blank; 420–421 blanks.

422–424	`@app.get("/api/medicines/<medicine_id>")`
	`def get_medicine(medicine_id):`
	`    """Return one medicine, with the stores stocking it."""`
		txt	<code>@app.get("/api/medicines/<medicine_id>")</code> / <code>get_medicine</code> (422–424): docstring — one medicine, with the stores stocking it.

425–425	`    record = store.get_by_id("medicines", medicine_id)`	L4	<code>record = store.get_by_id("medicines", medicine_id)</code>.
426–428	`    if record is None:` `        return error(f"no medicine with id {medicine_id}", 404)`	X:flask	427–428: None → <code>error(f"no medicine with id {medicine_id}", 404)</code> (flask.md). 426 blank.
429–431	`    medicines, _ = store.read_all("medicines")` `    stores, _ = store.read_all("stores")`	L4	430–431: read the medicines and stores tables whole — the inputs the stocking question needs. 429 blank.
432–432	`    stocked = registry.stores_with_medicine(record["name"], medicines, stores)`	L4	<code>stocked = registry.stores_with_medicine(record["name"], medicines, stores)</code> — the L4 delegation; matching is by name because the store CSV records medicine names, not ids.
433–436	`    return jsonify({"medicine": record, "stocked_by": stocked})`	X:flask	434: <code>return jsonify({"medicine": record, "stocked_by": stocked})</code> (flask.md). 433 blank; 435–436 blanks.

437–439	`@app.get("/api/medicines/<medicine_id>/stores")`
	`def medicine_stores(medicine_id):`
	`    """Return only the stores stocking a given medicine."""`
		txt	<code>@app.get("/api/medicines/<medicine_id>/stores")</code> / <code>medicine_stores</code> (437–439): docstring — return only the stores stocking a given medicine. Same data as above, trimmed to a different question the UI asks.

440–440	`    record = store.get_by_id("medicines", medicine_id)`	L4	<code>record = store.get_by_id("medicines", medicine_id)</code> again.
441–443	`    if record is None:` `        return error(f"no medicine with id {medicine_id}", 404)`	X:flask	442–443: None → the same shaped 404, message naming the id (flask.md). 441 blank.

444–447	`    medicines, _ = store.read_all("medicines")`
	`    stores, _ = store.read_all("stores")`
	`    stocked = registry.stores_with_medicine(record["name"], medicines, stores)`
		L4	445–447: two table reads, then the same <code>registry.stores_with_medicine(...)</code> call — identical logic to get_medicine, deliberately reusing the registry helper rather than re-deriving. 444 blank.

448–450	`    return jsonify({"medicine_id": medicine_id, "name": record["name"],` `"count": len(stocked), "stores": stocked})`	X:flask	449–450: the reply — <code>count</code> plus the <code>stores</code> list, a response shaped for a chips row rather than a full page (flask.md).
451–454	`# ---------------------------------------------------------- stores`	txt	451–452 blanks; 453 the stores section banner; 454 blank.

455–458	`@app.get("/api/stores")`
	`def list_stores():`
	`    """Return one page of medical stores."""`
	`    return registry_page("stores", "stores")`
		X:flask	<code>@app.get("/api/stores")</code> / <code>list_stores</code> (455–457), 458 <code>return registry_page("stores", "stores")</code> — the registry-page helper again, fourth use.

459–464	`@app.post("/api/stores")`
	`def create_store():`
	`    """Add a medical store."""`
	`    body = payload()`
		X:flask	459–460 blanks; 461–464 the create route header — <code>@app.post("/api/stores")</code>, handler, docstring, <code>body = payload()</code> (flask.md).

465–465	`    cleaned, errors = registry.validate_record("stores", body)`	**L4**	<code>registry.validate_record("stores", body)</code>.
466–468	`    if errors:` `        return error("validation failed", 400, errors=errors)`	**X:flask**	467–468: errors → 400 (flask.md). 466 blank.
469–471	`    cleaned["store_id"] = store.next_id("stores", "S-")` `    stored = store.append_row("stores", cleaned)`	**L4**	470–471: <code>store.next_id("stores", "S-")</code> then <code>store.append_row(...)</code>. 469 blank.
472–475	`    return jsonify({"created": full_record("stores", stored)}), 201`	**X:flask**	473: <code>jsonify</code>, status 201, whole row echoed (flask.md). 472 blank; 474–475 blanks.

476–478	`@app.get("/api/stores/<store_id>")`
	`def get_store(store_id):`
	`    """Return one store with the medicines it stocks resolved."""`
		txt	<code>@app.get("/api/stores/<store_id>")</code> / <code>get_store</code> (476–478): docstring — one store with the medicines it stocks resolved.

479–479	`    record = store.get_by_id("stores", store_id)`	**L4**	<code>record = store.get_by_id("stores", store_id)</code>.
480–482	`    if record is None:` `        return error(f"no store with id {store_id}", 404)`	**X:flask**	481–482: None → 404, id named in the message (flask.md). 480 blank.
483–485	`    medicines, _ = store.read_all("medicines")` `    held = registry.stock_set_for(record)`	**L4**	484–485: read the medicines table, then <code>held = registry.stock_set_for(record)</code> — the registry helper turns the store's pipe-list into a set.
486–486	`    stocked = [m for m in medicines if m["medicine_id"] in held]`	**L3**	<code>stocked = [m for m in medicines if m["medicine_id"] in held]</code> — the comprehension (P3) whose filter is a set membership test: the set lookup makes the scan cheap and reads like English.

487–491	`    return jsonify({"store": record, "stock_count": len(stocked),`
	
	`"medicines": stocked})`
		X:flask	488–489: <code>jsonify</code> with <code>stock_count</code> and the medicine rows (flask.md). 487 blank; 490–491 blanks.

492–493	`# ------------------------------------------------------------` `hospitals`	**txt**	492 the hospitals section banner; 493 blank.

494–497	`@app.get("/api/hospitals")`
	`def list_hospitals():`
	`    """Return one page of hospitals."""`
	`    return registry_page("hospitals", "hospitals")`
		X:flask	<code>@app.get("/api/hospitals")</code> / <code>list_hospitals</code> (494–496), 497 <code>registry_page("hospitals", "hospitals")</code> — the pattern is now unmistakable: one helper, five registries.

498–503	`@app.post("/api/hospitals")`
	`def create_hospital():`
	`    """Add a hospital."""`
	`    body = payload()`
		X:flask	498–499 blanks; 500–503 the create route header — <code>@app.post("/api/hospitals")</code>, handler, docstring, <code>body = payload()</code> (flask.md).

504–504	`    cleaned, errors = registry.validate_record("hospitals", body)`	**L4**	<code>registry.validate_record("hospitals", body)</code>.
505–507	`    if errors:` `        return error("validation failed", 400, errors=errors)`	**X:flask**	506–507: errors → 400 (flask.md). 505 blank.
508–510	`    cleaned["hospital_id"] = store.next_id("hospitals", "H-")` `    stored = store.append_row("hospitals", cleaned)`	**L4**	509–510: <code>store.next_id("hospitals", "H-")</code>, <code>store.append_row(...)</code>. 508 blank.
511–514	`    return jsonify({"created": full_record("hospitals", stored)}), 201`	**X:flask**	512: <code>jsonify</code>, 201, full row echoed (flask.md). 511 blank; 513–514 blanks.

515–517	`@app.get("/api/hospitals/<hospital_id>")`
	`def get_hospital(hospital_id):`
	`    """Return one hospital with the doctors practising there."""`
		txt	<code>@app.get("/api/hospitals/<hospital_id>")</code> / <code>get_hospital</code> (515–517): docstring — one hospital with the doctors practising there.

518–518	`    record = store.get_by_id("hospitals", hospital_id)`	**L4**	<code>record = store.get_by_id("hospitals", hospital_id)</code>.
519–521	`    if record is None:` `        return error(f"no hospital with id {hospital_id}", 404)`	**X:flask**	520–521: None → 404 (flask.md). 519 blank.
522–523	`    doctors = store.doctors_at(hospital_id)`	**L4**	523: <code>doctors = store.doctors_at(hospital_id)</code> — the store helper that filters and date-sorts the hospital's roster. 522 blank.

524–526	`    return jsonify({"hospital": record, "doctor_count": len(doctors),`
	
	`"doctors": doctors})`
		X:flask	525–526: <code>jsonify</code> with <code>doctor_count</code> and the doctor rows (flask.md).

527–530	`# ---------------------------------------------------------- doctors`	**txt**	527–528 blanks; 529 the doctors section banner; 530 blank.

531–538	def attach_hospital(doctors): """Copy the hospital's name, area and city onto each doctor row. A doctor is only identified by the hospital they work at, so those three fields have to be present *before* the search runs. Joining afterwards means a query for a hospital or a locality can never match, even though the interface offers both. """	txt	<code>attach_hospital(doctors)</code> and its docstring (532–538): copy the hospital's name, area and city onto each doctor row. The docstring gives the crucial ordering rule — those fields must be present before the search runs, since a doctor is only identified by the hospital they work at; joining afterwards would make queries for a hospital or a locality unmatchable.
539–539	hospitals, _ = store.read_all("hospitals")	L4	<code>hospitals, _ = store.read_all("hospitals")</code> — the hospitals table, skip report discarded.
540–540	by_hospital = {h["hospital_id"]: h for h in hospitals}	L3	<code>by_hospital = {h["hospital_id"]: h for h in hospitals}</code> — the dict comprehension index (P3), the join key of the whole operation.
541–546	for row in doctors: hospital = by_hospital.get(row["hospital_id"], {}) row["hospital_name"] = hospital.get("name", "") row["hospital_area"] = hospital.get("area", "") row["hospital_city"] = hospital.get("city", "")	L3	542–546: <code>for row in doctors:</code> — <code>hospital = by_hospital.get(row["hospital_id"], {})</code>, then three copies — <code>row["hospital_name"] = hospital.get("name", "")</code> etc. The <code>{}</code> default so an unlinked doctor gets blanks, not a crash — the same .get discipline throughout. 541 blank.
547–550	return doctors	L4	548: <code>return doctors</code> — the same list, enriched in place. 547 blank; 549–550 blanks.
551–554	@app.get("/api/doctors") def list_doctors(): """Return one page of doctors, with their hospital attached.""" return registry_page("doctors", "doctors", after=attach_hospital)	X:flask	<code>@app.get("/api/doctors")</code> / <code>list_doctors</code> (551–553), 554 <code>return registry_page("doctors", "doctors", after=attach_hospital)</code> — the route that actually needs the hook.
555–560	@app.post("/api/doctors") def create_doctor(): """Add a doctor.""" body = payload()	X:flask	555–556 blanks; 557–560 the create route header — <code>@app.post("/api/doctors")</code>, handler, docstring, <code>body = payload()</code> (flask.md).
561–561	cleaned, errors = registry.validate_record("doctors", body)	L4	<code>registry.validate_record("doctors", body)</code>.
562–564	if errors: return error("validation failed", 400, errors=errors)	X:flask	563–564: errors → 400 (flask.md). 562 blank.

565–567	L4	`    cleaned["doctor_id"] = store.next_id("doctors", "D-")` `    stored = store.append_row("doctors", cleaned)`	566–567: <code>store.next_id("doctors", "D-")</code>, <code>store.append_row(...)</code>. 565 blank.
568–571	X:flask	`    return jsonify({"created": full_record("doctors", stored)}), 201`	569: <code>jsonify</code>, 201, full row echoed (flask.md). 568 blank; 570–571 blanks.

572–574	txt	`@app.get("/api/doctors/<doctor_id>")`
		`def get_doctor(doctor_id):`
		`    """Return one doctor, their hospital, and their recent consultations."""`
			<code>@app.get("/api/doctors/<doctor_id>")</code> / <code>get_doctor</code> (572–574): docstring — one doctor, their hospital, and their recent consultations.

575–575	L4	`    record = store.get_by_id("doctors", doctor_id)`	<code>record = store.get_by_id("doctors", doctor_id)</code>.
576–578	X:flask	`    if record is None:` `        return error(f"no doctor with id {doctor_id}", 404)`	577–578: None → 404, message naming the id (flask.md). 576 blank.
579–580	L1	`    hospital = store.get_by_id("hospitals", record["hospital_id"]) or {}`	580: <code>hospital = store.get_by_id("hospitals", record["hospital_id"]) or {} </code> — the <code>or {} </code> idiom (P1): a falsy None becomes an empty dict, so the code below can always call <code>.get</code> on it.
581–583	L4	`    visits = store.visits_for_doctor(doctor_id)` `    patients, _ = store.read_all("patients")`	582–583: <code>visits = store.visits_for_doctor(doctor_id)</code> (newest first), and the patients table read whole. 581 blank.
584–584	L3	`    by_patient = {p["patient_id"]: p for p in patients}`	<code>by_patient = {p["patient_id"]: p for p in patients}</code> — the index comprehension again (P3), third use.

585–588	L3	`    for visit in visits:`
		`        patient = by_patient.get(visit["patient_id"], {})`
		`        visit["patient_name"] = patient.get("name", "")`
			586–588: for each visit, <code>patient = by_patient.get(visit["patient_id"], {})</code> and <code>visit["patient_name"] = patient.get("name", "")</code> — one human name per consultation, blank-safe. 585 blank.

589–595	X:flask	`    return jsonify({`
		`        "doctor": dict(record, hospital_name=hospital.get("name", "")),`
		`        "hospital": hospital,`
		`        "visit_count": len(visits),`
		`        "visits": visits,`
		`    })`
			590–595: the reply (flask.md). The interesting line is 591: <code>"doctor": dict(record, hospital_name=hospital.get("name", ""))</code> — <code>dict(mapping, **kw)</code> copies a dict and adds keys in one call (Practical 3 dict use): the doctor row plus the hospital name, without touching the original.

596–599	txt	`# ---------- rare conditions`	596–597 blanks; 598 the rare conditions section banner; 599 blank.

600–608

```txt
@app.get("/api/rare-conditions")
def list_rare_conditions():
    """One page of the HPO derived registry, plus its shape.

    The registry is 11,655 diseases. The figures ride along with the page
    rather than sitting on a second request, because the About card and the
    list must not be able to describe different sizes, and both come free
    from the same pass over the same rows.
    """
```

<code>@app.get("/api/rare-conditions")</code> / <code>list_rare_conditions</code> with its docstring (602–608): one page of the HPO-derived registry plus its shape — the figures ride along with the page rather than sitting on a second request, because the About card and the list must not be able to describe different sizes; both come free from the same pass over the same rows.

609–610

```L4
    records, _ = store.read_all("rare_conditions")
    vocabulary, _ = store.read_all("hpo_symptoms")
```

609–610: read the <code>rare_conditions</code> table and the <code>hpo_symptoms</code> vocabulary — the two files this view joins.

611–616

```L3
    by_hpo = {}
    for row in records:
        for hpo_id in row["hpo_ids"].split("|"):
            if hpo_id:
                by_hpo[hpo_id] = by_hpo.get(hpo_id, 0) + 1
```

612–616: the HPO counting loop — <code>by_hpo = {}</code>, a nested <code>for row ... for hpo_id in row["hpo_ids"].split("|")</code> (the P2 split feeding a P3 tally), <code>if hpo_id:</code> skips empties, and <code>by_hpo[hpo_id] = by_hpo.get(hpo_id, 0) + 1</code> — the get-default counting pattern. 611 blank.

617–619

```L3
    lay = {v["hpo_id"]: (v["lay_term"] or v["term"]).lower()
           for v in vocabulary}
```

618–619: <code>lay = {v["hpo_id"]: (v["lay_term"] or v["term"]).lower() for v in vocabulary}</code> — a comprehension (P3) whose value expression uses the <code>or</code> fallback (P1): prefer the lay wording, else the formal term, always lowercased.

620–628

```L3
    common = []
    for hpo_id, count in sorted(by_hpo.items(),

key=lambda kv: -kv[1])[:12]:
        common.append({
            "hpo_id": hpo_id,
            "word": lay.get(hpo_id, hpo_id.lower()),
            "diseases": count,
        })
```

621–628: the twelve most common findings — <code>sorted(by_hpo.items(), key=lambda kv: -kv[1])[:12]</code> negates the count inside the sort key (the P3 sorted(key=) trick for descending order), then builds a card dict per finding: <code>word</code> falls back to <code>hpo_id.lower()</code> when the vocabulary misses. 620 blank.

629–635

```L3
    inheritance = {}

    for row in records:
        for mode in row["inheritance_mode"].split("|"):
            if mode:
                inheritance[mode] = inheritance.get(mode, 0) + 1
```

630–635: the inheritance tally — the identical nested-loop counting pattern applied to <code>inheritance_mode</code>, again over a pipe-split. Same shape, second use — pattern reuse as policy. 629 blank.

636–641	txt	`    # One page of the registry, in the same shape every other lis t endpoint`
		`    # returns, so the interface keeps one b rowse component inste ad of`
		`    # growing a secon d one for this view. The figures travel wi th the page`
		`    # rather than sit ting on their own req uest, so the About ca rd and the`
		`    # list cannot end up describing differe nt sizes.`
			The comment (637–641): one page of the registry in the same shape every other list endpoint returns, so the interface keeps one browse component instead of growing a second one; the figures travel with the page so the About card and the list cannot end up describing different sizes. 636 blank.

642–643	L2	`    _limit, _offset = page_args()` `    window = records [_offset:_offset + _l imit]`	642–643: <code>_limit, _offset = page_args()</code> then <code>window = records[_offset:_offset + _limit]</code> — the slice (P2) done by hand here because this route needs its own body shape.

644–651	X:flask	`    return jsonify({`
		`        "count": len (window),`
		`        "total": len (records),`
		`        "offset": _of fset,`
		`        "limit": _lim it,`
		`        "has_more": _ offset + len(window) < len(records),`
		`        "conditions": window,`
			645–651: <code>jsonify</code> opens the reply: <code>count</code>, <code>total</code>, <code>offset</code>, <code>limit</code>, <code>has_more</code> and the <code>conditions</code> window — the same six fields <code>paged</code> would produce (flask.md). 644 blank.

652–652	L3	`        "facets": [],`	<code>"facets": []</code> — an empty list keeps the reply shape uniform: this registry defines no facets, and the frontend can rely on the key existing.

653–658	L3	`        "with_mondo": sum(1 for r in record s if r["mondo_id"]),`
		`        "with_inherit ance": sum(1 for r in records if r["inherit ance_mode"]),`
		`        "distinct_fin dings": len(by_hpo),`
		`        "lay_wordin g": sum(1 for v in vo cabulary`
		
		`if v["has_lay_wordin g"] == "yes"),`
		`        "common_findi ngs": common,`
			653–657: the four shape numbers, each built from the already-loaded rows — <code>sum(1 for r in records if r["mondo_id"])</code> counts truthy cells over a generator inside sum (P3 comprehension-vocabulary), likewise <code>with_inheritance</code> and <code>lay_wording</code>; <code>distinct_findings</code> is <code>len(by_hpo)</code>. 658 blank.

659–663	L3	`        "inheritance_ modes": sorted(`
		`            [{"mode": k, "diseases": v} for k, v in inheritance.i tems()],`
		`            key=lambd a x: -x["diseases"],`
		`        )[:8],`
		`    })`
			659–663: <code>inheritance_modes</code> — a list of {mode, diseases} dicts (P3) sorted with the same negated <code>key=lambda x: -x["diseases"]</code>, trimmed <code>[:8]</code>, then the dict closes. A statistics block built entirely from Practical 3 vocabulary.

664–665	txt		Blank lines.

666–668	txt	`@app.get("/api/rare-conditions/<condition_id>")`
		`def get_rare_condition(condition_id):`
		`    """One rare disease, with its findings expanded into words."""`
			<code>@app.get("/api/rare-conditions/<condition_id>")</code> / <code>get_rare_condition</code> (666–668): docstring — one rare disease with its findings expanded into words.

669–669	L4	`    record = store.get_by_id("rare_conditions", condition_id)`	<code>record = store.get_by_id("rare_conditions", condition_id)</code>.
670–672	X:flask	`    if record is None:` `        return error(f"no condition with id {condition_id}", 404)`	671–672: None → the shaped 404 (flask.md). 670 blank.
673–674	L4	`    vocabulary, _ = store.read_all("hpo_symptoms")`	674: <code>vocabulary, _ = store.read_all("hpo_symptoms")</code> — the HPO lookup table. 673 blank.

675–677	L3	`    by_id = {v["hpo_id"]: v for v in vocabulary}`
		
		`    findings = []`
			675: <code>by_id = {v["hpo_id"]: v for v in vocabulary}</code> (P3); 677: <code>findings = []</code> the accumulator. 676 blank.

678–680	L3	`    for hpo_id, word in zip(record["hpo_ids"].split("|"),` `record["symptoms"].split("|")):`	679–680: <code>for hpo_id, word in zip(record["hpo_ids"].split("|"), record["symptoms"].split("|")):</code> — zip walks the two pipe lists in lockstep, so each finding keeps its id and its wording paired (P3 sequence vocabulary). 678 blank.
681–681	L3	`        term = by_id.get(hpo_id, {})`	<code>term = by_id.get(hpo_id, {})</code> — the blank-safe lookup once more.

682–689	L3	`        findings.append({`
		`            "hpo_id": hpo_id,`
		`            "word": word,`
		`            "term": term.get("term", ""),`
		`            "lay_term": term.get("lay_term", ""),`
		`            "is_lay_wording": term.get("has_lay_wording") == "yes",`
		`        })`
			683–689: <code>findings.append</code> one dict per finding — <code>word</code> from the CSV, <code>term</code> and <code>lay_term</code> from the vocabulary, and <code>is_lay_wording = term.get("has_lay_wording") == "yes"</code>, the True/False cast from a string cell (P1 comparison producing a Boolean). 682 blank.

690–695	X:flask	`    return jsonify({`
		`        "condition": record,`
		`        "findings": findings,`
		`        "inheritance": [m for m in record["inheritance_mode"].split("|") if m],`
		`    })`
			691–695: the reply — <code>condition</code>, <code>findings</code>, and <code>inheritance</code> as a comprehension splitting the pipe column and dropping blanks (694, P3) inside the jsonify (flask.md). 690 blank.

696–699	txt	`# ------------------------------------------------------------ search`	696–697 blanks; 698 the search section banner; 699 blank.

700–707	txt	`@app.get("/api/search/<record_type>")`
		`def search(record_type):`
		`    """Fuzzy search a registry. Returns everything scoring above zero.`
		
		`    A query matching nothing is a 200 with count 0, not a 404. An empty`
		`    query is a 400, since that is a malformed request rather than an honest negative result.`
		`    """`
			<code>@app.get("/api/search/<record_type>")</code> / <code>search</code> with its docstring (702–707): fuzzy search a registry, returning everything scoring above zero. The contract: query matching nothing is a 200 with count 0, not a 404; an empty query is a 400, since that is a malformed request rather than an honest negative.

708–710	X:flask	`    if record_type not in SEARCHABLE:`
		`        return error(f"cannot search {record_type}", 404,`
		
		`searchable=list(SEARCHABLE))`
			708–710: <code>if record_type not in SEARCHABLE: return error(..., 404, searchable=list(SEARCHABLE))</code> — a tuple membership test (P3) guards the path parameter, and the error body lists the allowed values (flask.md).

711–712	X:flask	`    query = request.args.get("q", "")`	712: <code>query = request.args.get("q", "")</code> — the query from the request object (flask.md). 711 blank.
713–715	X:flask	`    if not query.strip():` `        return error("query parameter q is required and cannot be blank")`	714–715: <code>if not query.strip(): return error("query parameter q is required and cannot be blank")</code> — whitespace-only input counts as missing (P2 strip, P1 falsiness). 713 blank.
716–717	L4	`    records, _ = store.read_all(record_type)`	717: <code>records, _ = store.read_all(record_type)</code> — read whichever registry was named. 716 blank.

718–721	L4	`    # Joined before searching, not after. See attach_hospital.`
		`    if record_type == "doctors":`
		`        attach_hospital(records)`
			718–721: the comment — joined before searching, not after (see attach_hospital) — and <code>if record_type == "doctors": attach_hospital(records)</code>: hospital name, area and city must already be on the row or the search can never see them.

722–723	L4	`    results = registry.search_records(records, query, record_type)`	723: <code>results = registry.search_records(records, query, record_type)</code> — the scoring itself lives in the registry layer; the route only babysits the request. 722 blank.
724–725	txt	`    # Page size is honoured only when it would not hide a short result set.`	724 blank; 725 the comment: page size is honoured only when it would not hide a short result set.
726–727	L1	`    try:` `        limit = int(request.args.get("limit", SEARCH_PAGE_SIZE))`	726–727: <code>try: limit = int(request.args.get("limit", SEARCH_PAGE_SIZE))</code> — the cast-with-default (P1) on the request parameter.
728–729	X:flask	`    except ValueError:` `        return error("limit must be a whole number")`	728–729: <code>except ValueError: return error("limit must be a whole number")</code> — a non-numeric limit is a 400, not a defaulting (flask.md): unlike paging, an explicit bad value should say so here.
730–731	L1	`    limit = max(1, min(limit, 500))`	731: <code>limit = max(1, min(limit, 500))</code> — the clamp again (P1), floor 1 so <code>?limit=0</code> cannot request an empty page. 730 blank.
732–733	L1	`    total = len(results)`	733: <code>total = len(results)</code> — the true matched count, before trimming. 732 blank.

734–734

```
    page = results[:limit]
```

L2

<code>page = results[:limit]</code> — the slice (P2) that keeps only what will be sent.

735–743

```
    return jsonify({
        "query": query,
        "record_type": record_type,
        "count": len(page),
        "total_matched": total,
        "truncated": total > len(page),
        "results": page,
    })
```

X:flask

736–743: the reply (flask.md) — <code>query</code> echoed, <code>record_type</code>, <code>count</code>, <code>total_matched</code>, and 741 <code>"truncated": total > len(page)</code>, the Boolean (P1) that tells the interface whether more matching rows exist.

744–747

```
# ---------------------------------------------------------- triage
```

txt

744–745 blanks; 746 the triage section banner; 747 blank.

748–755

```
# ---------------------------------------------------------- disease articles
#
# A condition page is one written article plus four things that are computed
# rather than typed: what else it could be, who has had it, what treats it,
# and where to get it. That is the whole point of the page. A summary on its
# own is a paragraph; a summary with the differentials and the pharmacy
# counter attached is something a person can act on.
```

txt

748–749 the disease-articles banner and hash rule; 750–754 the paragraph that explains the page: a condition page is one written article plus four things that are computed rather than typed — what else it could be, who has had it, what treats it, and where to get it. A summary on its own is a paragraph; a summary with the differentials and the pharmacy counter attached is something a person can act on. 755 blank.

756–757

```
def article_for(disease_id):
    """The written article, or None if this condition has not been written up."""
```

txt

<code>article_for(disease_id)</code> and its one-line docstring: the written article, or None if this condition has not been written up.

758–758

```
    record = store.get_by_id("disease_articles", disease_id)
```

L4

<code>record = store.get_by_id("disease_articles", disease_id)</code> — the articles registry, still a registry.

759–761

```
    if record is None:
        return None
```

L1

760–761: <code>if record is None: return None</code> — the honest None; the route will decide the status code, not the helper. 759 blank.

762–765

```
    return {
        "disease_id": record["disease_id"],
        "summary": record["summary"],
```

L3

763–765: the article dict opens — <code>disease_id</code> and <code>summary</code> copied as plain values (P3 dict building). 762 blank.

766–768

```
        # The columns are pipe delimited and empty means none, not the word
        # none. Saying "Red flags: none" reads like reassurance; leaving the
        # list empty reads like the absence of a list.
```

txt

The comment (766–768): the columns are pipe-delimited and empty means none, not the word none — saying "Red flags: none" reads like reassurance; leaving the list empty reads like the absence of a list. The next three lines exist because of it.

769–771	`"red_flags": [f for f in record["red_flags"].split("|") if f],`
	`"refer_when": [f for f in record["refer_when"].split("|") if f],`
	`"medicine_ids": [m for m in record["medicine_ids"].split("|") if m],`
		L3	769–771: three list comprehensions (P3) turn the pipe columns into real lists — <code>[f for f in record["red_flags"].split("|") if f]</code>, and the same for <code>refer_when</code> and <code>medicine_ids</code>: the trailing <code>if f</code> drops the empty cells the comment warns about.

772–773	`"provenance": record["provenance"],` `}`	L3	772–773: <code>"provenance"</code> recorded verbatim, then the dict closes. The article is now pure serviceable data.
774–775		txt	Blank lines.

776–781	`def condition_detail(disease_id):`
	`"""One condition with everything a reader needs to act on it.`
	
	`Returns None for an unknown id so the route can decide the status code,`
	`rather than the helper returning a 404-shaped body.`
	`"""`
		txt	<code>condition_detail(disease_id)</code> and its docstring (777–781): one condition with everything a reader needs to act on it — and it returns None for an unknown id so the route can decide the status code, rather than the helper returning a 404-shaped body. Helper/route separation, stated.

782–782	`disease = store.get_by_id("diseases", disease_id)`	L4	<code>disease = store.get_by_id("diseases", disease_id)</code> — the triage knowledge base row.
783–785	`if disease is None:` `return None`	L1	784–785: <code>if disease is None: return None</code> — the sentinel return again. 783 blank.

786–790	`enriched = dict(disease)`
	`enriched["symptom_set"] = sorted(`
	`triage.normalise_symptoms(disease["symptoms"]))`
	`enriched["article"] = article_for(disease_id)`
		L4	787–789: <code>enriched = dict(disease)</code> — a copy, so the CSV row is never mutated (P3 dict constructor as copy); <code>symptom_set</code> via <code>sorted(triage.normalise_symptoms(...))</code>; 789 <code>article = article_for(disease_id)</code> — both L4 delegations. 786 blank.

791–795	`# What else it could be, from the same set algebra the triage engine`
	`# runs. The condition's own symptom list is used as the query, so the`
	`# answer is "conditions whose symptoms overlap most with this one",`
	`# which is the question a reader actually has after reading the summary.`
		txt	The differentials comment (792–795): what else it could be, from the same set algebra the triage engine runs — the condition's own symptom list is used as the query, so the answer is "conditions whose symptoms overlap most with this one", which is the question a reader actually has after reading the summary. 791 blank.

796–796	`differentials = []`	L1	<code>differentials = []</code> — the accumulator.

797–801	`    for other in triage.rank_diseases(`
	`            disease["symptoms"], limit=triage.MAX_RESULTS + 1):`
	`        if other["disease_id"] != disease_id:`
	`            differentials.append(other)`
		L4	798–801: loop <code>triage.rank_diseases(disease["symptoms"], limit=triage.MAX_RESULTS + 1)</code> — one extra, because the condition itself will appear and is skipped by <code>if other["disease_id"] != disease_id</code> (P1 inequality). L4 delegation driving a filter.

802–803	`    enriched["differentials"] = differentials[:6]`	L2	803: <code>enriched["differentials"] = differentials[:6]</code> — the slice (P2) that keeps the response at a page-sized six. 802 blank.

804–809	`    # Where to get the treatment. The article carries explicit catalogue ids`
	`    # rather than prose that has to be matched, so this is a lookup and not a`
	`    # guess. Most conditions have none, because the catalogue holds twenty`
	`    # medicines and the registry covers ninety-six conditions, and the page`
	`    # says the clinic does not dispense it rather than implying otherwise.`
		txt	The medicines comment (805–809): where to get the treatment — the article carries explicit catalogue ids rather than prose that has to be matched, so this is a lookup and not a guess. Most conditions have none, because the catalogue holds twenty medicines and the registry covers ninety-six conditions, and the page says the clinic does not dispense it rather than implying otherwise. 804 blank.

810–811	`    medicines, _ = store.read_all("medicines")` `    stores, _ = store.read_all("stores")`	L4	810–811: read the <code>medicines</code> and <code>stores</code> tables — the two inputs the lookup needs.
812–812	`    by_id = {m["medicine_id"]: m for m in medicines}`	L3	<code>by_id = {m["medicine_id"]: m for m in medicines}</code> — the index comprehension, fourth sighting (P3).
813–814	`    ids = (enriched.get("article") or {}).get("medicine_ids") or []`	L3	814: <code>ids = (enriched.get("article") or {}).get("medicine_ids") or []</code> — the chained-or idiom (P1): article may be None, medicine_ids may be empty — either way the expression bottoms out at an empty list. 813 blank.
815–815	`    counters = [by_id[i] for i in ids if i in by_id]`	L3	<code>counters = [by_id[i] for i in ids if i in by_id]</code> — the comprehension with an in-dict filter (P3): a stale id in the article silently drops instead of KeyError-ing.
816–818	`    enriched["medicines"] = counters` `    enriched["in_catalogue"] = bool(counters)`	L3	817–818: <code>enriched["medicines"] = counters</code> and <code>enriched["in_catalogue"] = bool(counters)</code> — the bool() cast (P1) turns an empty list into an explicit False for the UI. 816 blank.

819–822	`    if counters:`
	`        enriched["medicine_id"] = counters[0]["medicine_id"]`
	`        enriched["medicine"] = counters[0]`
		L1	820–822: <code>if counters:</code> — truthiness (P1) — then <code>counters[0]</code> promoted to top-level <code>medicine</code> and <code>medicine_id</code> keys: the first one is the headline treatment. 819 blank.

823–829	`    stocked_ids = set()`
	
	`    for counter in counters:`
	`        for shop in registry.stores_with_medicine(counter["name"], medicines,`
	
	`stores):`
	`            stocked_ids.add(shop["store_id"])`
		L3	824–829: <code>stocked_ids = set()</code> — the set accumulator (P3) — then a nested loop: for each counter medicine, for each shop <code>registry.stores_with_medicine(...)</code> returns (L4 delegation), <code>stocked_ids.add(shop["store_id"])</code>. A set because a store stocking three of the medicines must appear once. 823 blank.

830–831	`    pharmacies = [shop for shop in stores if shop["store_id"] in stocked_ids]`	L3	831: <code>pharmacies = [shop for shop in stores if shop["store_id"] in stocked_ids]</code> — the comprehension filtered by set membership (P3), the same store list re-derived in the original order.
832–834	`    enriched["pharmacies"] = pharmacies` `    enriched["pharmacy_count"] = len(pharmacies)`	L3	833–834: <code>pharmacies</code> attached, <code>pharmacy_count = len(pharmacies)</code>. 832 blank.
835–837	`    # Who has had it. A count, not a list of names: this is a front desk,` `    # and the useful question is "is this common here", not "who".`	txt	The case-history comment (836–837): who has had it — a count, not a list of names: this is a front desk, and the useful question is "is this common here", not "who". 835 blank.
838–839	`    visits, _ = store.read_all("visits")` `    seen = [v for v in visits if v["diagnosis"] == disease["name"]]`	L3	838–839: read <code>visits</code>, then <code>seen = [v for v in visits if v["diagnosis"] == disease["name"]]</code> — the filter comprehension (P3), an exact string match on the recorded diagnosis.
840–841	`    enriched["case_count"] = len(seen)`	L1	841: <code>case_count = len(seen)</code> — the count from the comment. 840 blank.
842–843	`    enriched["last_seen"] = max((v["scheduled_date"] for v in seen), default="")` `    enriched["severity_seen"] = sorted({v["severity"] for v in seen if v["severity"]})`	L3	842: <code>last_seen = max((v["scheduled_date"] for v in seen), default="")</code> — max over a generator with a default, so zero cases yield an empty string instead of a ValueError; 843: <code>sorted({v["severity"] for v in seen if v["severity"]})</code> — a set comprehension (dedupe) inside sorted (P3 twice over).
844–847	`    return enriched`	L4	844: <code>return enriched</code> — one dict, every question answered. 845–847 blanks.

848–850	`@app.get("/api/diseases/<disease_id>")`
	`def get_disease(disease_id):`
	`    """One condition, as a readable article with live cross-links."""`
		txt	<code>@app.get("/api/diseases/<disease_id>")</code> / <code>get_disease</code> (848–850): docstring — one condition, as a readable article with live cross-links.

851–851	`    detail = condition_detail(disease_id)`	L4	<code>detail = condition_detail(disease_id)</code> — the whole helper above, called.
852–854	`    if detail is None:` `        return error(f"no condition with id {disease_id}", 404)`	X:flask	853–854: None → the shaped 404 with the id in the message (flask.md) — the exact split of duties the docstring promised. 852 blank.
855–858	`    return jsonify({"condition": detail})`	X:flask	856: <code>return jsonify({"condition": detail})</code> — one key, everything inside it (flask.md). 855 blank; 857–858 blanks.

859–868	`@app.get("/api/diseases")`
	`def list_diseases():`
	`    """Return one page of the triage knowledge base.`
	
	`    This endpoint used to hand back the whole 96-condition list in a body`
	`    with no total, offset or has_more. The shared browse component reads`
	`    those fields to state its count line, so the conditions view crashed`
	`    formatting a number that was never sent. Paged like every other`
	`    registry now, through the same helper.`
	`    """`
		txt	<code>@app.get("/api/diseases")</code> / <code>list_diseases</code> (859–860) with a docstring (861–868) that documents its own bug: this endpoint used to hand back the whole 96-condition list with no total, offset or has_more, and the shared browse component — which reads those fields for its count line — crashed formatting a number that was never sent. Now paged like every other registry, through the same helper.

869–869	`    return registry_page("diseases", "diseases", after=attach_symptom_sets)`	L4	<code>return registry_page("diseases", "diseases", after=attach_symptom_sets)</code> — the fix itself: the shared helper plus the symptom-set hook.
870–871		txt	Blank lines.

872–879	`@app.post("/api/triage")`
	`def run_triage():`
	`    """Rank diseases for a set of reported symptoms.`
	
	`    Requires at least one symptom. Optional conditions trigger`
	`    contraindication warnings, and an optional age adds a range note.`
	`    Neither is an error: the result is still a 200 with warnings.`
	`    """`
		txt	<code>@app.post("/api/triage")</code> / <code>run_triage</code> with its docstring (874–879): rank diseases for a set of reported symptoms. Requires at least one symptom; optional conditions trigger contraindication warnings and an optional age adds a range note — neither is an error, the result is still a 200 with warnings.

880–881	`    body = payload()` `    symptoms = body.get("symptoms", [])`	X:flask	880–881: <code>body = payload()</code> then <code>symptoms = body.get("symptoms", [])</code> — the request body again, with the get-default (flask.md + P3).

882–885	`    # normalise_symptoms already splits a pipe or comma delimited string,`
	`    # so it must be passed straight through. Wrapping it in a list first`
	`    # would turn the whole string into one bogus token.`
		txt	The comment (883–885): normalise_symptoms already splits a pipe or comma delimited string, so it must be passed straight through — wrapping it in a list first would turn the whole string into one bogus token. Knowledge about the callee, written where the caller needs it.

886–888	`    if not triage.normalise_symptoms(symptoms):`
	`        return error("at least one symptom is required",`
	`                     required_field="symptoms")`
		X:flask	886–888: <code>if not triage.normalise_symptoms(symptoms): return error("at least one symptom is required", required_field="symptoms")</code> — an empty normalised set (after splitting, casefolding and dropping blanks) is the guard; the error names the field. flask.md.

889–891	`    conditions = body.get("conditions", []) or []` `    age = body.get("age")`	**L1**	890–891: <code>conditions = body.get("conditions", []) or []</code> — the get-default and the or-idiom stacked, because a null in JSON must read as an empty list too; <code>age = body.get("age")</code> stays None when absent. 889 blank.
892–893	`    results = triage.rank_diseases(symptoms, conditions=conditions, age=age)`	**L4**	893: <code>results = triage.rank_diseases(symptoms, conditions=conditions, age=age)</code> — the whole triage engine in one delegation call. 892 blank.
894–896	`    patient_ref = body.get("patient_id", "")` `    patient = {}`	**L1**	895–896: <code>patient_ref = body.get("patient_id", "")</code> and <code>patient = {}</code> — an optional attachment, defaulting to an empty dict the reply can still render. 894 blank.

897–901	`    if patient_ref:`
	`        found = store.get_by_id("patients", patient_ref)`
	`        if found is not None:`
	`            patient = found`
		L4	898–901: <code>if patient_ref:</code> truthiness, then <code>found = store.get_by_id("patients", patient_ref)</code> and <code>if found is not None: patient = found</code> — an unknown patient_id is not an error here, the triage simply runs anonymous. 897 blank.

902–903	`    warnings = [w for r in results for w in r["warnings"]]`	**L3**	903: <code>warnings = [w for r in results for w in r["warnings"]]</code> — the double-for comprehension (P3) that flattens per-disease warning lists into one. 902 blank.

904–913	`    return jsonify({`
	`        "count": len(results),`
	`        "results": results,`
	`        "warnings": warnings,`
	`        "patient": patient,`
	`        "report": triage.render_report(patient or {"name": "anonymous"}, results, conditions),`
	`        "disclaimer": triage.DISCLAIMER,`
	`    })`
		X:flask	905–913: the reply (flask.md) — count, results, the flattened warnings, whatever <code>patient</code> was resolved, and 910–911 <code>triage.render_report(patient or {"name": "anonymous"}, results, conditions)</code> (the or-idiom feeding an always-renderable patient), finished with <code>triage.DISCLAIMER</code> — the Lab 5 banner rule that the disclaimer is the final word.

914–917	`# ----------------------------------------------------------------------- analytics`	**txt**	914–915 blanks; 916 the analytics section banner; 917 blank.

918–922	`@app.get("/api/analytics")`
	`def list_analytics():`
	`    """Return the recorded profile of every dataset."""`
	`    return jsonify({"datasets": analytics.list_datasets(),`
	`"passing": analytics.passing_datasets()})`
		X:flask	918–920 the catalog route header — <code>@app.get("/api/analytics")</code>, handler, docstring — 921–922 <code>jsonify({"datasets": analytics.list_datasets(), "passing": analytics.passing_datasets()})</code>, two L4 delegation calls in one reply (flask.md).

923–927	`@app.get("/api/analytics/risk-bands")`
	`def list_risk_band_options():`
	`    """Return which dataset columns have risk bands defined."""`
		X:flask	923–924 blanks; 925–927 <code>@app.get("/api/analytics/risk-bands")</code> / <code>list_risk_band_options</code> and its docstring: which dataset columns have risk bands defined — the picker data.

928–931	`    options = {}`
	
	`    for name in analytics.passing_datasets():`
	`        options[name] = analytics.available_risk_bands(name)`
		L4	928–931: <code>options = {}</code> then per passing dataset <code>options[name] = analytics.available_risk_bands(name)</code> — a loop resting entirely on analytics delegation (L4).

932–933	`    return jsonify({"options": options})`	**X:flask**	933: <code>return jsonify({"options": options})</code> (flask.md). 932 blank.
934–937	`def dataset_or_404(name):` `    """Return the dataset name if valid, or a 404 response tuple."""`	**txt**	934–935 blanks; 936–937 <code>dataset_or_404(name)</code> and its docstring: return the dataset name if valid, or a 404 response tuple.

938–940	`    if name not in analytics.passing_datasets():`
	`        return None, error(f"unknown dataset {name}", 404,`
	
	`available=analytics.passing_datasets())`
		X:flask	938–940: <code>if name not in analytics.passing_datasets(): return None, error(f"unknown dataset {name}", 404, available=analytics.passing_datasets())</code> — the membership test (P3) and the error carrying the list of valid datasets (flask.md).

941–942	`    return name, None`	**L4**	941: <code>return name, None</code> — the twin-return (value, failure) pattern (P4 functions): callers unpack and route on whichever slot is filled. 942 blank.

943–946	`@app.get("/api/analytics/<name>")`
	`def dataset_profile(name):`
	`    """Return describe, groupby and value_counts for one dataset."""`
		txt	943 blank; 944–946 <code>@app.get("/api/analytics/<name>")</code> / <code>dataset_profile</code> with its docstring: describe, groupby and value_counts for one dataset.

947–947	`    valid, failure = dataset_or_404(name)`	**L4**	<code>valid, failure = dataset_or_404(name)</code> — the twin unpack.
948–950	`    if failure:` `        return failure`	**X:flask**	949–950: <code>if failure: return failure</code> — the helper's 404 shape passes through untouched (flask.md). 948 blank.
951–952	`    return jsonify(analytics.profile(valid))`	**X:flask**	952: <code>return jsonify(analytics.profile(valid))</code> — the assembled profile straight out of the analytics layer (flask.md). 951 blank.

953–960	`@app.get("/api/analytics/<name>/risk-bands")`
	`def dataset_risk_bands(name):`
	`    """Return risk band distributions for a dataset.`
	
	`    With no column, returns every column that has thresholds defined.`
	`    """`
		txt	953–954 blanks; 955–960 <code>@app.get("/api/analytics/<name>/risk-bands")</code> / <code>dataset_risk_bands</code> and its docstring — risk band distributions for a dataset; with no column, returns every column that has thresholds defined.

961–961	`    valid, failure = dataset_or_404(name)`	**L4**	<code>valid, failure = dataset_or_404(name)</code> again.
962–964	`    if failure:` `        return failure`	**X:flask**	963–964: the pass-through, as above. 962 blank.
965–966	`    column = request.args.get("column")`	**X:flask**	966: <code>column = request.args.get("column")</code> — None means "all of them" (flask.md). 965 blank.

967–969	`    if not column:` `        available = analytics.available_risk_bands(valid)`	**L4**	968–969: <code>if not column:</code> — truthiness branch (P1) — <code>available = analytics.available_risk_bands(valid)</code>. 967 blank.
970–972	`        if not available:` `            return error(f"{valid} has no risk band columns defined", 404)`	**X:flask**	971–972: no columns defined → 404 <code>f"{valid} has no risk band columns defined"</code> (flask.md). 970 blank.

973–976	`        return jsonify({"dataset": valid,`
	`"columns": {c: analytics.risk_bands(valid, c)`
	`for c in available}})`
		X:flask	974–976: all defined columns at once — <code>"columns": {c: analytics.risk_bands(valid, c) for c in available}</code>, a dict comprehension (P3) directly inside the jsonify (flask.md). 973 blank.

977–979	`    try:` `        return jsonify(analytics.risk_bands(valid, column))`	**X:flask**	978–979: <code>try: return jsonify(analytics.risk_bands(valid, column))</code> — the single-column answer (flask.md). 977 blank.

980–982	`    except KeyError as e:`
	`        return error(str(e), 404,`
	`available=analytics.available_risk_bands(valid))`
		X:flask	980–982: <code>except KeyError as e: return error(str(e), 404, available=analytics.available_risk_bands(valid))</code> — the analytics layer's KeyError becomes a 404 carrying an 'available' list; the two layers agree on the contract (flask.md).

983–987	`@app.get("/api/analytics/<name>/charts")`
	`def dataset_charts(name):`
	`    """Render charts and return the filenames written."""`
		txt	983–984 blanks; 985–987 <code>@app.get("/api/analytics/<name>/charts")</code> / <code>dataset_charts</code> and its docstring: render charts and return the filenames written.

988–988	`    valid, failure = dataset_or_404(name)`	**L4**	<code>valid, failure = dataset_or_404(name)</code>, third use.
989–991	`    if failure:` `        return failure`	**X:flask**	990–991: the pass-through. 989 blank.
992–993	`    written = analytics.save_charts(valid)`	**L4**	993: <code>written = analytics.save_charts(valid)</code> — renders the three standard figures. 992 blank.
994–994	`    return jsonify({"dataset": valid, "charts": written})`	**X:flask**	<code>return jsonify({"dataset": valid, "charts": written})</code> (flask.md).

995–999	`@app.get("/charts/<path:filename>")`
	`def serve_chart(filename):`
	`    """Serve a generated chart PNG."""`
		txt	995–996 blanks; 997–999 <code>@app.get("/charts/<path:filename>")</code> / <code>serve_chart</code> and its docstring: serve a generated chart PNG.

1000–1001	`    directory = (os.environ.get("MERIDIAN_CHART_DIR")` `                 or os.path.join(os.getcwd(), "static", "charts"))`	**std**	1000–1001: <code>directory = os.environ.get("MERIDIAN_CHART_DIR") or os.path.join(os.getcwd(), "static", "charts")</code> — the Vercel override pointing at the writable copy, or the local default; the or-idiom (P1) woven around a P4 path join.
1002–1003	`    if not os.path.exists(os.path.join(directory, filename)):`	**L4**	1003: <code>if not os.path.exists(os.path.join(directory, filename)):</code> — existence before serving (the P4 check). 1002 blank.
1004–1004	`        return error(f"no chart named {filename}", 404)`	**X:flask**	<code>return error(f"no chart named {filename}", 404)</code> — the honest miss (flask.md).

1005–1006	`    return send_from_directory(directory, filename)`	**X:flask**	1006: <code>return send_from_directory(directory, filename)</code> — the PNG goes out. 1005 blank.
1007–1010	`# ---------- graphs`	**txt**	1007–1008 blanks; 1009 the graphs section banner; 1010 blank.

1011–1018	`@app.get("/api/graphs")`
	`def graph_catalog():`
	`    """Return the graph kinds and, per dataset, the columns they take.`
	
	`    The browser builds its drop-downs from this one call instead of`
	`    guessing column names, so a kind the dataset cannot feed is never`
	`    offered in the first place.`
	`    """`
		txt	<code>@app.get("/api/graphs")</code> / <code>graph_catalog</code> with its docstring (1013–1018): the graph kinds and, per dataset, the columns they take. The browser builds its drop-downs from this one call instead of guessing column names, so a kind the dataset cannot feed is never offered in the first place.

1019–1023	`    return jsonify({`
	`        "kinds": analytics.graph_kinds(),`
	`        "datasets": {name: analytics.graph_columns(name)`
	
	`for name in analytics.passing_datasets()},`
	`    })`
		L7	1019–1023: <code>"kinds": analytics.graph_kinds()</code> and <code>"datasets": {name: analytics.graph_columns(name) for name in analytics.passing_datasets()}</code> — a dict comprehension (P3) over the catalog, the graph feature's menu data in one reply (flask.md carries the envelope).

1024–1025		**txt**	Blank lines.

1026–1033	`@app.get("/api/graphs/<name>/<kind>")`
	`def build_graph(name, kind):`
	`    """Render one graph of <kind> from dataset <name>.`
	
	`    Columns arrive as ?x= and, for scatter and line, ?y=. A bad kind or an`
	`    unsuitable column is a 400 whose message names what would have worked,`
	`    following the same contract as the risk-bands route.`
	`    """`
		txt	<code>@app.get("/api/graphs/<name>/<kind>")</code> / <code>build_graph</code> with its docstring (1028–1033): render one graph of the chosen kind from a dataset. Columns arrive as <code>?x=</code> and, for scatter and line, <code>?y=</code>. A bad kind or an unsuitable column is a 400 whose message names what would have worked, following the same contract as the risk-bands route.

1034–1034	`    valid, failure = dataset_or_404(name)`	**L4**	<code>valid, failure = dataset_or_404(name)</code> — the twin unpack, fourth use.
1035–1037	`    if failure:` `        return failure`	**X:flask**	1036–1037: the pass-through of the 404. 1035 blank.

1038–1042	`    try:`
	`        built = analytics.render_graph(valid, kind,`
	`x=request.args.get("x"),`
	`y=request.args.get("y"))`
		L7	1039–1042: inside <code>try:</code> — <code>built = analytics.render_graph(valid, kind, x=request.args.get("x"), y=request.args.get("y"))</code> — the renderer called with whatever query parameters arrived (the request reads themselves are flask.md). 1038 blank.

1043–1046	`    except KeyError as e:`
	`        return error(str(e), 400,`
	`available=[{"kind": k["kind"], "label": k["label"]}`
	`for k in analytics.graph_kinds()])`
		X:flask	1043–1046: <code>except KeyError as e:</code> — unknown dataset or kind — <code>return error(str(e), 400, available=[{"kind": k["kind"], "label": k["label"]} for k in analytics.graph_kinds()])</code> — a 400 that teaches the client the full kind list (flask.md).

1047–1049	`    except ValueError as e:`
	`        columns = analytics.graph_columns(valid)`
	`        return error(str(e), 400, columns=columns)`
		X:flask	1047–1049: <code>except ValueError as e:</code> — wrong-shaped columns — and the 400 carries <code>columns=analytics.graph_columns(valid)</code>, what this dataset could feed instead (flask.md).

1050–1051	`    return jsonify({"graph": built})`	**X:flask**	1051: <code>return jsonify({"graph": built})</code> — the metadata dict straight back. 1050 blank.
1052–1055	`# ---------- backup`	**txt**	1052–1053 blanks; 1054 the backup section banner; 1055 blank.

1056–1058	`@app.post("/api/backup")`
	`def run_backup():`
	`    """Copy the five registry CSVs into a timestamped folder."""`
		txt	<code>@app.post("/api/backup")</code> / <code>run_backup</code> with its docstring (1058): copy the five registry CSVs into a timestamped folder.

1059–1059	`    written = store.backup_all()`	**L4**	<code>written = store.backup_all()</code> — the whole backup is one store delegation.
1060–1062	`    return jsonify({"count": len(written), "files": written})`	**X:flask**	1060: <code>return jsonify({"count": len(written), "files": written})</code> — the manifest, not just a boolean (flask.md). 1061–1062 blanks.
1063–1064	`if __name__ == "__main__":` `    app.run(host="127.0.0.1", port=8000, debug=True)`	**std**	1063–1064: <code>if __name__ == "__main__": app.run(host="127.0.0.1", port=8000, debug=True)</code> — the local dev entrypoint (loopback only), with <code>debug=True</code>'s auto-reloader; on Vercel the app object is imported by api/index.py instead of run, which is why this block stays out of the way.